Pearls AQI Predictor

A Serverless, Self-Monitoring Machine-Learning System

for Three-Day Air-Quality Forecasting Across Pakistan

An end-to-end Feature-Training-Inference pipeline that forecasts the Air Quality Index up to 72 hours ahead for 22 cities, retrains and validates itself automatically, explains every prediction, and monitors its own data drift and forecast error.

SUBMITTED BY

Hasnain Irshad

Project Thesis

10Pearls Data Science Internship

2026

Abstract

Pakistan contains some of the most polluted urban air on earth, yet the air-quality information available to its residents is almost entirely a *nowcast*: a single number describing the air as it is at this moment, for one monitoring site, with no explanation of why and no indication of what it will be tomorrow. The decisions that actually protect health - whether to keep a child indoors, whether to reschedule outdoor work, whether an asthmatic should carry a mask - require lead time, and therefore require a forecast. A forecast, in turn, is not a model trained once: air quality is strongly seasonal, so a static model decays silently. What is needed is not a model but a system that keeps itself current.

This thesis presents the Pearls AQI Predictor, an end-to-end, effectively serverless machine-learning system that forecasts the Air Quality Index up to 72 hours ahead for 22 cities spanning every province of Pakistan as well as the Islamabad Capital Territory, Gilgit-Baltistan and Azad Jammu & Kashmir. It is built as three decoupled Feature-Training-Inference pipelines that communicate only through a managed feature store and model registry. An hourly feature pipeline ingests weather and pollutant observations and engineers a leakage-controlled feature table; a daily training pipeline rebuilds a multi-horizon supervised dataset, compares four model families, and admits a new model only through a champion-challenger promotion gate; and a batch inference pipeline loads the reigning champion and writes the 72-hour forecast curve, with calibrated prediction intervals, for every city. All computation runs on free scheduled infrastructure, so the running cost is negligible.

The contribution is the operational layer around the model. Accuracy is reported per forecast horizon and under a rolling walk-forward backtest rather than as one averaged number. The champion gradient-boosted model attains a validation RMSE of 19.69, MAE of 12.80 and R^2 of 0.85, beating a persistence baseline by 22% overall and at every horizon from two hours to three days, with a mean backtest RMSE of 20.55. Every forecast carries a plain-language explanation derived from SHAP attributions; a what-if simulator lets a user perturb the drivers and watch the model respond; and the system monitors itself for feature drift and scores its own past forecasts against the air quality that actually arrived. The results are delivered through a FastAPI service and a React dashboard deployed to the public internet, together with a Model Context Protocol server exposing the same grounded tools to a language-model advisor.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Brief

Air pollution is the environmental risk factor that shortens the most lives, and South Asia carries a disproportionate share of that burden. In Pakistan, the winter months bring a persistent smog season in which the Air Quality Index (AQI) in the large cities of the Punjab plain routinely reaches values that public health agencies classify as unhealthy or worse for days at a time. The people most affected by it - children, the elderly, and anyone with a respiratory or cardiac condition - have very little information with which to protect themselves. What information does exist is almost invariably a *nowcast*: a single number describing the air as it is at this moment, at one monitoring location, offered without explanation and without any statement of what the air will be like tomorrow.

That is the wrong shape of information for the decisions people actually make. Whether to send a child to school on foot, whether to move an outdoor shift indoors, whether an asthmatic should carry a mask or reschedule an errand: these are decisions taken hours or days in advance, and they need a forecast, not a reading. A number that tells someone the air is dangerous *now* arrives too late to be acted upon.

This thesis presents the Pearls AQI Predictor, an end-to-end machine-learning system that forecasts the United States Environmental Protection Agency AQI up to 72 hours ahead for 22 cities spanning every province of Pakistan together with the Islamabad Capital Territory, Gilgit-Baltistan and Azad Jammu & Kashmir. The system is not a model in a notebook. It ingests fresh observations every hour, retrains itself every night, refuses to deploy a new model that is not measurably better than the one already serving, explains every forecast it makes, watches its own inputs for distributional drift, scores its own past forecasts against the air quality that actually arrived, and serves all of this through a public web dashboard and a documented programming interface. It does all of this on free scheduled infrastructure, at an ongoing compute cost of effectively zero. Figure 1.1 shows the system at a glance.

Figure 1.1 divides the system into three regions. On the left, a single free and keyless data provider supplies both pollutant and weather observations, and also the weather *forecast* that the inference stage needs. In the centre, a compute plane of scheduled, disposable jobs runs the three pipelines that give the architecture its name. On the right,

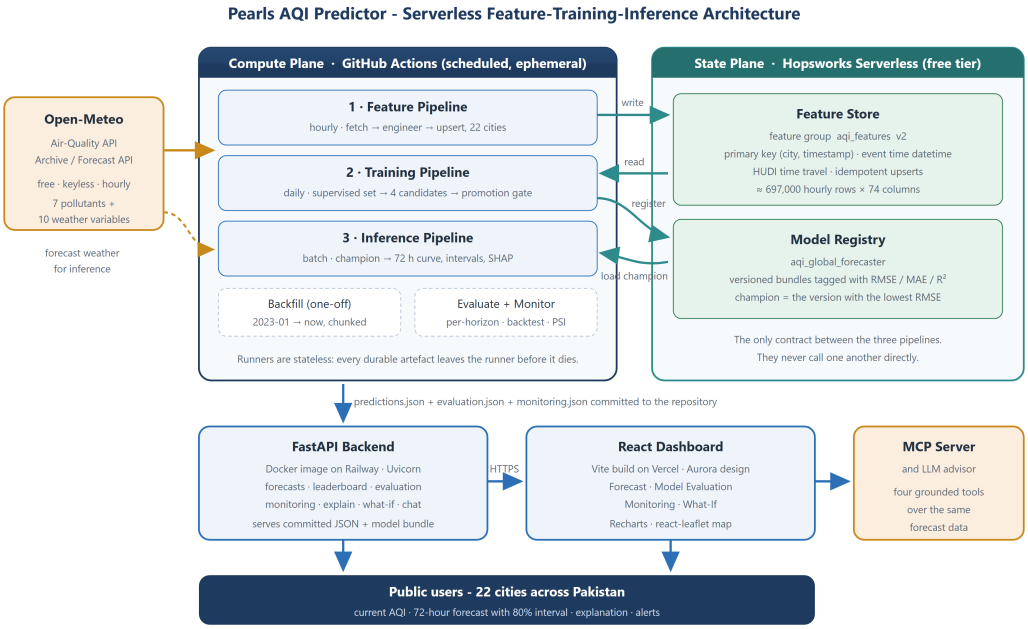

Figure 1.1: High-level architecture of the Pearls AQI Predictor: three decoupled pipelines on a scheduled compute plane, communicating only through a managed feature store and model registry, with the results served by a container-hosted API and a static dashboard.

a managed feature store and model registry hold every piece of durable state, so that no pipeline depends on any other pipeline still being alive. Beneath them, an application programming interface and a browser application deliver the result to the public. The remainder of this thesis develops each of these components in detail.

1.2 Relevance to Course Modules

The project is deliberately broad and exercises the core of a computer science and data science curriculum. Table 1.1 maps the principal concepts the project applies to the areas in which they are taught.

Table 1.1: Relevance of the project to the principal subject areas.

Module / Area	Concepts applied in this project
Machine Learning	Supervised regression, gradient-boosted trees, regularised linear models, bagged ensembles, hyper-parameter choice, bias-variance reasoning, baseline comparison.
Time-Series Analysis	Autocorrelation, lag and rolling features, seasonality, cyclical encodings, direct multi-horizon forecasting, chronological validation, walk-forward backtesting, persistence baselines.
Deep Learning	A recurrent (LSTM) sequence model trained and evaluated on the same task, and the reasoned decision not to promote it.

Module / Area	Concepts applied in this project
Data Engineering	Ingestion from a public API with retry and backoff, chunked historical backfill, idempotent upserts, primary keys and event time, a feature store and feature views, and a backend-agnostic storage facade.
Software Engineering	Layered package design, configuration as a single source of truth, dependency inversion at the storage boundary, unit testing, iterative lifecycle, and documentation.
MLOps and DevOps	Scheduled pipelines, continuous integration, a model registry, champion-challenger promotion, artefact versioning, containerisation, and production deployment.
Statistics	The Population Stability Index, empirical residual quantiles and split-conformal prediction intervals, correlation analysis, distributional comparison.
Explainable AI	Shapley additive explanations, global and per-instance attribution, and the translation of attributions into natural language.
Web Technologies	A typed asynchronous HTTP API, cross-origin policy, a single-page application, responsive charting, and interactive cartography.
Human-Computer Interaction	Information hierarchy on a public dashboard, colour semantics for a standardised health scale, uncertainty communication, and graceful degradation when a capability is unavailable.

1.3 Project Background

Air-quality forecasting has three broad traditions. The first is *deterministic chemical transport modelling*: physically simulating the emission, transport, chemistry and deposition of pollutants over a gridded domain. These models are the scientific gold standard and underpin national forecasting services, but they require an emissions inventory, meteorological boundary conditions and considerable computing capacity, none of which is available to an individual developer. The second is *classical statistical time-series modelling*, in which an autoregressive or moving-average model is fitted to the pollutant series for a single site. Such models are cheap and interpretable, but they capture the persistence of the series rather than its drivers, and they degrade quickly beyond a short lead time because they cannot use a weather forecast. The third, and the one this project belongs to, is *supervised machine learning*, in which a model learns the mapping from weather, calendar and recent pollution state to future pollutant concentration directly from historical data.

The machine-learning approach has one decisive practical advantage for the problem at hand: the most informative predictors of tomorrow's air quality - tomorrow's wind, temperature, humidity and rainfall - are themselves available as a free public forecast. Air quality is, to a first approximation, a story about dispersion. Wind clears pollutants; a temperature inversion traps them under a warm lid; rain scavenges particulates out of the air column. If the weather for the next three days can be obtained at no cost, then

a model that has learned how weather and pollution interact can produce a genuinely forward-looking forecast without any physical simulation at all.

What the machine-learning approach does not solve by itself is the problem of *staying* correct. A model trained on last year's data and left alone will decay, because the relationship it encodes is not stationary: seasons turn, emission patterns change, and the distribution of the inputs moves away from the distribution the model was fitted on. A great many student and research projects stop at a trained model and a reported accuracy figure. The interesting engineering problem - and the one this project treats as its centre of gravity - begins immediately afterwards. How does a model get retrained without a human present? How does one prevent a nightly retrain from silently shipping a worse model? How is the reigning model stored so that it survives the machine that trained it? How does the system notice that the world has changed? How does a user know why the system believes what it says? These are the questions of machine-learning operations, or MLOps, and they are what this thesis is principally about.

1.4 Motivation

Three observations motivated this project.

The first is a matter of public health. Twenty-two cities, spanning every administrative unit of the country, are covered by this system; between them they are home to a very large share of Pakistan's urban population. A forecast with three days of lead time changes what a household can do with the information. It converts a warning into a plan. The exploratory analysis presented in Chapter 5 shows exactly how large the effect being forecast is: the mean AQI across the whole record sits above 140 in five of the 22 cities studied, close to the threshold the EPA labels *unhealthy*, and the seasonal signal is pronounced: January averages 146.8 against 91.5 in April, the cleanest month. This is not a marginal environmental nuisance; it is a recurring, seasonal, predictable public-health event, and predictability is precisely what a forecasting system can exploit.

The second is a matter of engineering discipline. There is a large gap between a model that scores well on a held-out split and a system that keeps working unattended. Closing that gap normally implies infrastructure and therefore cost, which is why so much student work stops at the notebook. One goal of this project was to demonstrate that the entire lifecycle - scheduled ingestion, a feature store, automated retraining, a gated model registry, evaluation, drift monitoring and public deployment - can be assembled from free tiers and scheduled runners, and that doing so imposes real constraints which are themselves instructive. Some of the most valuable findings in this thesis, reported in Chapter 6, came from failures induced by exactly those constraints: an insert that hung because a free-tier executor stalled, a statistics profiler that exhausted its memory on a

74-column table, and a deployment tool that silently omitted the model file because the repository ignored it.

The third is a matter of trust. A forecast that cannot be interrogated is a forecast that will not be believed, particularly when it disagrees with what a person can see from their window. The system therefore never presents a bare number. Each forecast carries an 80% prediction interval, so the user can see how confident the model is; each carries a plain-language explanation generated from the model's own Shapley attributions, so the user can see what is driving it; and the model's complete promotion history, per-horizon accuracy, backtest results and current drift status are all published in the same interface, so the claim of accuracy can be checked rather than merely asserted.

1.5 Problem Statement

Existing air-quality information for Pakistan is current-only, sparse, unexplained and manually maintained, and the machine-learning work that addresses it typically stops short of an operating system. Official and commercial sources report the AQI as it stands at a monitoring station, which tells a user what has already happened rather than what is about to; where a forecast is offered it generally covers a single major city, so residents of the smaller cities in Khyber Pakhtunkhwa, Balochistan, Gilgit-Baltistan and Azad Jammu & Kashmir are served by nothing at all. No mainstream source explains its number, so a user cannot distinguish a reading driven by a genuine pollution episode from one driven by an unusual meteorological configuration, and none publishes its own error, so its reliability cannot be assessed.

The research literature closes part of this gap and leaves the rest open. Supervised models for pollutant forecasting are well established, and gradient-boosted trees in particular are repeatedly found competitive with, or superior to, deep sequence models on tabular, weather-driven environmental data. What is far less commonly addressed is the operational half of the problem. A published accuracy figure is usually computed on a single held-out split and reported as one average number, which conceals the fact that error grows steeply with lead time and thereby overstates usefulness at the horizons a user actually cares about. Retraining, if discussed at all, is assumed rather than implemented; promotion is unguarded, so a scheduled retrain can degrade a service without anyone noticing; the trained artefact is left on the machine that produced it, which in a serverless setting means it is destroyed minutes later; and drift, which is the specific mechanism by which such a system decays, is rarely instrumented.

There is therefore a need for a system that forecasts AQI several days ahead for many cities rather than one; that reports its accuracy honestly, per horizon and under rolling validation rather than a single split; that retrains itself without human intervention while refusing to promote a model that is not demonstrably better; that stores its champion

durably and independently of any single machine; that quantifies its own uncertainty and watches its own inputs and outputs for decay; that explains each individual prediction in language a non-specialist can act on; and that delivers all of this to the public reliably and at negligible cost. A fuller treatment of the problem, of the shortcomings of current systems and of the specific gaps this project addresses is given in Chapter 3.

1.6 Aims and Objectives

The overarching aim of the project is to design, implement, deploy and evaluate a serverless, self-monitoring machine-learning system that forecasts the Air Quality Index three days ahead for 22 Pakistani cities and explains its predictions. This aim is decomposed into the following objectives.

1. **Data and features.** To ingest weather and pollutant observations automatically from a free public source, and to engineer from them a rich but strictly leakage-controlled feature representation that generalises across many cities rather than being fitted to one.
2. **Serverless pipeline architecture.** To realise the system as three decoupled Feature, Training and Inference pipelines, orchestrated by a free scheduler and communicating only through a managed feature store and model registry, so that each may run, fail and be replaced independently.
3. **Accurate multi-horizon forecasting.** To train a single global model that serves every lead time from one hour to 72 hours by treating the horizon as an input feature, to measure it against a persistence baseline at every horizon rather than only on average, and to attach calibrated prediction intervals to every forecast it produces.
4. **MLOps discipline.** To implement a champion-challenger promotion gate backed by a persistent leaderboard and a durable versioned model registry, and to evaluate the model rigorously through per-horizon metrics and walk-forward backtesting rather than a single averaged score.
5. **A trust layer.** To explain each forecast through Shapley attributions rendered as plain language, to allow a user to interrogate the model through a what-if simulator, and to monitor the system continuously for input drift and realised forecast error.
6. **Delivery.** To serve the whole system publicly through a typed HTTP interface and an interactive dashboard, and to expose the same grounded data to a language-model advisor through a standard tool protocol.

1.7 Scope of the Project

The project delivers a complete, running system. In scope are: hourly ingestion and feature engineering for 22 cities; a full historical backfill from January 2023 to the

present; exploratory analysis of the resulting dataset; a daily training pipeline comparing four model families; a global multi-horizon forecaster covering lead times from one to 72 hours; empirical prediction intervals; a feature store and a model registry; champion-challenger promotion with a persistent leaderboard; per-horizon evaluation and walk-forward backtesting; Shapley-based explanation in natural language; a what-if simulator; drift and forecast-error monitoring; hazard alerting; a FastAPI backend; a React dashboard; a Model Context Protocol server and a language-model advisor; and live public deployment of the whole.

The following are explicitly out of scope. The project does not train a bespoke foundation model or perform physical chemical-transport simulation; it uses a hosted language model through its API for the advisor and a free public data service for its observations. It does not attempt street-level or sub-hourly resolution: the finest granularity of the data source, and therefore of the system, is one city and one hour. It does not guarantee accuracy during unprecedented events - an uncontrolled industrial fire, for example - that have no analogue in the training record. It does not offer a mobile application, and it does not use any paid data source. Finally, the system forecasts the AQI; it does not attribute pollution to sources, and the what-if simulator is explicitly presented as a model simulation rather than a causal claim.

1.8 Methodology and Software Lifecycle

The project followed an iterative and incremental lifecycle, organised as a sequence of modules, and illustrated in Figure 1.2. Each iteration delivered one independently runnable and independently testable capability, and each was allowed to inform the plan for the next. This was not merely a convenient way to organise the work; it was the only defensible one, because the behaviour of several components could be established only empirically. Whether a free-tier feature store could absorb a three-and-a-half-year backfill for 22 cities, whether a sequence model would beat a tree ensemble on this data, and whether a nightly retrain would in fact produce an improvement rather than noise were all questions that had to be answered by running the thing rather than by reasoning about it in advance.

1.8.1 Planning

Each iteration began by selecting the next module and identifying its principal risk. Risk drove ordering: the historical backfill, which depended on an external free-tier service absorbing hundreds of thousands of rows, was attempted early precisely because a failure there would have invalidated the architecture.

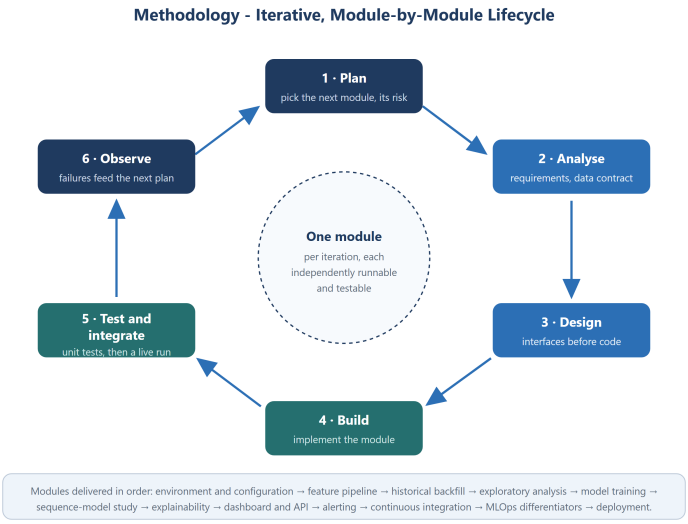

Figure 1.2: The iterative, module-by-module lifecycle followed during development. Each iteration delivers one working, testable increment, and each observed failure feeds directly into the next iteration's plan.

1.8.2 Analysis

Requirements for the increment were expressed as concrete scenarios - "a resident of Multan wants to know whether Thursday afternoon is safe for a child's football practice" - and reduced to functional and non-functional requirements, which Chapter 4 sets out in full.

1.8.3 Design

Interfaces were fixed before code was written, with particular care at the two boundaries that determine whether the architecture holds: the contract between a pipeline and the feature store, and the contract between a training run and the model registry. Chapter 5 documents these.

1.8.4 Implementation

Each module was implemented as a small, separately importable Python module with one responsibility, so that ingestion, feature construction, training, evaluation, explanation, monitoring and serving could each evolve without disturbing the others. Chapter 6 presents the implementation.

1.8.5 Testing and Integration

Each increment was covered by unit tests over its critical logic - most importantly the tests that assert the absence of target leakage - and then run end to end against live data on a scheduled runner before being considered complete. Chapter 8 reports the results.

1.8.6 Maintenance

Between iterations, the system was hardened in response to observed failures. The backfill gained non-blocking writes and a history-aware resume after it froze partway through a run; the training pipeline gained an explicit datetime coercion after the feature store returned its event-time column as strings; and the deployment gained explicit negations in the ignore file after the deployment tool silently omitted the model bundle.

1.8.7 System Development Lifecycle Summary

The iterative approach suited a project whose central components could only be judged by measurement. Each shortcoming observed in a real run became the first item of the next iteration's plan, which produced steady and demonstrable improvement - most visibly in the model itself, whose first champion was promoted at a validation RMSE of 20.60 and then displaced by a better one at 19.69 once the full historical record was available.

1.9 The Feature-Training-Inference Pattern in Brief

The architectural backbone of the system is the Feature-Training-Inference pattern, shown in Figure 1.3. Rather than a single program that loads data, trains a model and predicts, the work is divided into three pipelines that never call one another. The feature pipeline turns raw observations into features and writes them to a feature store. The training pipeline reads features from that store, produces a model and writes it to a model registry. The inference pipeline reads the model from that registry and produces forecasts. Each runs on its own schedule; each can fail, be re-run or be rewritten without the others being aware of it; and the store and the registry are the only contract between them.

This decoupling is what makes a serverless implementation possible. Because the only durable state lives in the feature store and the model registry, the compute itself can be entirely disposable: every pipeline runs on an ephemeral runner that is destroyed when the job finishes, and nothing of value is lost when it is.

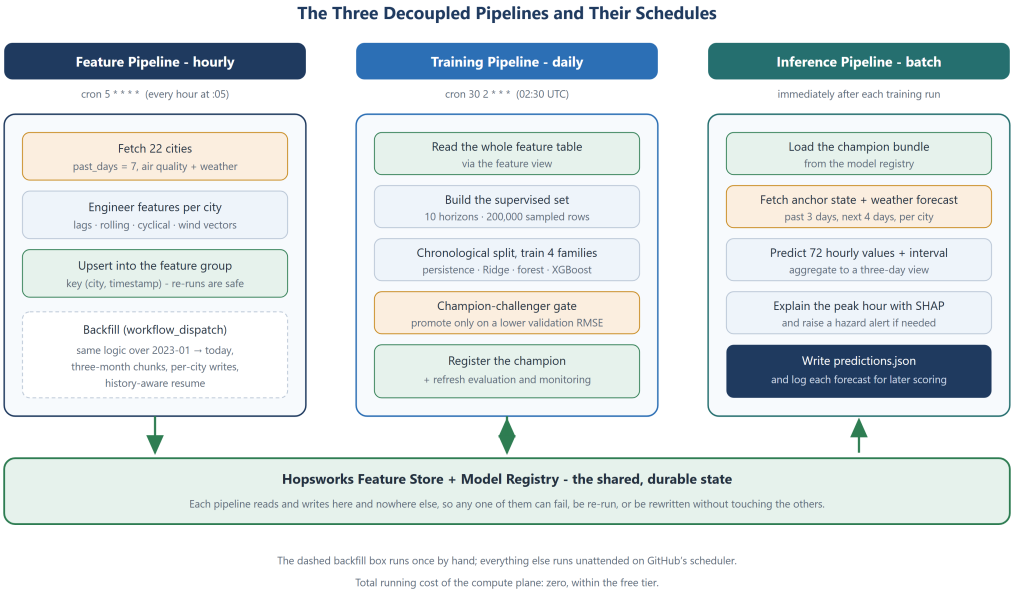

Figure 1.3: The three decoupled pipelines, their schedules and the shared state through which they communicate.

1.10 Significance of the Project

The significance of this work is threefold. *Practically*, it puts a three-day, explained, uncertainty-quantified air-quality forecast into the hands of residents of 22 Pakistani cities, most of which are served by no forecast at all, and it does so at a running cost that is effectively zero - which matters, because a public-interest service that is expensive to run is a service that stops running. *Technically*, it demonstrates that the full MLOps lifecycle - feature store, scheduled retraining, gated promotion, durable registry, rigorous evaluation and self-monitoring - can be implemented in its entirety on free infrastructure, and it documents the concrete constraints that such infrastructure imposes and how each was overcome. *Academically*, it provides a fully documented reference design for an operational forecasting system, including the components that are most often omitted from published work: the promotion gate, the per-horizon and walk-forward evaluation that reveals how skill actually decays with lead time, and the drift and realised-error monitoring that tells an operator when the system has stopped being trustworthy.

1.11 Thesis Organisation

The remainder of this thesis is organised as follows. Chapter 2 reviews related work in air-quality forecasting, feature stores and the Feature-Training-Inference pattern, MLOps practice, explainable machine learning and time-series validation. Chapter 3 defines the problem and the gaps in existing systems. Chapter 4 presents the requirement analysis.

Chapter 5 details the system design and architecture, and Chapter 6 its implementation. Chapter 7 describes the user interface. Chapter 8 reports the testing and the empirical evaluation. Chapter 9 concludes and outlines future work, and Chapter 10 collects the abbreviations and glossary, followed by the references.

1.12 Summary of Introduction

This chapter introduced the Pearls AQI Predictor and motivated it against the current state of air-quality information in Pakistan, which is a nowcast for a few cities rather than a forecast for many. It stated the problem, set out the aims, objectives and scope, described the iterative methodology by which the system was built, and previewed the Feature-Training-Inference pattern that gives the architecture its shape. The next chapter surveys the body of work on which the project builds.

Chapter 2

Literature Review

2.1 Introduction

This chapter surveys the work on which the Pearls AQI Predictor builds. The survey is organised around the two halves of the problem the system addresses. The first half is scientific: what the Air Quality Index measures, how air quality has been forecast, and which modelling families are appropriate for a tabular, weather-driven, multi-horizon regression problem. The second half is operational: how a trained model is kept alive in production, which is the concern of the literature on feature stores, machine-learning operations, distribution drift and model governance. A third, shorter strand covers explainability and the honest evaluation of forecasts, both of which the system treats as first-class requirements rather than afterthoughts. The chapter closes with a literature matrix, an analysis of the gap this project addresses, and a statement of how each body of work informed the design.

2.2 Air Quality, Health and the Air Quality Index

2.2.1 The Health Burden

The World Health Organization's global air-quality guidelines [1] substantially tightened the recommended limits for fine particulate matter in 2021, setting an annual mean guideline of 5 μg/m^3 for $PM_{2.5}$ and a 24-hour guideline of 15 μg/m^3, on the basis of accumulated evidence linking exposure to cardiovascular disease, respiratory disease, adverse birth outcomes and premature mortality. The State of Global Air report [2] places air pollution among the leading risk factors for death worldwide and identifies South Asia as the region carrying the heaviest exposure. Annual world air-quality reporting [3] has for several consecutive years ranked Pakistan among the most polluted countries measured, with several Punjabi cities appearing among the most polluted urban areas globally. A World Bank assessment of the economic cost of outdoor air pollution in Pakistan [4] quantified the burden in terms of health expenditure and lost productivity and argued for monitoring and forecasting capacity as a prerequisite for any policy response.

Two features of this burden are directly relevant to the design of a forecasting system. The first is that it is dominated by particulate matter rather than by gaseous pollutants,

which justifies the decision, described in Chapter 6, to compute the index from $PM_{2.5}$ and PM_{10} where the index has to be computed at all. The second is that it is strongly seasonal: the exposure is concentrated in a winter smog season driven by temperature inversions, crop residue burning and reduced mixing height. Seasonality of that strength is exactly what a model with calendar and weather features can learn, and exactly what makes a static model decay.

2.2.2 The Air Quality Index

The Air Quality Index used in this project is the United States Environmental Protection Agency index, defined in the agency's technical assistance document for daily air-quality reporting [5]. The index is not measured; it is computed. For a pollutant concentration C falling in the breakpoint interval $[C_{lo}, C_{hi}]$, whose corresponding index interval is $[I_{lo}, I_{hi}]$, the sub-index is the piecewise-linear interpolation

$$I = \frac{I_{hi} - I_{lo}}{C_{hi} - C_{lo}} (C - C_{lo}) + I_{lo}, \tag{2.1}$$

and the reported index is the *maximum* of the pollutant sub-indices, on the principle that health risk is governed by the worst offender rather than by an average. The index is divided into six named health categories, from *Good* to *Hazardous*, each carrying standard advisory language. This categorical structure matters for the interface: a user does not need to interpret the number 163, but does need to know that it means "unhealthy" and that outdoor exertion should be limited.

An important practical subtlety, and a common source of error in student implementations, is that the EPA defines gaseous breakpoints in parts per billion or parts per million while most data services report gas concentrations in micrograms per cubic metre. Applying the tables without a molar-mass conversion produces a wildly inflated sub-index. This project's treatment of that issue is described in Section 6.5.

2.3 Approaches to Air-Quality Forecasting

2.3.1 Deterministic Chemical Transport Modelling

The physically grounded approach simulates emission, advection, chemical transformation and deposition over a gridded domain. The Copernicus Atmosphere Monitoring Service [6] operates such a system globally and publishes composition forecasts, and regional implementations built on coupled meteorology-chemistry models are the basis of most national air-quality services. These models are indispensable for scientific attribution because they represent the physics explicitly. They are, however, impractical for a project of this kind: they require a maintained emissions inventory, meteorological

boundary conditions and substantial computing capacity, none of which is available for free. The present work consumes the *output* of such systems indirectly, since the reanalysis and forecast products it ingests are themselves derived from them.

2.3.2 Statistical Time-Series Modelling

The classical alternative fits an autoregressive integrated moving-average model, or a seasonal variant, to the pollutant series at a single site. Hyndman and Athanasopoulos [9] give the standard modern treatment. Such models are cheap, interpretable and often surprisingly hard to beat at very short lead times, because pollutant series are strongly autocorrelated. Their limitation is structural: a univariate model extrapolates the series' own history and cannot consume a weather forecast, so its skill decays towards the series mean as the horizon lengthens. This is precisely the regime in which the present system must operate, since a three-day forecast is dominated by the weather that has not yet happened rather than by the pollution that already has.

The same literature supplies the baseline against which any forecaster must be judged. The naive or *persistence* forecast - predicting that the value at $t + h$ equals the value at t - is the standard reference point, and Hyndman and Athanasopoulos are emphatic that a model failing to beat it has demonstrated nothing. This project therefore scores persistence explicitly in every training run and at every horizon.

2.3.3 Machine-Learning Regression

The dominant contemporary approach treats forecasting as supervised regression from engineered features to a future concentration. Yu et al. [19] demonstrated a random-forest approach to urban air-quality prediction from heterogeneous sensing data, and Pan [20] reported gradient-boosted trees performing strongly on hourly $PM_{2.5}$ prediction. The appeal of the approach for this problem is that the strongest predictors of future pollution - future wind, temperature, humidity and precipitation - are themselves available as a free forecast, so the model can be given genuine information about the period it is predicting rather than being restricted to extrapolating the past.

The two tree ensembles used here rest on well-established foundations. Random forests [14] reduce variance by averaging decorrelated trees grown on bootstrap samples with randomised feature subsets. Gradient boosting [15] instead fits trees sequentially to the residuals of the current ensemble, performing gradient descent in function space; XGBoost [16] is the scalable, regularised implementation of that idea that has become the default choice for tabular problems, and it is the champion model in this system. Ridge regression [13] is included as a regularised linear reference point, which is useful diagnostically: a large gap between the linear model and the tree ensembles is evidence that the relationship being learned is genuinely non-linear and interactive.

2.3.4 Deep Sequence Models

Recurrent networks, and in particular the long short-term memory architecture [17], consume a sequence directly and can in principle learn temporal structure that a tabular model must be given explicitly through lag features. Li et al. [18] applied an LSTM to air-pollutant concentration prediction and reported improvements over shallower baselines, and a substantial literature has followed in that direction.

Against this, a body of careful comparative work argues that deep architectures do not dominate on tabular problems. Shwartz-Ziv and Armon [21] found that gradient-boosted trees outperformed several purpose-built deep tabular architectures across a range of datasets, and Grinsztajn et al. [22] analysed *why*: tree ensembles cope better with irregular target functions, are less affected by uninformative features, and are not biased towards rotationally invariant solutions in a space where the individual features carry meaning. This project treats the question empirically rather than by assumption: an LSTM was implemented and evaluated on the same task and the same chronological split as the tree ensembles, and the resulting comparison, reported in Chapter 8, informed the decision about which model to promote.

2.3.5 Multi-Horizon Forecasting Strategies

When a forecast is required at many lead times, three strategies are available: *recursive*, in which a one-step model is applied repeatedly to its own output; *direct*, in which a separate model is trained for each horizon; and the hybrid or multi-output variants. Ben Taieb et al. [12] compared these strategies systematically and identified the essential trade-off: the recursive strategy accumulates its own errors as the horizon grows, while the direct strategy avoids that accumulation at the cost of training and maintaining one model per horizon.

The design adopted here is a variant of the direct strategy that avoids its cost. The horizon h is supplied to a single model as an input feature, so one global estimator serves every lead time. This eliminates error accumulation, requires exactly one artefact to be versioned, registered, explained and monitored, and allows every horizon to be trained on the whole dataset rather than on a slice of it. The design and its consequences are developed in Section 5.8.

2.4 Honest Evaluation of Forecasts

Two methodological points from the forecasting literature shape how this system reports its accuracy.

The first concerns validation. Tashman [10] reviewed out-of-sample testing practice and argued for rolling-origin evaluation over a single holdout, and Bergmeir and

Benítez [11] examined the use of cross-validation for time-series predictors and set out the conditions under which random k-fold partitioning is invalid. The essential point is that a randomly chosen validation set for a time series lets the model train on the future and test on the past, which flatters it. This project therefore splits strictly chronologically and, beyond that, reports a five-fold walk-forward backtest in which each fold trains on the past and tests on the window that follows.

The second concerns reporting granularity. A multi-horizon forecaster summarised by one averaged error figure conceals the fact that skill decays sharply with lead time; a single number that mixes a one-hour and a 72-hour forecast is dominated by whichever horizon happens to be more numerous in the validation set, and it overstates usefulness at the horizons a user actually cares about. This project therefore reports RMSE, MAE and R^2 separately at every modelled horizon, alongside the persistence baseline at that same horizon.

2.5 Uncertainty Quantification

A point forecast without an interval invites false confidence. The most theoretically satisfying framework for attaching a distribution-free interval to an arbitrary regressor is conformal prediction, introduced by Vovk, Gammerman and Shafer [23]; the computationally practical inductive or *split* variant is due to Papadopoulos et al. [24], and Lei et al. [25] give a modern treatment with finite-sample coverage guarantees. In its simplest form, split conformal prediction fits the model on a training set, computes residuals on a disjoint calibration set, and constructs the interval from the empirical quantiles of those residuals; the resulting interval attains the nominal coverage under exchangeability.

The intervals in this system are constructed in exactly this spirit: residual quantiles are computed on the held-out chronological validation set and stored per horizon, so that the interval widens with lead time as the model's actual errors do. The exchangeability assumption is, strictly, not satisfied for a non-stationary time series, and Chapter 8 discusses the consequences of that honestly rather than claiming a guarantee the construction does not earn.

2.6 Explainable Machine Learning

Ribeiro et al. [26] introduced LIME, which explains an individual prediction by fitting an interpretable surrogate in its neighbourhood. Lundberg and Lee [27] subsequently unified several attribution methods under the framework of Shapley values from cooperative game theory, producing SHAP: the unique additive attribution satisfying local accuracy, missingness and consistency. The practical obstacle to Shapley values is their exponential cost, which Lundberg et al. [28] removed for tree ensembles with TreeSHAP,

a polynomial-time exact algorithm that also supports aggregation of local attributions into global importance.

Two properties make SHAP the right choice here. It is additive, so the attributions for a single forecast sum exactly to the difference between that forecast and the model's base value - which means an explanation can be rendered as an arithmetic statement a non-specialist can follow, rather than as a ranking whose units are unclear. And TreeSHAP is exact and fast for the gradient-boosted champion, so an explanation can be computed inside a batch job without materially lengthening it. Section 6.16 describes how the attributions are translated into natural language.

2.7 Feature Stores and the Feature-Training-Inference Pattern

Sculley et al. [29] made the observation that frames the operational half of this project: in a real machine-learning system the model is a small fraction of the code, and the surrounding infrastructure - data collection, feature extraction, configuration, serving, monitoring - is where the complexity and the accumulated debt actually live. They identified *training-serving skew*, in which features are computed one way for training and another way for inference, as a recurring and particularly damaging failure mode.

The feature store emerged as the industrial answer to that problem. Introduced publicly as a component of Uber's Michelangelo platform [31], it is a versioned, queryable repository of engineered features with a defined schema, primary keys and event time, written by feature pipelines and read by both training and inference. Because both sides read the same table, skew is eliminated by construction rather than by discipline. Hopsworks, the platform used in this project, provides such a store together with an integrated model registry [33].

The architectural pattern that follows from this is the Feature-Training-Inference decomposition [32], in which a machine learning system is expressed as exactly three pipelines that share no code path and communicate only through the feature store and the model registry. The pattern's value for this project is specific and practical: because the only durable state lives in managed services, the compute can be entirely ephemeral, which is what allows the whole system to run on a free scheduler.

2.8 Machine-Learning Operations

Breck et al. [30] proposed a rubric for production readiness in machine-learning systems, scoring tests across four areas - features and data, model development, infrastructure, and monitoring - and observing that most deployed systems score poorly on the last two. Their rubric supplies, in effect, a checklist against which the present system can be assessed, and several of its items map directly onto components implemented here: that

the model is tested against a simpler baseline, that training is reproducible and scheduled, that model quality is validated before serving, and that the model can be rolled back.

Two of those items deserve individual attention.

2.8.1 Gated Promotion and the Champion-Challenger Pattern

The champion-challenger pattern originates in credit scoring and marketing analytics, where it is standard practice that an incumbent model continues to make decisions until a challenger has demonstrated superiority on an agreed metric; Siddiqi [35] documents it in that setting alongside the monitoring practice discussed below. Transplanted into an automated retraining pipeline, it answers a question that automation creates: if a system retrains every night, what stops it from deploying whatever it happened to produce last night? Without a gate, a retrain on an unrepresentative window can quietly degrade a live service. The gate implemented here, described in Section 6.13, promotes a challenger only if it improves on the champion's validation RMSE, and records every decision - including refusals - in a persistent leaderboard.

2.8.2 Drift Detection

Gama et al. [34] survey concept drift and its adaptation strategies, distinguishing between change in the input distribution and change in the relationship between inputs and target, and between abrupt, gradual and recurring change. For a seasonal environmental system, *recurring* drift is the normal condition rather than a fault, which is an important interpretive point: a drift alarm in November does not mean the system is broken.

The measure used in this project is the Population Stability Index, long established in credit-risk monitoring [35]. For a reference distribution binned into B bins with proportions r_b, and a current distribution with proportions c_b over the same bin edges,

$$\mathrm{PSI} = \sum_{b=1}^{B} (c_b - r_b) \ln\left(\frac{c_b}{r_b}\right), \tag{2.2}$$

with the conventional interpretation that values below 0.10 indicate a stable population, values between 0.10 and 0.25 moderate shift, and values above 0.25 significant shift. PSI is attractive here because it is non-parametric, cheap, computed per feature so that the responsible variable is identified, and interpretable against a threshold that does not need to be tuned. Its weakness - that it measures input drift only, and says nothing about whether accuracy has actually suffered - is why this system pairs it with direct measurement of realised forecast error against observed outcomes.

2.9 Language Models as an Interface Layer

The final component of the system exposes its forecasts to a large language model, so that a user can ask a question in ordinary language rather than reading a chart. The relevant methodological concern is grounding: a model asked about air quality will otherwise produce a plausible number from its parameters rather than the system's actual forecast. The mitigation adopted is tool use, in which the model is given a declared set of functions and returns a structured call rather than prose, and the answer is composed from the function's real return value. The Model Context Protocol [46] standardises the declaration and invocation of such tools across clients, which allows the same four grounded functions to serve both the dashboard's advisor and any external protocol client.

2.10 Literature Review Matrix

Table 2.1 condenses the survey, relating each strand of work to what it contributes and to what it leaves open for this project.

Table 2.1: Literature review matrix.

Strand	Representative work	Contribution and limitation for this project
Health burden and index definition	WHO [1]; EPA [5]; IQAir [3]	Defines what is being predicted and why it matters; establishes that the burden is particulate-dominated and seasonal. Does not address forecasting.
Chemical transport modelling	CAMS [6]	Physically rigorous and the scientific reference. Requires an emissions inventory and computing capacity that a free-tier project cannot supply.
Statistical time series	Hyndman and Athanasopoulos [9]	Supplies the persistence baseline and the discipline of comparing against it. Univariate models cannot consume a weather forecast, so they fade at long horizons.
Tree ensembles	Breiman [14]; Friedman [15]; Chen and Guestrin [16]	The modelling core: strong on tabular, interactive, weather-driven data and cheap to retrain daily. Requires temporal structure to be supplied explicitly through engineered features.
Deep sequence models	Hochreiter and Schmidhuber [17]; Li et al. [18]	Learns temporal structure directly and is a legitimate contender. Costly to train, single-horizon as usually formulated, and harder to explain.
Trees versus deep learning on tabular data	Shwartz-Ziv and Armon [21]; Grinsztajn et al. [22]	Provides the principled reason to test rather than assume, and predicts the outcome observed here.

Strand	Representative work	Contribution and limitation for this project
Multi-horizon strategy	Ben Taieb et al. [12]	Frames the direct versus recursive choice. Leaves open the practical cost of one model per horizon, which the horizon-as-feature design resolves.
Validation methodology	Tashman [10]; Bergmeir and Benítez [11]	Justifies chronological splitting and walk-forward backtesting over random cross-validation.
Uncertainty	Vovk et al. [23]; Lei et al. [25]	Gives a distribution-free construction for intervals. Its exchangeability assumption is imperfect for a seasonal series.
Explainability	Lundberg and Lee [27]; Lundberg et al. [28]	Exact, fast, additive attributions for tree models, which can be rendered arithmetically in plain language.
Feature stores and FTI	Sculley et al. [29]; Michelangelo [31]; Dowling [32]	Supplies the architecture that eliminates training-serving skew and permits stateless compute. Assumes a managed store is available, which the free tier constrains.
MLOps practice	Breck et al. [30]; Siddiqi [35]	Provides the readiness rubric, the promotion gate and the drift measure. Neither is specific to environmental forecasting.
Drift	Gama et al. [34]	Distinguishes recurring seasonal drift from genuine decay, which is essential to interpreting this system's own alarms.
Grounded language interfaces	MCP [46]	Standardises tool declaration so a language model answers from real forecasts rather than from its parameters.

2.11 Research Gap Analysis

Read together, the literature leaves four gaps that this project addresses.

Gap 1 - Coverage. Published air-quality forecasting work for Pakistan is overwhelmingly concerned with Lahore or Karachi, and models are typically fitted per site. A single global model conditioned on location, trained across 22 cities spanning every province and territory, is not the usual formulation, and it carries a specific benefit: a city with a shorter or noisier record borrows strength from the rest.

Gap 2 - Operation rather than experiment. The overwhelming majority of the modelling literature reports an accuracy figure for a model trained once. Scheduled retraining, gated promotion, a durable registry and self-monitoring are described in the MLOps literature and in industrial platform papers, but are rarely instantiated in published air-quality work. This project's contribution is to assemble the two.

Gap 3 - Reporting that reflects use. Where multi-horizon forecasts are published, accuracy is usually reported as a single averaged number. Reporting per horizon, against a per-horizon baseline, and under walk-forward backtesting, gives a materially different -

and less flattering, therefore more useful - picture of what a three-day forecast is actually worth.

Gap 4 - Explanation and uncertainty as delivered features. SHAP and conformal intervals are both well established as techniques, but they are usually evaluated in isolation. Here they are delivered: every forecast in the public interface carries an interval and a sentence explaining what drove it, and the what-if simulator lets a user perturb the drivers and watch the model respond.

2.12 Relevance to This Project

Each strand of the survey translated into a concrete design decision.

The health and index literature fixed the target: the EPA AQI, with its six categories and their advisory language, computed from particulate matter where it must be computed at all. The comparison of forecasting traditions ruled out physical simulation on resource grounds and univariate statistics on horizon grounds, and pointed to supervised regression with forecast weather as inputs. The tabular-learning literature justified making XGBoost the primary candidate while requiring that the sequence-model alternative be measured rather than dismissed. The multi-horizon literature motivated the horizon-as-feature formulation. The validation literature dictated chronological splitting, a persistence baseline at every horizon and a walk-forward backtest. The conformal literature supplied the interval construction. The explainability literature supplied TreeSHAP and the additive property that makes plain-language rendering possible. The feature-store and FTI literature supplied the architecture, and the MLOps literature supplied the promotion gate, the registry and the drift measure that make the system self-operating. The tool-use literature supplied the grounding discipline for the advisor.

2.13 Summary of Literature Review

This chapter reviewed the health and regulatory basis of the Air Quality Index, the three traditions of air-quality forecasting and the reasons for choosing supervised machine learning among them, the model families relevant to a tabular multi-horizon regression and the evidence on trees versus deep networks, the methodology of honest forecast evaluation and uncertainty quantification, the explainability techniques appropriate to tree ensembles, and the operational literature on feature stores, the Feature-Training-Inference pattern, gated promotion and drift. It identified four gaps - coverage, operation, reporting granularity, and delivered explanation - which the remainder of this thesis addresses. The next chapter states the problem formally.

Chapter 3

Problem Definition

3.1 Introduction

Chapter 1 introduced the project and Chapter 2 surveyed the work it builds on. This chapter states the problem precisely. It sets out what is wrong with the air-quality information currently available to the Pakistani public, examines the systems that exist and where each of them stops, distils the shortcomings into a numbered set of gaps, outlines the solution this project proposes against each of them, records the assumptions and constraints under which that solution was built, assesses its feasibility, and enumerates the deliverables by which its completion can be judged.

3.2 Problem Statement

Air quality in Pakistan is a severe, recurring and largely predictable public-health problem, and the information infrastructure available to respond to it is inadequate in four distinct ways.

It reports the present rather than the future. Official and commercial sources publish the Air Quality Index as it stands at a monitoring station. This is a measurement, not a forecast, and it arrives too late to inform the decisions people take: whether to walk a child to school tomorrow morning, whether to shift an outdoor work detail to a cleaner day, whether an asthmatic should refill a prescription before a smog episode. The World Health Organization's guidelines [1] are framed around exposure, and exposure is something that can only be reduced by acting *before* it occurs.

It covers a few places and ignores the rest. Where forecasts do exist, they overwhelmingly concern Lahore and Karachi. The exploratory analysis conducted for this project, presented in Section 5.6, found that Faisalabad's mean AQI over the study period was in fact *higher* than Lahore's, and that Sargodha, Multan, Gujranwala and Sialkot all sit above the threshold the EPA labels unhealthy. Residents of these cities, and of the smaller cities of Khyber Pakhtunkhwa, Balochistan, Gilgit-Baltistan and Azad Jammu & Kashmir, are served by nothing at all. A per-city modelling approach scales badly to this problem, because each new city requires its own model, its own training run and its own maintenance.

It explains nothing and quantifies nothing. A bare index value gives a user no way to distinguish a genuine pollution episode from an unusual meteorological configuration, no indication of how confident the source is, and no means of judging whether the source has been right in the past. Trust in a public-health signal is not established by assertion; it is established by showing the working. The literature offers the tools - Shapley attributions [27, 28] for the first concern and conformal intervals [25] for the second - but they are seldom delivered to an end user.

It is maintained by hand, if at all. This is the deepest of the four problems, and the one the machine-learning literature most often overlooks. Air quality is not stationary: the relationship between weather, calendar and pollution shifts with the season, with agricultural cycles and with changes in emission. A model trained once and left alone therefore decays, and the decay is silent - the model continues to emit confident numbers that are progressively less correct. Sculley et al. [29] identified precisely this class of failure as the dominant source of technical debt in deployed machine-learning systems, and Gama et al. [34] characterised the drift mechanism behind it. A forecasting service for a seasonal phenomenon must therefore be self-maintaining, or it will quietly stop working within a year of deployment.

Stated formally: *there is a need for an automated system that forecasts the Air Quality Index several days ahead for many Pakistani cities rather than one, that retrains and validates itself without human intervention while refusing to deploy a model that is not measurably better than the one in service, that quantifies its own uncertainty and monitors its own inputs and outputs for decay, that explains each individual prediction in language a non-specialist can act on, and that is delivered publicly and reliably at negligible cost.*

3.3 Existing Systems and Current Solutions

Four families of existing system bear on this problem. Each solves part of it and stops somewhere specific.

3.3.1 Government and Regulatory Monitoring

Provincial environmental protection agencies operate reference-grade monitoring stations and publish index readings. These are the authoritative source for *what the air is now*, and nothing in this project attempts to displace them. Their limitations for the present purpose are that the station network is sparse relative to the population it serves, that the published output is a nowcast, and that no explanation or forecast accompanies it.

3.3.2 Consumer Air-Quality Applications

A number of commercial applications and websites aggregate station and satellite data into a consumer-facing index, and the better ones include a short forecast. Their weaknesses, from the standpoint of this problem, are that the model behind the forecast is undisclosed and unevaluated - no per-horizon accuracy is published, so a user cannot know whether the third day of a three-day forecast carries any skill at all - that no explanation is offered, and that coverage of smaller Pakistani cities is thin. They are products rather than scientific instruments, and they are not designed to be checked.

3.3.3 Physical Chemical Transport Models

Services such as the Copernicus Atmosphere Monitoring System [6] produce genuinely predictive composition forecasts from first principles. They are the correct tool for scientific attribution and for regional policy analysis. They are also global-scale scientific infrastructure: they presuppose an emissions inventory, meteorological boundary conditions and substantial computing capacity. They cannot be reproduced by an individual developer, they are not tuned to individual Pakistani cities, and their output is not delivered in a form a resident can act on.

3.3.4 Published Machine-Learning Studies

The academic literature on machine-learning air-quality forecasting is large [19, 20, 18], and it establishes convincingly that supervised models can forecast pollutant concentrations with useful skill. Its characteristic limitation is that it addresses the modelling question and stops. A typical paper reports a single averaged accuracy figure for a model trained once on a fixed dataset, frequently for one city, sometimes on a randomly partitioned validation set that allows the model to train on the future. There is usually no retraining pipeline, no promotion gate, no durable registry, no drift instrumentation, no deployed interface and no ongoing measurement of whether the model is still right. The model is a result; it is not a service.

3.4 Identified Problems and Gaps

The preceding survey reduces to seven concrete gaps, each of which is addressed by a specific component of the system described in this thesis.

Table 3.1: Identified gaps and the component of this system that addresses each.

ID	Gap in current practice	How this project addresses it
G1	Information is a nowcast, so it arrives after the decision it should have informed.	A 72-hour hourly forecast, refreshed daily, aggregated into both an hourly curve and a three-day view.
G2	Coverage is limited to one or two major cities; per-city models scale badly.	A single global model conditioned on location, trained across 22 cities covering every province, the capital territory, Gilgit-Baltistan and Azad Jammu & Kashmir.
G3	Accuracy is unreported, or reported as one averaged number that conceals decay with lead time.	Per-horizon RMSE, MAE and R^2 against a per-horizon persistence baseline, plus a five-fold walk-forward backtest, published in the interface.
G4	Point forecasts are presented without uncertainty.	An 80% prediction interval derived from empirical residual quantiles, widening with horizon, drawn as a band on every chart.
G5	Predictions are unexplained, so they cannot be interrogated or trusted.	Per-forecast SHAP attributions rendered as a plain-language sentence, plus an interactive what-if simulator over the model's own drivers.
G6	Models are trained once and decay silently as the relationship shifts.	An hourly feature pipeline and a nightly retraining pipeline, with a champion-challenger gate that promotes only a measurable improvement and records every decision.
G7	Nothing watches the system after deployment.	Population Stability Index drift monitoring per feature, plus a forecast log joined back to realised outcomes so that live error is measured rather than assumed.

3.5 Proposed Solution Overview

The proposed solution is an operating system rather than a model, built on the Feature-Training-Inference pattern [32] and running entirely on free scheduled infrastructure. Its shape is as follows.

An **hourly feature pipeline** fetches the last seven days of pollutant and weather observations for all 22 cities from a free, keyless public API [8], engineers a 74-column feature table for each city independently, and upserts the result into a managed feature store keyed on city and timestamp. A one-off **backfill** runs the same logic over the full history from January 2023 to the present, in three-month chunks, so that the store contains roughly 697,000 hourly rows before the first model is trained.

A **daily training pipeline** reads the entire feature table, builds a multi-horizon supervised dataset in which the forecast horizon is itself an input feature, splits it chronologically, trains four candidate models - a persistence baseline, a regularised linear model, a random forest and a gradient-boosted ensemble - and scores each on a

validation window that lies strictly in the future relative to the training window. The best fitted candidate becomes the *challenger*, and is promoted to *champion* only if it improves on the incumbent's validation RMSE. Every run, promoted or refused, is recorded in a persistent leaderboard, and every promoted champion is written to a durable, versioned model registry.

A **batch inference pipeline** loads the reigning champion from that registry, fetches each city's current pollution state and the coming four days of forecast weather, assembles one model input per future hour, predicts the whole 72-hour curve, widens each point by the residual quantiles recorded for its horizon, explains the peak hour with SHAP, raises a hazard alert if the peak crosses the unhealthy threshold, and writes a single forecast document.

An **observability layer** computes the Population Stability Index for each monitored feature between a recent window and the historical reference, appends every issued forecast to a log, and joins that log back to the air quality that subsequently arrived so that realised error can be measured directly.

A **delivery layer** serves the forecast document, the leaderboard, the evaluation results and the monitoring report through a typed HTTP API, presents them in a four-tab browser dashboard with an interactive map of Pakistan, and exposes the same grounded data to a language-model advisor through a standard tool protocol [46].

3.6 Assumptions and Constraints

3.6.1 Assumptions

1. **The data source is representative and available.** Open-Meteo's air-quality product is derived from the Copernicus atmospheric composition reanalysis and forecast, and its weather archive from ERA5 [7]. These are modelled rather than station-measured values for a given city, and the system assumes they are an adequate proxy for the air a resident of that city breathes.
2. **The weather forecast is usefully accurate.** At training time the model sees the weather that actually occurred at the target hour; at inference time it sees the forecast weather for that hour. The system assumes that the gap between the two is small enough not to invalidate the learned relationship. This is standard practice in operational forecasting, but it means the reported validation error is a slight under-estimate of true operational error, and Chapter 8 treats that honestly.
3. **The target definition is stable.** The EPA index definition and its category boundaries [5] are assumed not to change during the system's life.

4. **Recent history is available at inference time.** The anchor state - the observed pollution at the moment the forecast is issued - is assumed to be retrievable for every city at every run.

3.6.2 Constraints

1. **Zero budget.** Every component must sit within a free tier. This ruled out managed training infrastructure, paid data, always-on servers and GPU compute, and it directly shaped several design decisions documented in Chapter 6, including the decision to disable feature-store statistics after the profiler exhausted the free executor's memory.
2. **Ephemeral compute.** Scheduled runners are destroyed when a job ends, so no artefact may be left on local disk if it is needed later. This is the constraint that forces a genuine model registry rather than a saved file.
3. **Job time limits.** A scheduled job must finish inside the platform's cap, which constrains the size of the training set and forced the backfill to be chunked, resumable and non-blocking.
4. **Development platform.** Development was carried out on Windows, where the feature-store client cannot be installed because one of its transitive cryptographic dependencies fails to build. All feature-store work therefore runs on Linux continuous integration, and local development uses an interchangeable Parquet backend behind the same interface.
5. **Source granularity.** The finest resolution available is one city and one hour; street-level or sub-hourly forecasting is therefore impossible regardless of modelling choices.
6. **Public exposure.** The deployed interface is public and unauthenticated, which constrains it to read-only, non-sensitive data and makes cost predictability a design concern.

3.7 Feasibility Analysis

3.7.1 Technical Feasibility

The technical risks were identified at the outset and each was resolved by a concrete mechanism.

The first risk was whether a free-tier feature store could absorb a three-and-a-half-year, 22-city backfill. It could, but not naively: a blocking insert waited indefinitely on a stalled materialisation job and froze the run at the tenth city. Converting the write to a non-blocking submission, writing city by city rather than in one batch, and making

the resume logic history-aware rather than presence-aware resolved it, and the resulting design is idempotent, so a failed run can simply be repeated.

The second risk was whether a single global model could serve 22 cities at ten horizons without being worse than a per-city model at any of them. The evidence in Chapter 8 is that it can: the global model beats the persistence baseline at every horizon from two hours to 72, by margins of between 16% and 41%, and it retains a substantial margin at the long horizons where a local or univariate approach is weakest. The one exception, a one-hour lead time at which the trivial predictor is better, is reported and explained rather than omitted.

The third risk was whether the daily retraining job would fit inside the scheduler's time limit. Subsampling the supervised set to 200,000 rows brings a full four-candidate training run comfortably inside the cap, and this is the principal reason the gradient-boosted model rather than the recurrent model is the production choice: it trains in about a minute against roughly twenty-five for the sequence model on the same hardware.

The fourth risk was whether the trained model could be served without a live feature-store connection, since the deployment image deliberately omits that dependency. It can, because the inference stage writes a self-contained forecast document and the model bundle is shipped with the image.

3.7.2 Economic Feasibility

The system's recurring cost is essentially zero, which was a requirement rather than an accident. Data acquisition is free and requires no key. Scheduled compute uses the free allowance of the repository host's continuous-integration service; the hourly feature job and the nightly training job together consume a small fraction of it. Feature storage and the model registry use a free managed tier. The frontend is served from a free static edge host. The backend runs as a small container on a low-cost container host. The only component with a genuinely variable cost is the optional language-model advisor, which is metered per request and degrades gracefully - returning a clear message rather than an error - when no key is configured. The consequence is that the service can keep running indefinitely without a funding decision, which is precisely what a public-interest tool needs.

3.7.3 Operational Feasibility

Operationally, the system is designed to require no one. Ingestion, retraining, evaluation, inference and monitoring are all scheduled; the refreshed artefacts are committed automatically; the frontend reads them without redeployment. The promotion gate means that an unattended retrain cannot degrade the service, since a challenger that fails to

improve is simply refused and the incumbent continues. Each pipeline logs its decisions in a form an operator can audit after the fact, and the leaderboard, evaluation report and monitoring report are all published in the interface rather than buried in logs. The realistic operational burden is periodic review rather than daily intervention.

3.8 Deliverables and Development Requirements

Table 3.2 lists the deliverables against which the project's completion is assessed.

Table 3.2: Project deliverables.

ID	Deliverable	Description
D1	Ingestion and feature engineering	A retrying client for three Open-Meteo endpoints, an EPA index implementation, and a per-city feature builder producing 74 columns from 21 raw ones.
D2	Populated feature store	Roughly 697,000 hourly rows for 22 cities from January 2023 onward, upserted on a composite key, with a resumable backfill.
D3	Exploratory analysis	City ranking, seasonality, diurnal profile, weather correlation and distribution figures over the full record.
D4	Multi-horizon supervised builder	Assembly of target-time, anchor-state and horizon features with a chronological split and unit tests asserting the absence of leakage.
D5	Training pipeline	Four candidate families, honest validation, residual quantile intervals, and a bundled artefact carrying the estimator, column order, interval table and metrics.
D6	Champion-challenger gate and registry	A persistent leaderboard, a promotion rule, and a durable versioned model registry from which inference loads.
D7	Evaluation suite	Per-horizon metrics against a per-horizon baseline, a five-fold walk-forward backtest, and a published report and figures.
D8	Sequence-model study	An LSTM trained and evaluated on the same task and split, with a reasoned promotion decision.
D9	Inference pipeline	A 72-hour forecast with intervals, categories, alerts and explanations for every city, written as a single document.
D10	Explainability	TreeSHAP attributions rendered in plain language, per-forecast in the interface and globally as an importance figure.
D11	What-if simulator	Live re-prediction under user-overridden drivers, presented as baseline versus scenario.

ID	Deliverable	Description
D12	Monitoring	Per-feature PSI drift with status thresholds, a forecast log, and realised error scored against observed outcomes.
D13	API and dashboard	A typed HTTP interface and a four-tab React application with charting, cartography and alerting.
D14	Advisor and tool server	Four grounded tools exposed both to an in-app advisor and over the Model Context Protocol.
D15	Continuous integration and deployment	Three scheduled workflows, a container image for the backend, a static build for the frontend, and a live public deployment.
D16	Tests and documentation	Unit tests over the critical logic, and this thesis.

3.9 Summary of Problem Definition

This chapter defined the problem the Pearls AQI Predictor addresses: air-quality information for Pakistan that is current-only, geographically narrow, unexplained, unquantified and manually maintained. It examined the four families of existing system - regulatory monitoring, consumer applications, physical transport models and published machine-learning studies - and identified where each stops. It distilled seven gaps, outlined the pipeline-based solution proposed against them, stated the assumptions and the free-tier constraints under which that solution was built, argued its technical, economic and operational feasibility, and enumerated sixteen deliverables. The next chapter converts this into a formal requirement specification.

Chapter 4

Requirement Analysis

4.1 Introduction

This chapter converts the problem stated in Chapter 3 into a specification. It describes how the requirements were elicited, identifies the stakeholders and what each of them needs from the system, presents the use case model, narrates the principal use cases in full, enumerates the functional and non-functional requirements, records the hardware and software the system depends on, and closes with a traceability matrix linking every requirement back to the gap that motivated it and forward to the chapter in which it is realised.

4.2 Requirement Elicitation Method

The project had no external client, so requirements could not simply be collected by interview. Four complementary techniques were used instead.

Scenario construction. Concrete situations were written down and used as the primary source of functional requirements. Three of them shaped the interface directly: a parent in Faisalabad deciding on Tuesday whether Thursday's outdoor sports practice is safe; a construction supervisor in Multan deciding which of the next three days to schedule outdoor work; and a person with asthma in Lahore deciding whether to carry a mask tomorrow morning. Each scenario was traced through the interface until it either produced an answer or exposed a missing capability - the three-day aggregated view, the hazard banner and the "cleanest time of day" question all originated this way.

Analysis of the data. A substantial exploratory study of the assembled dataset, reported in Section 5.6, was carried out before the interface was designed. Several requirements came directly out of it. The finding that January averages 146.8 against 91.5 in the cleanest month established that the model would face strong recurring drift and therefore that drift monitoring was necessary. The finding that the cities differ enormously - Faisalabad's mean is roughly double Gilgit's - established that a city comparison view was worthwhile and motivated the map.

Adoption of published practice. The operational requirements were taken from the literature rather than invented. The production-readiness rubric of Breck et al. [30] supplied the requirements that the model be compared against a simpler baseline, that

training be reproducible, that model quality be validated before serving and that rollback be possible. The champion-challenger discipline [35] supplied the promotion gate, and the drift literature [34] supplied the monitoring requirement.

Iterative refinement from observed failure. Because the lifecycle was iterative, several requirements were discovered by running the system. The requirement that ingestion be resumable and idempotent (FR-3) was not in the original plan; it was added after a backfill froze partway through. The requirement that the interface degrade gracefully when the backend is unavailable (NFR-11) was added after the first deployment attempt served an interface with no data behind it.

4.3 Stakeholder Analysis

Table 4.1 identifies the stakeholders, their interest in the system and the requirements that follow from that interest.

Table 4.1: Stakeholder analysis.

Stakeholder	Interest in the system	Requirements arising
General public (resident)	Wants to know whether the air will be safe over the next few days, in their own city, in terms they can act on.	Multi-day forecast; city selection; health category and advice rather than a bare number; a prominent warning when the air will be dangerous; a plain-language explanation.
Vulnerable individuals	Children, the elderly and people with respiratory or cardiac conditions face harm at lower exposure than the general population.	Hourly resolution so that a safer time of day can be identified; explicit advice per category; the earliest hour at which the unhealthy threshold is crossed.
Analyst or operator	Needs to know whether the system is still working and whether its output can be trusted.	Published leaderboard and promotion history; per-horizon accuracy; walk-forward backtest; drift status; realised forecast error.
Assessor or reviewer	Needs to judge the engineering, not merely the output.	Reproducibility; version history; documented architecture; unit tests; an audit trail of model decisions.
Data provider (Open-Meteo)	Provides the observations free of charge and without authentication.	Polite request behaviour with retry and backoff; no unnecessary re-fetching; attribution in the interface.
Platform providers	Supply free scheduling, feature storage and hosting.	Operation within free-tier limits; jobs that finish inside time caps; no unbounded storage growth.

4.4 Use Case Diagram

Figure 4.1 presents the use case model. It distinguishes three kinds of participant: the human actors who consume or operate the system, the external data provider, and the

scheduler, which is treated as a system actor because it initiates the majority of the system's activity without any person being present.

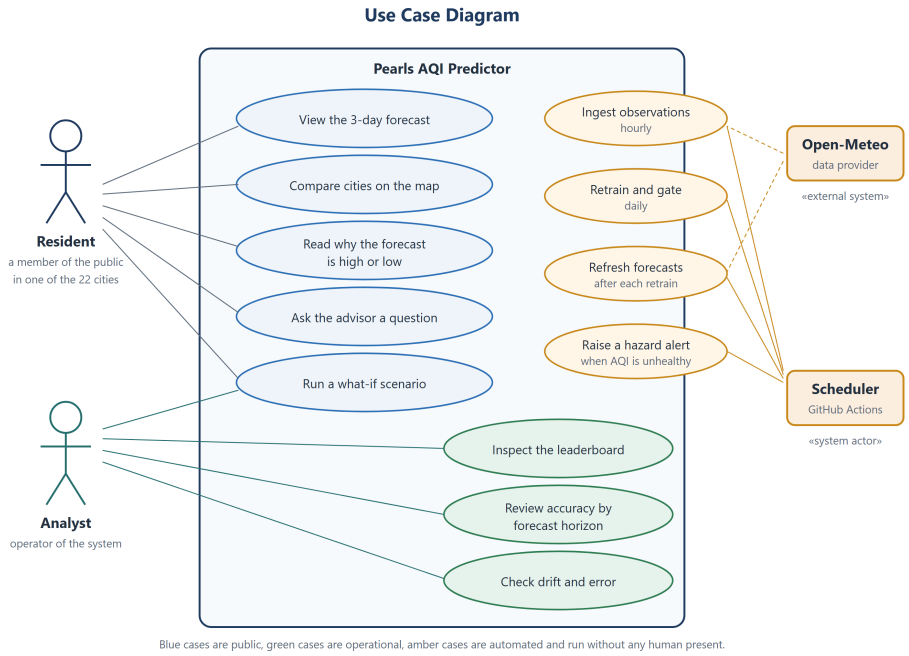

Figure 4.1: Use case diagram. Blue cases are public, green cases are operational, and amber cases are automated and are initiated by the scheduler rather than by a person.

The most important structural feature of the diagram is that the four amber cases have no human actor attached to them at all. This is the property that distinguishes the system from a conventional application: its principal behaviour - ingesting, retraining, gating, forecasting and monitoring - occurs on a schedule, and the human actors interact with the results rather than with the process.

4.5 Use Case Narratives

4.5.1 View the Three-Day Forecast

Use case ID	UC-1
Actor	Resident
Goal	To learn what the air quality will be in a chosen city over the next three days.
Preconditions	A forecast document produced by a completed inference run is available to the interface.

Main flow	1. The user opens the dashboard. 2. The system loads the forecast document and displays the default city. 3. The user selects a city from the list of 22. 4. The system displays the current index and its health category, a chart of the next 72 hours with the 80% prediction band, and the category colour scale. 5. The user switches to the daily view. 6. The system displays three calendar days, each with a mean index, a band and a category.
Alternative flow	3a. The user instead selects a city by clicking its marker on the map; the system updates the selection identically.
Exception flow	2a. The forecast document cannot be retrieved. The system displays an explanatory notice naming the failure rather than an empty chart.
Postcondition	The user has an hourly and a daily forecast, with uncertainty, for the selected city.
Requirements	FR-9, FR-10, FR-11, FR-19, NFR-3, NFR-11

4.5.2 Receive a Hazard Warning

Use case ID	UC-2
Actor	Resident, in particular a vulnerable individual
Goal	To be warned, in advance, that the air is forecast to become dangerous, and to be told when.
Preconditions	A forecast exists for the selected city.
Main flow	1. During inference the system scans the city's 72-hour curve. 2. If the peak reaches the unhealthy threshold it classifies the episode as a warning; if it reaches the very unhealthy threshold it classifies it as a danger. 3. It records the peak value, the hour of the peak, the category, the standard advice for that category and the first hour at which the warning threshold is crossed. 4. The dashboard renders this as a banner above the forecast whenever the severity is not none.
Alternative flow	2a. The peak stays below the warning threshold; no banner is shown and the forecast is presented normally.
Postcondition	The user knows that an episode is coming, how bad it will be, when it starts and what to do.
Requirements	FR-12, FR-19, NFR-3

4.5.3 Understand Why the Forecast Is High

Use case ID	UC-3
Actor	Resident
Goal	To understand what is driving the forecast, rather than being asked to accept a number.
Preconditions	A champion model and a forecast for the city exist.

Main flow	1. During inference the system identifies the hour with the highest predicted index. 2. It computes exact Shapley attributions for that single prediction against the model's base value. 3. It selects the five largest contributions by absolute magnitude and translates each feature name into a human-readable label. 4. It composes a sentence stating the forecast value, the base value and the signed contributions. 5. The dashboard shows the sentence and a signed bar for each contribution.
Exception flow	2a. The attribution computation fails. The failure is logged and the forecast is published without an explanation; the explanation card is simply omitted. Explainability must never prevent a forecast from being issued.
Postcondition	The user can see which drivers pushed the forecast up and which pushed it down, and by how much.
Requirements	FR-13, FR-14, NFR-4, NFR-8

4.5.4 Run a What-If Scenario

Use case ID	UC-4
Actor	Resident or analyst
Goal	To interrogate the model by changing its inputs and observing how the forecast responds.
Preconditions	The live backend and the champion model are reachable.
Main flow	1. The user opens the What-If tab. 2. The system builds the model input for the selected city at a lead time of 24 hours from live data and returns the current real value of each of the five adjustable drivers, so that the sliders begin at reality. 3. The user adjusts one or more sliders. 4. The user runs the simulation. 5. The system predicts once with the unmodified input and once with the overridden input and returns both values, both health categories and the difference. 6. The dashboard shows the two values side by side, colour-coded by category, with the change.
Exception flow	2a. The backend is unavailable, in which case the tab is not offered at all rather than being offered and failing.
Postcondition	The user has seen the model respond to a counterfactual, clearly labelled as a model simulation rather than a causal claim.
Requirements	FR-15, FR-16, NFR-6, NFR-11

4.5.5 Retrain and Gate the Model

Use case ID	UC-5
Actor	Scheduler (system actor)
Goal	To incorporate new data into the model without allowing an unattended run to degrade the service.
Preconditions	The feature store holds the accumulated history; the scheduled time has arrived.

Main flow	1. The scheduler starts the training job. 2. The job reads the whole feature table. 3. It constructs the multi-horizon supervised set and subsamples it to the configured limit. 4. It splits chronologically, holding out the most recent fifth as validation. 5. It scores the persistence baseline and trains the three learned candidates. 6. It selects the best fitted candidate by validation RMSE as the challenger. 7. It computes residual quantiles per horizon for the prediction intervals. 8. It compares the challenger with the reigning champion. 9. If the challenger is better it demotes the incumbent, records the promotion and writes the bundle to the model registry; otherwise it records a refusal and leaves the serving model untouched. 10. It refreshes the evaluation and monitoring reports and commits the artefacts.
Alternative flow	9a. No champion exists yet, in which case the challenger is promoted unconditionally as the first champion.
Exception flow	2a. The feature store is unreachable; the job fails loudly and the previous champion continues to serve, because promotion is the only path by which the serving model can change.
Postcondition	Either a strictly better model is in service, or the previous model is still in service; in both cases the decision is recorded.
Requirements	FR-4, FR-5, FR-6, FR-7, FR-8, NFR-1, NFR-7, NFR-9

4.5.6 Monitor Drift and Realised Error

Use case ID	UC-6
Actor	Analyst; initiated by the scheduler
Goal	To detect that the world has changed, and to know whether the forecasts have actually been right.
Preconditions	The feature store holds both historical and recent rows; previous forecasts have been logged.
Main flow	1. The system divides the feature history into a reference window and the most recent fourteen days. 2. For each monitored feature it bins the reference distribution into deciles and computes the Population Stability Index of the recent window against it. 3. It classifies each feature as stable, moderate or significant and identifies the worst. 4. It loads the forecast log and joins it to the observed index on city and target hour. 5. It computes MAE and RMSE over the joined rows and lists the five largest misses. 6. It writes the combined report, which the dashboard displays.
Alternative flow	4a. No forecast has yet matured into an observation, in which case the report states that scoring is waiting for actuals rather than reporting a spurious zero.
Postcondition	The operator can see both whether the inputs have moved and whether accuracy has suffered.
Requirements	FR-17, FR-18, NFR-9

4.5.7 Ask the Advisor a Question

Use case ID	UC-7
Actor	Resident
Goal	To ask a question in ordinary language and receive an answer grounded in the system's real forecasts.
Preconditions	The backend is reachable and an advisor credential is configured.
Main flow	1. The user types a question, or selects a suggested one. 2. The backend sends the question to the language model together with the declarations of four tools: list cities, get forecast, get history summary and explain prediction. 3. The model returns a structured tool call. 4. The backend executes the corresponding function against the real forecast data and returns the result. 5. Steps 3 and 4 repeat until the model produces a final answer, up to a bounded number of rounds. 6. The answer is returned and displayed.
Exception flow	1a. No credential is configured; the endpoint returns a clear, specific message and the interface reports that the advisor is unavailable rather than failing silently. 5a. The round limit is reached; the system asks the user to rephrase.
Postcondition	The user has an answer whose numbers came from the system's own forecasts rather than from the language model's parameters.
Requirements	FR-20, FR-21, NFR-5, NFR-11

4.6 Functional Requirements

Table 4.9 enumerates the functional requirements. Priorities are essential (E), important (I) or desirable (D).

Table 4.9: Functional requirements.

ID	Requirement	Pri.
FR-1	The system shall fetch hourly pollutant and weather observations for all 22 supported cities from a free public API, retrying transient failures with exponential backoff.	**E**
FR-2	The system shall engineer, per city and without cross-city contamination, a feature table containing calendar, cyclical, wind-vector, lag, rolling and change-rate features, and shall compute the EPA index as the target.	**E**
FR-3	The system shall upsert engineered features into a feature store on a composite key of city and timestamp, so that repeated or overlapping runs never duplicate rows, and shall support a resumable historical backfill.	**E**
FR-4	The system shall assemble a supervised dataset in which the forecast horizon is an input feature and the target is the index observed at that horizon ahead.	**E**
FR-5	The system shall split training and validation data strictly chronologically, never randomly.	**E**

ID	Requirement	Pri.
FR-6	The system shall train and score at least one baseline and three learned candidate families in every training run, and shall report RMSE, MAE and R^2 for each.	**E**
FR-7	The system shall promote a newly trained challenger to champion only if it improves on the incumbent champion's validation RMSE, and shall record every run, promoted or refused, in a persistent leaderboard.	**E**
FR-8	The system shall store the promoted champion in a durable versioned model registry that survives the destruction of the machine that trained it, and shall load the champion from that registry at inference.	**E**
FR-9	The system shall produce, for each supported city, a 72-hour hourly forecast of the index and a three-day aggregation of it.	**E**
FR-10	The system shall attach to every forecast point a prediction interval derived from the empirical residual quantiles for that horizon.	**E**
FR-11	The system shall label every forecast value with its EPA health category and the standard advice for that category.	**E**
FR-12	The system shall raise a graded hazard alert when a city's forecast peak reaches the unhealthy or very unhealthy threshold, reporting the peak value, the hour of the peak and the first crossing of the threshold.	**E**
FR-13	The system shall compute Shapley attributions for the peak forecast hour of each city and render the leading contributions as a plain-language sentence.	**I**
FR-14	The system shall produce a global feature-importance ranking from aggregated Shapley values.	**D**
FR-15	The system shall expose the current real value of each adjustable driver so that a what-if scenario begins from present conditions.	**I**
FR-16	The system shall re-predict under user-supplied driver overrides and return both the baseline and the scenario value with their categories.	**I**
FR-17	The system shall compute the Population Stability Index per monitored feature between a recent window and a historical reference, and classify each as stable, moderate or significant.	**I**
FR-18	The system shall log every issued forecast and later join it to the observed index, reporting realised MAE, RMSE and the largest misses.	**I**
FR-19	The system shall present the forecast, the map, the model leaderboard, the per-horizon and backtest evaluation, and the monitoring report through a public browser interface.	**E**
FR-20	The system shall expose forecast, history and explanation functions as declared tools so that a language model can answer questions from real data.	**D**
FR-21	The system shall publish the same tools over the Model Context Protocol for use by external clients.	**D**
FR-22	The system shall serve an on-demand forecast for an arbitrary coordinate pair, not only for the 22 pre-computed cities.	**D**
FR-23	The system shall run ingestion hourly and retraining, inference, evaluation and monitoring daily, without human initiation.	**E**

4.7 Non-Functional Requirements

Table 4.10 enumerates the non-functional requirements. Several of these were as influential on the design as the functional requirements: NFR-1 and NFR-2 between them dictated the entire architecture.

Table 4.10: Non-functional requirements.

ID	Category	Requirement
NFR-1	Cost	Every component shall operate within a free or negligible-cost tier, so that the service can run indefinitely without a funding decision.
NFR-2	Statelessness	No pipeline shall depend on state held on the machine that ran a previous pipeline; all durable state shall live in the feature store, the model registry or the repository.
NFR-3	Performance	A page view shall not wait for a model to run: the served forecast shall be a pre-computed document, read from disk.
NFR-4	Explainability	No forecast shall be presented as a bare number; each shall carry both an uncertainty band and, where computation permits, an explanation.
NFR-5	Groundedness	Any natural-language answer about air quality shall be derived from the system's own forecast data through a tool call, never generated from a language model's parametric knowledge.
NFR-6	Honesty of framing	The what-if simulator shall be labelled explicitly as a model simulation and shall not be presented as causal evidence.
NFR-7	Reproducibility	Training shall be deterministic given the same data: random seeds fixed, subsampling seeded, and the split defined by time rather than by chance.
NFR-8	Resilience	A failure in an auxiliary capability - explanation, monitoring, forecast logging - shall be caught and logged and shall never prevent the primary forecast from being produced.
NFR-9	Auditability	Every model decision shall leave a durable record: the leaderboard for promotions and refusals, the evaluation report for accuracy, the monitoring report for drift and realised error.
NFR-10	Portability	The storage layer shall be addressed through one facade with interchangeable backends, so that the full pipeline can run locally on a platform where the managed client cannot be installed.
NFR-11	Graceful degradation	Where a capability requires a live backend, the interface shall hide it rather than presenting it broken, and shall fall back to committed data snapshots when no backend is configured.
NFR-12	Maintainability	The codebase shall be organised into single-responsibility modules with dependencies flowing in one direction, and the supported cities, paths and horizons shall be defined in exactly one place.

ID	Category	Requirement
NFR-13	Usability	The interface shall communicate health status through the standard EPA colour scale and category names, shall be readable on a phone, and shall state when its data was generated.
NFR-14	Politeness to the data source	Requests shall be retried with backoff rather than repeated aggressively, historical windows shall be fetched in chunks, and data already held shall not be re-fetched.
NFR-15	Security	No credential shall appear in the repository; secrets shall be supplied through environment variables locally and through the continuous-integration secret store in automation.

4.8 Hardware and Software Requirements

Table 4.11 records the platforms and libraries the system depends on. The division between the development environment and the operating environment is significant: the feature-store client cannot be installed on the development platform at all, which is why the storage facade of NFR-10 exists.

Table 4.11: Hardware and software requirements.

Environment	Component	Requirement and purpose
Development	Workstation	A conventional laptop; no GPU is required, since the production model is a gradient-boosted ensemble that trains on CPU in about a minute.
Development	Python 3.11	Pinned below 3.12 for dependency compatibility across the machine-learning stack.
Development	Local Parquet store	Stands in for the feature store where the managed client cannot be installed.
Automation	Linux runner	Ubuntu, Python 3.11, ephemeral; runs the hourly, daily and manual workflows and is the only environment in which the feature-store client is installed.
Automation	Scheduler	Cron-triggered workflows with concurrency groups, so that two jobs never write to the feature store simultaneously.
Managed state	Feature store and model registry	Free serverless tier; holds roughly 697,000 feature rows and the versioned champion bundles.
Serving	Container host	A small always-on container running the API from a slim image; requires the OpenMP runtime for the gradient-boosting library.
Serving	Static edge host	Serves the compiled frontend bundle; no server process.
Libraries (data)	pandas, requests, pyarrow	Tabular processing, HTTP with retry, and columnar storage.

Environment	Component	Requirement and purpose
Libraries (models)	scikit-learn, XGBoost, SHAP, TensorFlow	Baselines and metrics, the champion model, attributions, and the sequence-model study.
Libraries (serving)	FastAPI, Uvicorn, Pydantic	Typed asynchronous HTTP interface with request validation.
Libraries (frontend)	React, Vite, Recharts, react-leaflet	Application framework, build tooling, charting and cartography.
Libraries (testing)	pytest	Unit tests over feature engineering, the index computation, the supervised builder and the promotion gate.

4.9 Requirements Traceability

Table 4.12 traces each gap identified in Chapter 3 through the requirements that address it to the chapter in which its realisation is documented and the means by which it is verified.

Table 4.12: Requirements traceability matrix.

Gap	Requirements	Realised in	Verified by
G1 Nowcast only	FR-1, FR-4, FR-9, FR-23	Sections 6.8 and 6.15	Live forecast documents containing 72 hourly points per city; endpoint checks.
G2 Narrow coverage	FR-1, FR-2, FR-4	Sections 5.5 and 5.8	22 cities present in the feature store and in every published forecast.
G3 Unreported accuracy	FR-5, FR-6, FR-7	Section 6.12	Per-horizon table, per-horizon baseline comparison and five-fold walk-forward backtest in Chapter 8.
G4 No uncertainty	FR-10	Section 5.9	Interval bounds present on every forecast point; band rendered in the interface.
G5 No explanation	FR-13, FR-14, FR-15, FR-16	Sections 6.16 and 6.17	Explanation object attached to each city's forecast; what-if endpoint returning baseline and scenario.
G6 Silent decay	FR-3, FR-7, FR-8, FR-23	Sections 6.13 and 6.22	Leaderboard showing a promotion, a further promotion and a refused tie; scheduled workflow history.

Gap	Requirements	Realised in	Verified by
G7 No observation	FR-17, FR-18	Section 6.18	Published PSI table with per-feature status; forecast log joined to observed values.

4.10 Summary of Requirement Analysis

This chapter set out how the requirements were elicited - from scenarios, from the data itself, from published operational practice and from observed failure - and identified the stakeholders and what each needs. It presented the use case model, whose defining feature is that the system's principal activity is initiated by a scheduler rather than by a person, and narrated seven use cases in full. It enumerated twenty-three functional and fifteen non-functional requirements, recorded the platforms and libraries the system depends on, and traced every gap from Chapter 3 through to its realisation and its verification. The next chapter presents the design that satisfies this specification.

Chapter 5

Design and Architecture

5.1 Introduction

This chapter presents the design of the Pearls AQI Predictor. It begins with the principles that governed every subsequent decision, then presents the system architecture and the alternatives that were considered and rejected. It decomposes the system into modules, specifies the data design, reports the exploratory analysis that informed the modelling choices, and then develops in turn the four design decisions that give the system its character: the feature representation and its leakage discipline, the horizon-as-feature formulation, the construction of prediction intervals, and the promotion gate. It closes with the communication design, the error-handling strategy and the deployment view.

5.2 Design Principles

Six principles were adopted at the outset and applied consistently.

P1 - Durable state, disposable compute. Every machine that runs this system is destroyed minutes after it starts. The design therefore treats compute as worthless and state as precious: nothing of value may remain on a runner when a job ends. This principle is what forces a genuine model registry rather than a saved file on disk, and it is the reason the architecture can be free.

P2 - Decoupling through shared state, not through calls. No pipeline invokes another. Their only contract is the feature store and the model registry. The consequence is that any pipeline can fail, be re-run, be rescheduled or be rewritten without the others being affected, and that a partial system is still a working system: if training fails tonight, yesterday's champion continues to serve.

P3 - The past may never see the future. Every feature attached to time t must contain only information that existed at t; every train-validation split must be chronological; every rolling statistic must be shifted so that it ends before the current observation; and no lag may cross a city boundary. This principle is enforced by construction and verified by unit tests, because target leakage produces a model that scores beautifully and then fails in production, and the failure is not visible in the metrics that would normally be trusted.

P4 - Nothing ships without proving it is better. An automated pipeline that retrains nightly will otherwise deploy whatever it happened to produce. Promotion is therefore gated on a measured improvement over the incumbent, and every decision - including every refusal - is recorded.

P5 - Auxiliary capability must never break the primary one. Explanation, monitoring and forecast logging are valuable but subordinate. Each is wrapped so that its failure is logged and the forecast is still published. A system that refuses to forecast because it could not explain itself is worse than one that forecasts without an explanation.

P6 - Publish the working. Accuracy per horizon, the backtest, the promotion history and the current drift status are part of the product, not internal diagnostics. They appear in the same interface as the forecast, because a public-health signal that cannot be checked will not be trusted.

5.3 System Architecture

The architecture, shown in Figure 5.1, is an instance of the Feature-Training-Inference pattern [32]. It divides into three planes.

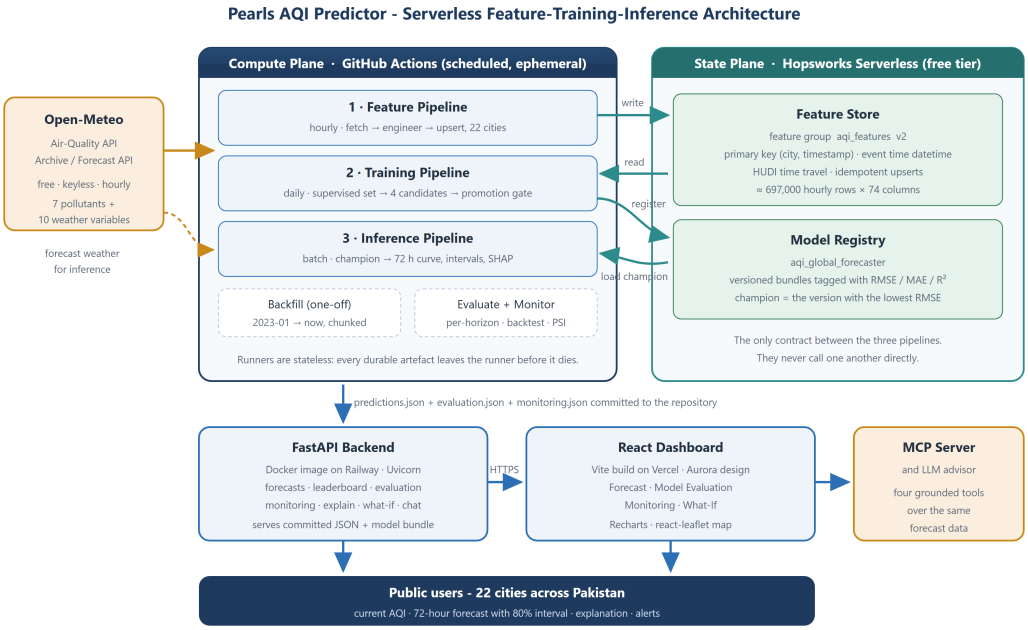

Figure 5.1: The system architecture: a source plane, a disposable compute plane, a durable state plane, and the serving layer beneath them.

The **source plane** is a single free, keyless provider [8] supplying three endpoints: an air-quality endpoint that serves both history and forecast, a weather forecast endpoint, and a weather archive endpoint derived from the ERA5 reanalysis [7]. The choice of a keyless provider was not incidental. Because there is no credential to manage, the

scheduled workflows need no secret for ingestion, which removes an entire class of operational failure.

The **compute plane** consists of scheduled, ephemeral jobs: the hourly feature pipeline, the daily training pipeline, the batch inference pipeline that follows it, the evaluation and monitoring steps, and a manually triggered backfill. None of them holds state between runs.

The **state plane** is a managed feature store and model registry. The feature store holds one feature group keyed on city and timestamp; the registry holds versioned model bundles tagged with their metrics. Together they are the sole interface between the pipelines.

The **serving layer** consists of a container-hosted API, a statically hosted browser application, and a tool server that exposes the same data over a standard protocol. Crucially, the serving layer never invokes a pipeline: it reads documents that a pipeline has already produced.

5.3.1 Alternatives Considered

Three plausible alternative architectures were considered and rejected.

A monolithic scheduled script. A single job that fetches, trains and predicts would be simpler to write and would need no feature store. It was rejected because it couples the schedules - ingestion must be hourly and training need only be daily - because a failure anywhere loses everything, and because it provides no mechanism for the trained model to outlive the run. It also reintroduces training-serving skew [29], since the features used for prediction would be recomputed rather than read.

A live prediction service. The model could be loaded into the API and invoked per request. This would allow any city and any horizon on demand. It was rejected as the primary path because it makes every page view depend on model inference, requires an always-on process sized for peak load, and reintroduces the possibility that a user-facing request fails because a model failed. A reduced form of it survives, however: the API exposes an on-demand endpoint for arbitrary coordinates and for what-if simulation, so the capability exists where it genuinely adds value without the whole product depending on it.

One model per city, or one model per horizon. Both were rejected for the same reason - artefact proliferation. Twenty-two cities times ten horizons is 220 models to train, version, register, explain and monitor, and each would be trained on a small slice of the available data. The global horizon-as-feature design developed in Section 5.8 achieves the same coverage with one artefact.

5.4 Module Decomposition

Figure 5.2 shows the decomposition of the Python package into layers. Dependencies flow strictly downwards.

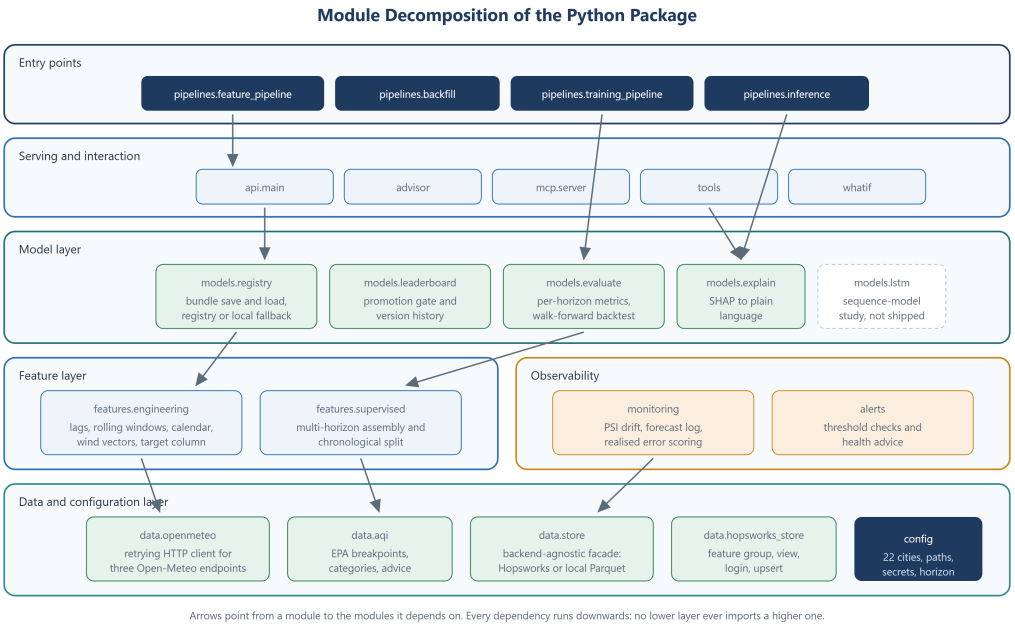

Figure 5.2: Module decomposition. Arrows point from a module to the modules it depends on; no lower layer imports a higher one.

At the bottom sits the **data and configuration layer**. A single configuration module is the sole source of truth for the supported cities, the filesystem layout, the feature-store object names and the forecast horizon; every other module imports from it, so adding a city is a one-line change. The ingestion module wraps the three provider endpoints behind a retrying session. The index module implements the EPA breakpoints, categories and advisory text. The store module is a facade over two interchangeable backends, which is what satisfies NFR-10.

Above it, the **feature layer** contains exactly two modules: one that turns raw observations into the engineered feature table, and one that turns the feature table into the multi-horizon supervised set and splits it chronologically. Keeping these separate matters, because the first is used by both the feature pipeline and the inference pipeline, while the second is used only by training and evaluation.

The **model layer** contains the registry facade, the leaderboard and promotion gate, the evaluation suite, the explanation module, and the sequence-model study, which is deliberately isolated so that its heavy dependency never reaches the deployment image.

The **observability layer** holds drift and error monitoring and the alerting rules, and the **serving layer** holds the API, the tool definitions, the advisor, the tool-protocol server

and the what-if simulator. The tool definitions are shared: the advisor and the protocol server call the same four functions, so a question answered in the dashboard and the same question asked from an external client cannot disagree.

5.5 Data Design

Figure 5.3 specifies the data design.

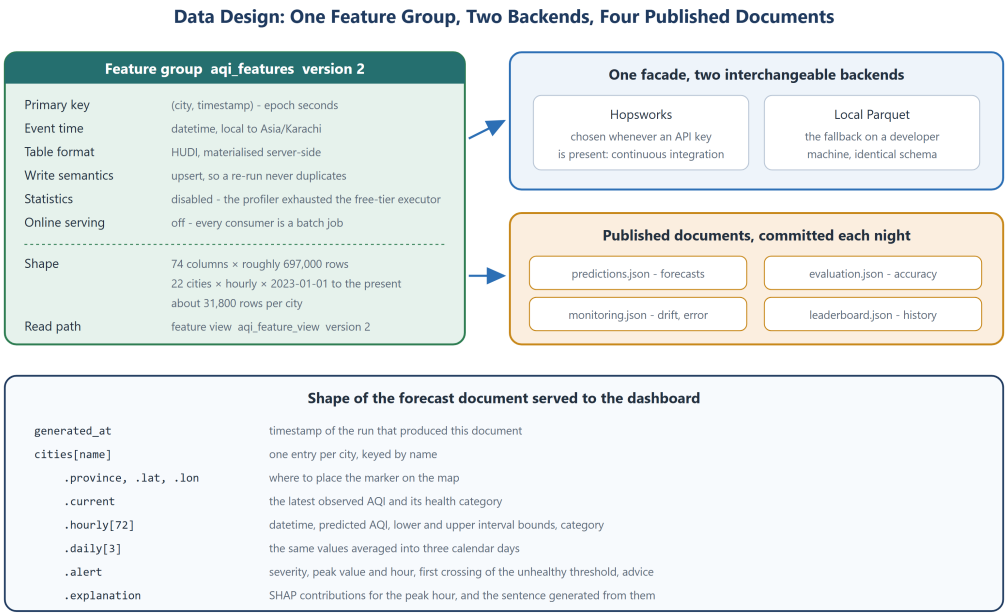

Figure 5.3: Data design: the feature group and its keys, the two interchangeable storage backends, the four published documents, and the structure of the forecast payload.

5.5.1 The Feature Group

All engineered features live in one feature group. Its **primary key** is the pair (city, timestamp), where timestamp is the observation hour expressed in epoch seconds; its **event time** is the corresponding datetime column. This choice is what makes ingestion idempotent: the hourly pipeline deliberately fetches the last seven days rather than the last hour, so that lag and rolling features for the newest observations have sufficient history, and the resulting overlap is absorbed by the upsert rather than producing duplicates. A failed run can therefore simply be repeated.

Two configuration decisions were forced by the free tier and are documented here because they are design decisions rather than accidents. The table format is set to a time-travel format that materialises server-side, because the platform's default format requires a client library that the base installation does not ship. Statistics computation is disabled, because the profiler over a 74-column table exhausted the memory of the

free-tier executor and failed the materialisation job; statistics are a convenience for the platform's own interface and are not read by any pipeline, so disabling them costs nothing and makes the job finish reliably.

5.5.2 The Storage Facade

Every pipeline addresses storage through one module exposing four operations: save features, read features, list which cities are present, and report the earliest date held per city. Two backends implement it. The managed feature store is selected whenever an API key is present; a local columnar store is used otherwise. Both hold the identical schema and honour the same primary key. This is a straightforward application of dependency inversion, and it earns its keep twice: it allows the entire pipeline to be developed and tested on a platform where the managed client cannot be installed, and it means the choice of vendor is confined to a single module.

5.5.3 Published Documents

The pipelines publish four documents, which are committed to the repository each night and are the only things the serving layer reads: the forecast payload, the evaluation report, the monitoring report and the leaderboard. Publishing them as committed files rather than as a live query is what allows the deployed backend to omit the feature-store dependency entirely, and what guarantees that a page view never waits for a model.

The structure of the forecast payload is given in the lower half of Figure 5.3. Each city carries its province and coordinates for the map, its current observed index and category, 72 hourly points each with a predicted value, interval bounds and a category, a three-day aggregation, an alert object, and an explanation object.

5.6 Exploratory Data Analysis

Before any model was designed, the assembled dataset - 696,960 hourly rows across 22 cities from 1 January 2023 to August 2026 - was analysed. Five findings shaped the design.

5.6.1 The Cities Differ Enormously

Figure 5.4 ranks the cities by mean index over the whole record. Faisalabad is the most polluted at a mean of 156.9, marginally ahead of Lahore at 151.5; Gilgit is the cleanest at 75.7. The spread is more than a factor of two.

Two design consequences follow. First, five cities have a *mean* above 140, which is to say that the typical condition in those cities is at or near the threshold the EPA calls unhealthy; a forecasting service for them is not a curiosity. Second, the fact that

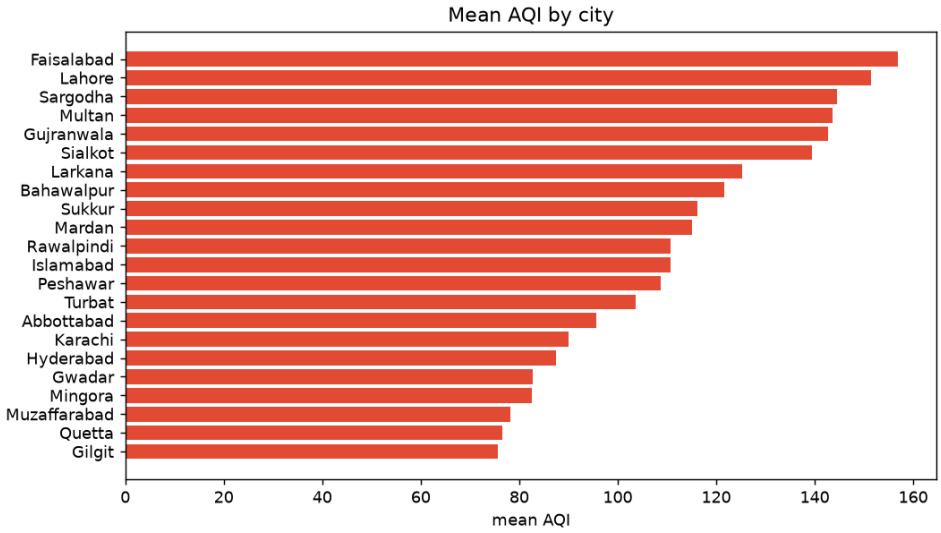

Figure 5.4: Mean Air Quality Index by city over the full record.

Faisalabad outranks Lahore is a direct argument for the multi-city scope of the project, since the city that receives essentially all of the public and academic attention is not in fact the worst affected.

5.6.2 Seasonality Is the Dominant Signal

Figure 5.5 shows the monthly profile. The winter peak is unmistakable: January is the worst month at a mean of 146.8, against 91.5 in April, the cleanest - a 60% difference between the extremes of the year, and a winter-to-summer ratio of about 1.25. This is consistent with the meteorological mechanism of the smog season: temperature inversions that trap emissions under a shallow mixing layer.

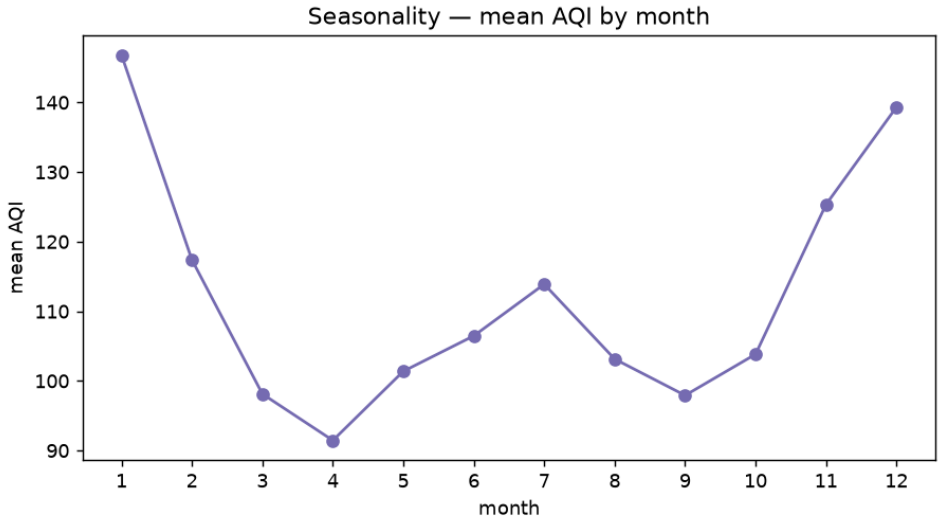

Figure 5.5: Monthly seasonality of the Air Quality Index. The winter smog season dominates the annual cycle.

This finding produced two design decisions. It motivated the cyclical encoding of month, so that December and January are adjacent in feature space rather than maximally distant. And it established, before any model existed, that the system would experience severe recurring input drift twice a year - which is why drift monitoring was specified as a requirement and why its output is interpreted with the seasonal explanation in mind rather than treated as a fault.

5.6.3 There Is a Strong Diurnal Cycle

Figure 5.6 shows the average profile across the hours of the day. The maximum falls in the evening, around 18:00 at a mean of 123.1, and the minimum in the morning, around 09:00 at 108.1 - a within-day swing of 15 index points, consistent with boundary-layer behaviour and the evening traffic peak.

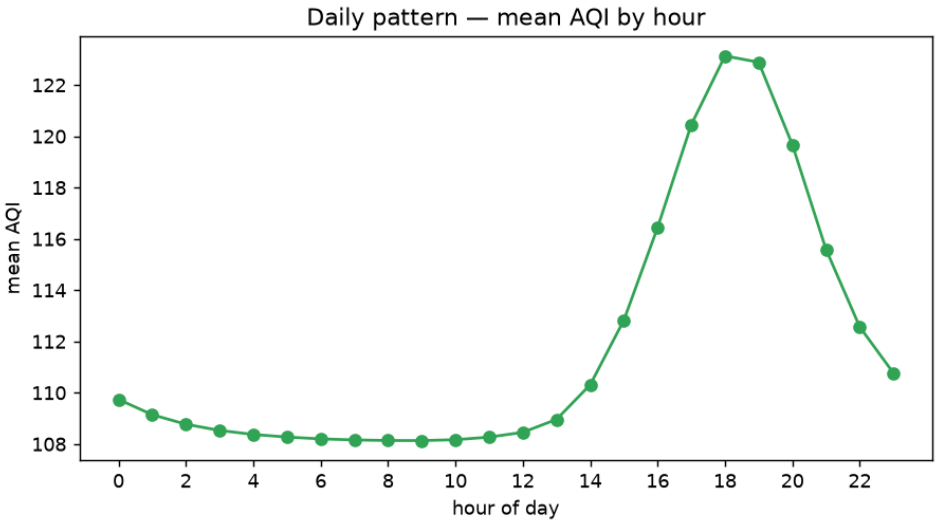

Figure 5.6: Average diurnal profile of the Air Quality Index.

This motivated the cyclical encoding of hour, and it also motivated an interface decision: because the within-day variation is large, an hourly view is genuinely useful to a user, not merely decorative, since it lets them identify a safer part of the day rather than writing off the day entirely.

5.6.4 Weather Is Informative but Not Individually Decisive

Figure 5.7 shows the correlation of each weather variable with the index. The strongest single relationship is with surface pressure, at a correlation of about 0.32 - meaningful, but far from sufficient on its own.

This is an argument for a non-linear model with interactions rather than a linear one. No single weather variable determines air quality; what matters is the combination - low wind together with high pressure and a cold night is the inversion signature - and that

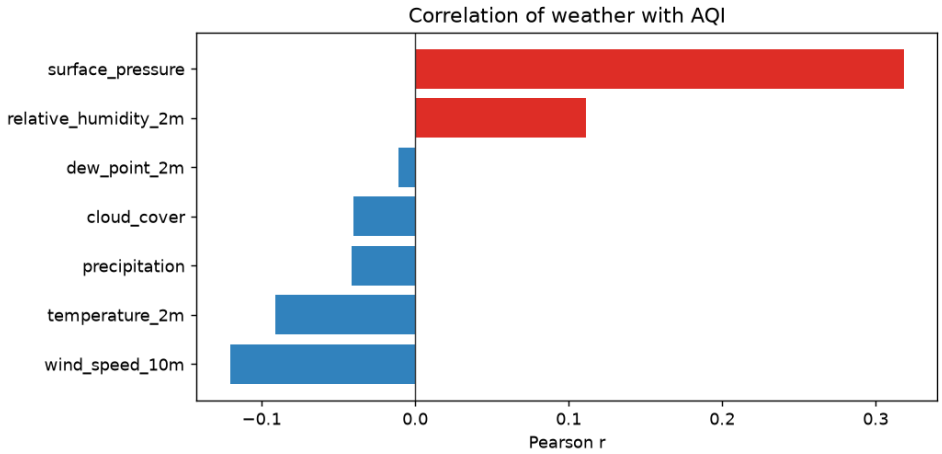

Figure 5.7: Correlation between weather variables and the Air Quality Index.

is precisely the kind of structure a tree ensemble captures and a linear model does not. The eventual gap between the Ridge and XGBoost candidates, reported in Chapter 8, confirms this reading.

5.6.5 The Target Distribution Is Skewed

Figure 5.8 shows the distribution of the index over the whole record: a long right tail, with the bulk of the mass in the moderate-to-unhealthy range and occasional excursions into the hazardous band.

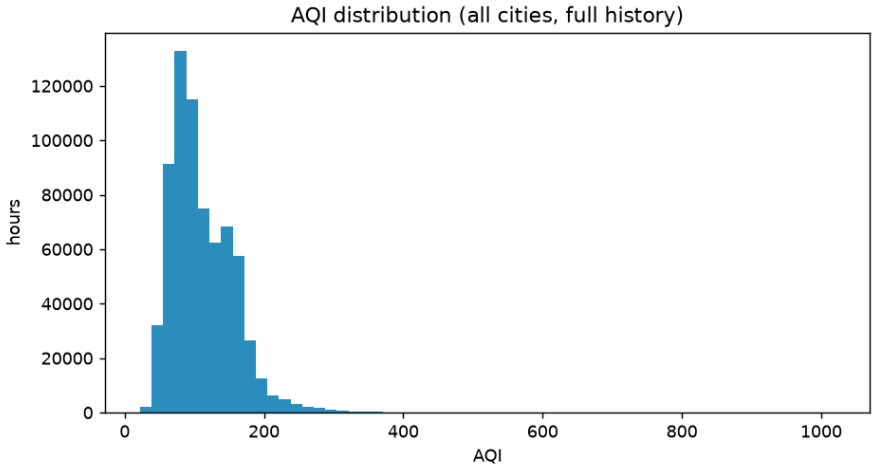

Figure 5.8: Distribution of the Air Quality Index across the whole record.

The skew has an important consequence for the interval design: a symmetric Gaussian interval would be a poor description of the error, which motivated the use of empirical residual quantiles rather than a parametric interval.

5.7 Feature Design

Figure 5.9 specifies the transformation from raw observations to model inputs.

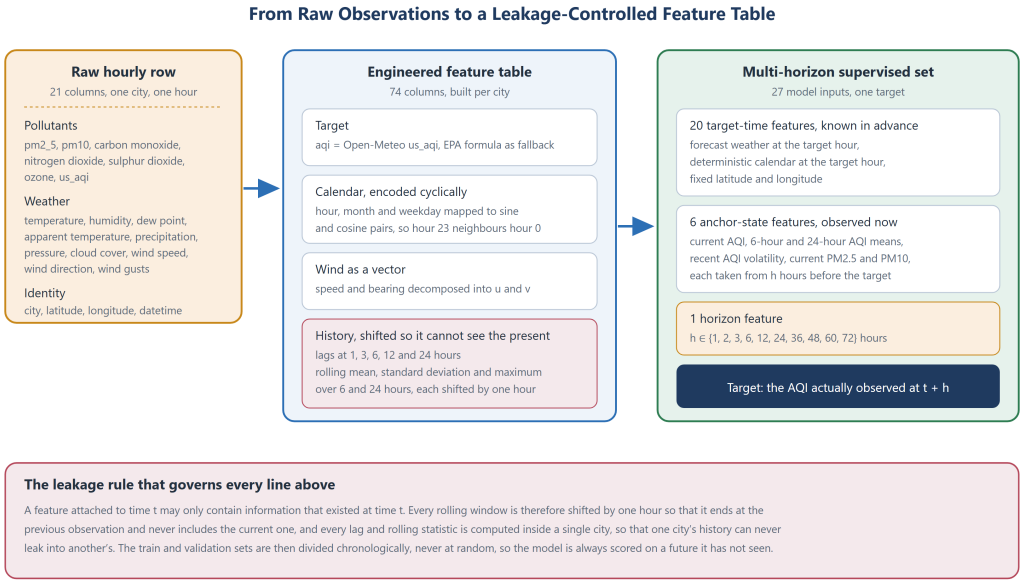

Figure 5.9: Feature design: 21 raw columns become a 74-column engineered table, from which a 27-input multi-horizon supervised set is assembled. The rule at the foot governs every step.

5.7.1 Feature Families

Four families of engineered feature are constructed, each for a stated reason.

Cyclical calendar features. Hour, month and day of week are each mapped to a sine and cosine pair. Encoding hour as the integer 23 places it maximally far from hour 0, which is false; the sine-cosine pair places them adjacent on a circle, which is true.

Wind as a vector. Wind direction is a bearing, so 359 degrees and 1 degree describe almost the same wind while differing maximally as numbers. Speed and bearing are therefore decomposed into orthogonal components, which the model can reason about linearly.

Lagged values. The index and both particulate species are lagged by 1, 3, 6, 12 and 24 hours, since pollutant series are strongly autocorrelated and the 24-hour lag additionally captures the diurnal cycle.

Rolling statistics. Mean, standard deviation and maximum over 6-hour and 24-hour windows summarise recent level, volatility and peak.

5.7.2 The Leakage Discipline

Every rolling window is shifted by one observation before it is computed, so that it ends at the previous hour and can never include the current one. Every lag and rolling statistic is computed within a single city, by grouping before engineering, so that one city's history can never leak into another's. Both properties are asserted by unit tests: one checks that each city independently shows exactly 24 missing values in its 24-hour lag column, which can only be true if the groups were processed separately, and another checks that the anchor value at horizon h equals the observed value h steps earlier.

5.7.3 The Target

The target is the EPA index. Where the provider supplies its own correctly averaged index value, that is used directly; where it is missing, the index is computed from first principles from particulate matter. Gaseous pollutants are deliberately excluded from the computed fallback, because the EPA defines their breakpoints in volumetric mixing ratios while the provider reports mass concentrations, and applying one to the other without a molar-mass conversion inflates the sub-index absurdly. Since the index is the maximum over sub-indices, a single wrong sub-index would corrupt every affected row. The exclusion is tested explicitly.

5.8 Multi-Horizon Forecasting Design

The central modelling decision is illustrated in Figure 5.10.

5.8.1 The Two Kinds of Input

A forecast issued at time t for the hour $\tau = t + h$ can draw on two distinct kinds of information, and the design separates them explicitly.

Target-time features describe the hour being predicted and are known in advance: the weather at τ, which is available as a forecast; the calendar at τ, which is deterministic; and the location, which is fixed. There are 20 of these.

Anchor-state features describe the observed pollution at the moment the forecast is issued: the current index, the 6-hour and 24-hour index means, the recent index volatility, and the current particulate concentrations. There are six of these, and they are obtained by shifting each series by h, so that the row describing τ carries the state as it was h hours earlier.

The horizon h itself is the twenty-seventh input. The model therefore learns

$$\mathrm{AQI}(\tau) \;=\; f\big(\text{target-time features at } \tau,\ \text{anchor state at } \tau - h,\ h\big).$$

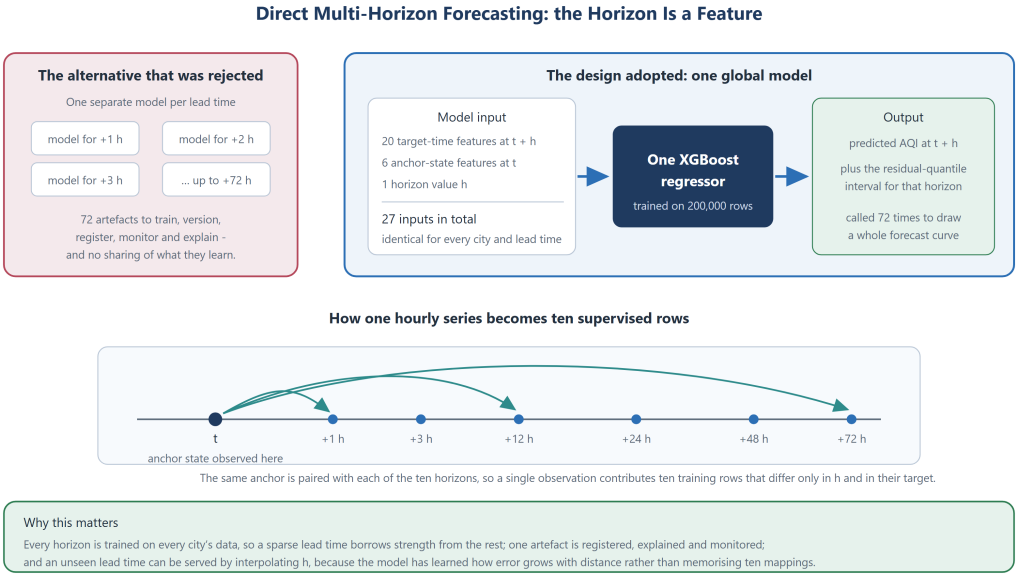

Figure 5.10: Direct multi-horizon forecasting with the horizon as an input feature: one global model serves 22 cities and ten lead times.

5.8.2 Why the Horizon Is a Feature

Making h an input rather than training one model per horizon has four consequences, all favourable. There is one artefact to version, register, explain and monitor, rather than ten or 220. Every horizon is trained on the whole dataset rather than on a tenth of it, so the long horizons - which are the sparsest and the hardest - borrow strength from the short ones. The model learns how uncertainty grows with distance as a smooth function of h rather than memorising ten unrelated mappings, so an unmodelled lead time can be served by interpolation. And unlike a recursive strategy [12], the model is never fed its own output, so errors do not compound across steps.

The design also incurs an honest cost, which Chapter 8 quantifies: at training time the model sees the weather that actually occurred at τ, whereas at inference it sees the forecast weather for τ. The validation score is therefore a mild over-estimate of operational skill.

5.8.3 The Supervised Set and Its Split

Ten horizons - 1, 2, 3, 6, 12, 24, 36, 48, 60 and 72 hours - are materialised. Each observation therefore contributes up to ten supervised rows differing only in h and in the target. The resulting set is subsampled to 200,000 rows with a fixed seed, which keeps a full four-candidate training run inside the scheduler's time limit while retaining ample data.

The split is strictly chronological: the most recent fifth of the time range becomes validation, and everything before it becomes training. Random partitioning is not used at any point, in accordance with the literature [11] and with principle P3.

5.9 Prediction Interval Design

A point forecast alone would violate NFR-4. Intervals are constructed in the spirit of split conformal prediction [24, 25]: the model is fitted on the training window, its residuals are computed on the disjoint validation window, and the interval is formed from the empirical quantiles of those residuals.

Two refinements are applied. The quantiles are computed *per horizon*, so that the interval widens with lead time exactly as the model's real errors do; a single global interval would be far too wide at one hour and far too narrow at 72. And the quantiles are empirical rather than parametric, which respects the skew observed in Section 5.6 instead of assuming a symmetric error distribution. The tenth and ninetieth percentiles are used, giving a nominal 80% interval. At inference the bounds are clipped to the valid index range, and a horizon that was not explicitly modelled takes the quantiles of the nearest modelled horizon.

The construction's assumption - exchangeability between the calibration and deployment periods - is not strictly satisfied by a seasonal, non-stationary series, so the 80% figure is a well-motivated estimate rather than a guarantee. Chapter 8 states this limitation explicitly rather than claiming coverage the construction does not earn.

5.10 Model Selection and Promotion Design

Four candidate families are scored in every run: a persistence baseline, which predicts that the index at $t+h$ equals the index at t; a standardised Ridge regression; a random forest; and a gradient-boosted ensemble. The baseline is scored but is never eligible for promotion; its role is to establish that the learned models have skill at all [9]. The linear model's role is diagnostic, since the gap between it and the ensembles measures how much of the relationship is non-linear.

The best fitted candidate by validation RMSE becomes the challenger, and is promoted only if it improves on the reigning champion's validation RMSE. The comparison margin is zero, which means an exact tie is a refusal: a new model never ships merely because it is newer. Every run appends an entry to a persistent leaderboard recording the version, the timestamp, the winning family, its metrics, the metrics of every candidate considered, the previous champion's score and the decision. The design and its observed behaviour are developed in Section 6.13.

5.11 Monitoring Design

The monitoring design pairs two measurements that are individually insufficient.

Input drift is measured by the Population Stability Index over seven features - the index itself, both particulate species, temperature, humidity, wind speed and surface pressure - comparing the most recent fourteen days against everything older. Deciles of the reference distribution define the bins, and the conventional thresholds of 0.10 and 0.25 classify each feature as stable, moderate or significant. PSI is non-parametric, cheap, and computed per feature so that the responsible variable is named rather than merely alarmed about.

Realised error is measured directly. Every forecast point issued is appended to a log with the hour it was issued for; once the feature pipeline has ingested the observation for that hour, the log is joined to it on city and target hour, and MAE, RMSE and the five largest misses are computed over the joined rows.

The pairing is the point. Input drift alone cannot distinguish a harmless seasonal transition from a genuine degradation, and for a system whose dominant signal is an annual cycle, a drift alarm is the expected condition for several months of the year. Realised error alone is a lagging indicator: it can only score a forecast once the future it described has arrived. Read together, they separate the two cases - drift with stable error is seasonality that the nightly retrain has absorbed; drift accompanied by rising error is decay.

5.12 Communication Design

The system exposes one HTTP interface, summarised in Table 5.1. Its design follows a single rule: endpoints that read a published document are cheap and always available, while endpoints that invoke the model are separate, explicitly labelled, and permitted to fail without affecting the others.

Table 5.1: The HTTP interface.

Endpoint	Kind	Purpose
/api/health	**document**	Liveness, and whether a forecast document is present.
/api/categories	**static**	The six index categories, their bounds and their advisory text, for the legend.
/api/cities	**static**	The supported cities with province and coordinates.
/api/predictions	**document**	The complete forecast payload for every city.
/api/predictions/{city}	**document**	One city's forecast.
/api/leaderboard	**document**	The promotion history and the current champion.

Endpoint	Kind	Purpose
/api/evaluation	**document**	Per-horizon metrics and the walk-forward back-test.
/api/monitoring	**document**	Drift status and realised forecast error.
/api/explain/{city}	**document**	The stored attribution and its sentence.
/api/predict	**model**	An on-demand forecast for an arbitrary coordinate pair.
/api/whatif/defaults	**model**	Current real values for the scenario sliders.
/api/whatif	**model**	Baseline versus scenario under overridden drivers.
/api/chat	**external**	The grounded language-model advisor.

The frontend is a single-page application that fetches these endpoints over HTTPS. Because the two are deployed to different origins, the API permits cross-origin requests; this is safe here because every endpoint is a read of public, non-sensitive data. When no backend address is configured at build time, the same frontend code reads committed snapshots shipped alongside the bundle and hides the two capabilities that genuinely require a live model, which is the graceful degradation required by NFR-11.

Figure 5.11 shows how these pieces interact over time, and makes the essential asymmetry visible: the left-hand portion is a scheduled batch job that may take minutes, while the right-hand portion is a file read that takes milliseconds, and nothing connects them except a document.

5.13 Error Handling and Recovery Design

Failure is treated as ordinary rather than exceptional, and is handled differently at each layer.

At the network boundary, transient provider failures are retried automatically with exponential backoff on the status codes that indicate throttling or a temporary server fault. Without this, a single momentary error would fail an unattended hourly job.

Within a multi-city loop, a failure is contained. If one city's ingestion or forecast raises, the error is logged and the loop continues, so that twenty-one cities are still served rather than none. The backfill goes further and writes each city as it completes, so an interruption loses at most one city's work.

At the storage boundary, writes are idempotent upserts on the primary key and are submitted without blocking on server-side materialisation. This design came directly from a real failure: a blocking write waited indefinitely on a stalled free-tier job and froze an entire backfill run. Because writes are idempotent and the resume logic is history-aware, simply re-running the job is always a safe recovery.

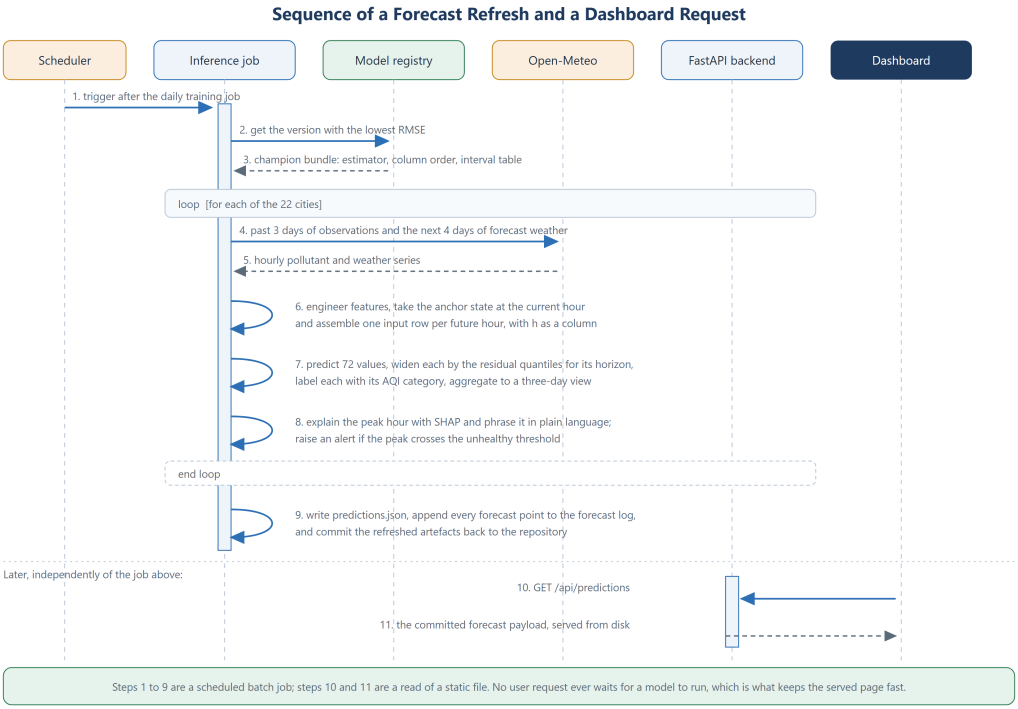

Figure 5.11: Sequence of a scheduled forecast refresh, followed by an independent dashboard request.

At the model boundary, the promotion gate is itself an error-handling mechanism. If a training run produces a bad model - because of a corrupt window, an ingestion gap or any other cause - the gate refuses it and the previous champion continues to serve. There is no path by which an unattended run can degrade the service.

At the auxiliary boundary, each subordinate capability is wrapped so that its failure is logged and execution continues, in accordance with principle P5. The monitoring endpoint carries this furthest, with three levels of fallback: it serves the committed snapshot if present, computes the report live if not, and returns a clear placeholder if neither is possible, so that the tab reports its own state rather than showing an error.

5.14 Deployment View

Figure 5.12 shows the deployment topology and the two modes in which the system can serve.

Four nodes participate. The **repository** holds the source and, uniquely, the published documents and the model bundle, which the nightly workflow commits back to it. The **runners** execute the pipelines and are the only nodes that ever contact the feature store. The **container host** runs the API from a deliberately slim image that omits the feature-store and deep-learning dependencies entirely, because it serves committed documents

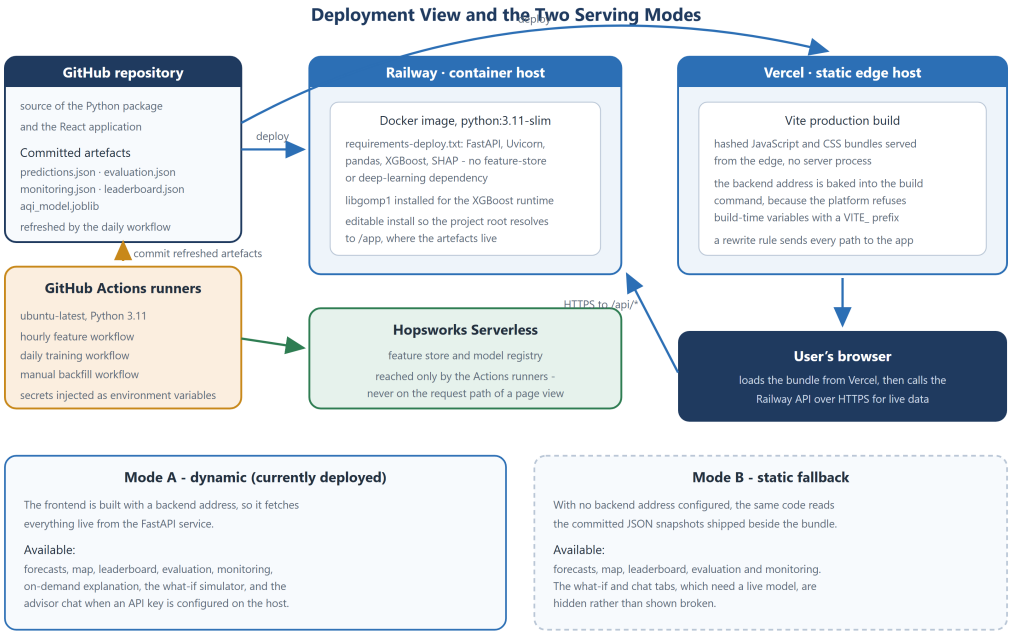

Figure 5.12: Deployment view: the repository, the scheduled runners, the container-hosted API, the statically hosted frontend, and the two serving modes.

and a bundled model rather than querying live state. The **static host** serves the compiled frontend from the edge with no server process at all.

The two serving modes are a direct expression of NFR-11. In *dynamic* mode, which is how the system is currently deployed, the frontend is built with the backend's address and fetches everything live, so the full capability set - including what-if simulation and the advisor - is available. In *static* mode, no backend address is configured, the same frontend code reads the committed snapshots, and the two capabilities that require a live model are hidden rather than shown broken. The second mode is not a degraded accident but a designed fallback: it guarantees that the forecast remains publicly readable even if the container host is unavailable.

5.15 Summary of Design and Architecture

This chapter set out the six principles that governed the design - durable state with disposable compute, decoupling through shared state, strict prevention of temporal leakage, gated promotion, subordination of auxiliary capability, and publication of the system's own working. It presented the three-plane Feature-Training-Inference architecture and the alternatives rejected in reaching it, decomposed the package into layers, and specified the data design around a single upsert-keyed feature group and four published documents. It reported the exploratory analysis that motivated the cyclical encodings, the non-linear model class, the empirical interval construction and the drift monitoring. It

then developed the four decisions that define the system: the leakage-controlled feature representation, the horizon-as-feature formulation, per-horizon empirical intervals, and the promotion gate. Finally it specified the HTTP interface, the layered error-handling strategy and the deployment topology with its two serving modes. The next chapter describes how this design was implemented.

Chapter 6

Implementation

6.1 Introduction

This chapter describes how the design of Chapter 5 was built. It begins with the technology stack and the reasoning behind each choice, then follows the data as it moves through the system: ingestion, index computation, feature engineering, the supervised builder, the three pipelines, the model layer, the promotion gate and the registry, inference, explanation, simulation and monitoring. It then covers the serving layer, the continuous-integration workflows and the production deployment, and closes with the failures encountered during development and what each of them required.

Code excerpts are presented as figures, captioned below and centred, in keeping with the convention that the document numbers only figures and tables. Excerpts are lightly abridged for length; the behaviour described is that of the running system.

6.2 Technology Stack

Table 6.1 records the technology choices and their justification.

Table 6.1: Technology stack and the reason for each choice.

Layer	Choice	Justification
Language	Python 3.11	The machine-learning ecosystem; pinned below 3.12 for compatibility across the dependency set at the time of development.
Data source	Open-Meteo [8]	Free, *keyless*, hourly, global, and serving both multi-year history and a multi-day forecast from the same request shape. The absence of a credential removes an entire class of operational failure from the scheduled jobs.
Feature store and model registry	Hopsworks Serverless [33]	A managed free tier providing both halves of the state plane, with primary-key upserts, event time and metric-tagged model versions.
Orchestration	GitHub Actions [43]	Free scheduled compute, co-located with the source, with secret injection and concurrency groups.
Tabular processing	pandas [37]	The standard library for the grouped, shifted, windowed operations that the feature design requires.

Layer	Choice	Justification
Classical models	scikit-learn [36]	Ridge, random forest, metrics, standardisation and the time-series splitter used for the backtest.
Champion model	XGBoost [16]	The strongest candidate measured; trains a 200,000-row, 27-feature problem in about a minute on CPU, which is what makes a nightly retrain feasible.
Sequence-model study	TensorFlow / Keras [38]	Used to build and evaluate the LSTM alternative; deliberately excluded from the deployment image.
Explainability	SHAP (TreeExplainer) [28]	Exact, polynomial-time attributions for tree ensembles, with the additive property that makes plain-language rendering possible.
Backend	FastAPI and Uvicorn [39]	Typed, asynchronous, with request-model validation and automatic interface documentation.
Frontend	React with Vite [40, 41]	Component model suited to a multi-tab dashboard; Vite produces a small static bundle deployable to an edge host.
Charting and maps	Recharts and react-leaflet [42]	Responsive charts with band series for the prediction interval, and an interactive map of Pakistan.
Advisor	Gemini or Groq (free tiers), or Claude, over the Model Context Protocol [46]	Structured tool calling, so answers are grounded in the system's own forecasts. The provider is chosen from whichever API key is present, so the advisor needs no paid account; MCP publishes the same tools to external clients.
Deployment	Docker on Railway; Vercel [44]	A container gives full control of the backend image; the frontend needs no server at all.
Testing	pytest [45]	Unit tests over the index computation, the feature builder and the supervised assembly.

6.3 Configuration as a Single Source of Truth

One module defines everything the rest of the system needs to agree on: the project root and the derived data and model directories, the credentials read from the environment, the feature-store object names and versions, the forecast horizon, and the list of supported cities. Each city is an immutable record of name, coordinates, province and timezone.

The consequence is that adding a city is a single line, and that no module anywhere else in the codebase contains a hard-coded path, city name or object version. The path derivation deserves note because it later mattered in deployment: the project root is computed relative to the configuration module's own location, which is why the deployment image installs the package in editable mode rather than copying it into the interpreter's library directory - an ordinary install would have resolved the root to a location containing none of the committed artefacts.

6.4 Data Ingestion

Ingestion wraps three provider endpoints behind one helper. The air-quality endpoint serves both history and forecast; weather is split between a forecast endpoint and an archive endpoint derived from ERA5 [7], and the helper selects between them according to whether an explicit date range was requested. Seven pollutant variables and ten weather variables are fetched and merged on the observation hour, and the merged frame is tagged with the city name and its coordinates, giving 21 raw columns.

The session is configured to retry, because these jobs run unattended. A momentary throttle or gateway error from the provider would otherwise fail an entire hourly run; instead it is retried with exponential backoff on precisely the status codes that indicate a transient condition, as Figure 6.1 shows.

```
def _session(retries: int = 4, backoff: float = 1.5) -> requests.Session:
    """Retry transient failures: this runs unattended every hour."""
    retry = Retry(
        total=retries,
        backoff_factor=backoff,          # waits 1.5s, 3s, 6s, ...
        status_forcelist=(429, 500, 502, 503, 504),
        allowed_methods=frozenset(["GET"]),
    )
    session = requests.Session()
    session.mount("https://", HTTPAdapter(max_retries=retry))
    return session
```

Figure 6.1: The retrying HTTP session used for every provider request.

6.5 The Air Quality Index

The index module implements the EPA definition [5] directly: the six index bands, the concentration breakpoints per pollutant, the piecewise-linear sub-index formula, the maximum-over-sub-indices rule, and the six health categories with their advisory text. The categories are used throughout the system - for the colour scale, the alert wording and the advisor's answers - so defining them once here keeps the interface, the API and the language model in agreement.

Two implementation decisions in this module matter.

The first is the treatment of gases. As noted in Chapter 2, the EPA specifies gaseous breakpoints in volumetric mixing ratios while the provider reports mass concentrations. Rather than apply an approximate conversion, the computed index is restricted to the two particulate species, whose breakpoints are already expressed in the units the provider uses. This is a defensible restriction on this data - particulate matter dominates the index in every city studied - and it avoids a class of error that would be invisible in aggregate metrics, since the index is a maximum and one wrong sub-index would silently dominate every affected row. The restriction is asserted by a unit test that feeds an enormous

carbon monoxide value alongside clean particulate readings and requires the index to be unaffected.

The second is precedence. Where the provider supplies its own index value it is preferred, because the provider applies the correct EPA averaging windows; the locally computed value is used only to fill gaps. The module thus serves both as the fallback computation and as the documentation of what the predicted number actually means.

6.6 Feature Engineering

The feature builder takes a raw hourly frame and produces the 74-column engineered table specified in Chapter 5. It runs in a fixed order, because the change, lag and rolling families all require the target column to exist first.

The two properties that make the feature table trustworthy are visible in Figure 6.2. Every rolling window is shifted by one observation before aggregation, so that it terminates at the previous hour, and every family is computed inside a single city, because the multi-city entry point groups by city and engineers each group independently before concatenating.

```
def _add_rolling_features(df: pd.DataFrame) -> pd.DataFrame:
    # The .shift(1) is the anti-leakage guard: the window ends at the
    # PREVIOUS hour and never includes the current observation.
    for col in _SERIES_COLS:
        for win in _ROLL_WINDOWS:
            roll = df[col].shift(1).rolling(
                win, min_periods=max(2, win // 2))
            df[f"{col}_roll_mean_{win}h"] = roll.mean()
            df[f"{col}_roll_std_{win}h"]  = roll.std()
            df[f"{col}_roll_max_{win}h"]  = roll.max()
    return df


def build_features_all_cities(raw_all, *, dropna_target=True):
    # Lags must never reach across a city boundary: Lahore's value from
    # 24 hours ago is meaningless for Karachi.
    parts = [build_features(g, dropna_target=dropna_target)
             for _, g in raw_all.groupby("city", sort=False)]
    return pd.concat(parts, ignore_index=True)
```

Figure 6.2: The two mechanisms that prevent leakage: a shifted rolling window and per-city grouping.

The builder also computes the feature group's primary key, converting the observation datetime to epoch seconds by subtracting the Unix epoch and dividing by a one-second interval. This slightly indirect formulation was adopted because the direct conversion helper's behaviour differs across major pandas versions, and the primary key must be stable for the upsert semantics to hold.

6.7 The Multi-Horizon Supervised Builder

The supervised builder implements Section 5.8. For each city it sorts by time and then, for each of the ten horizons, constructs a block in which the target-time features are taken from the current row, the anchor-state features are taken from h rows earlier by shifting, the horizon column is set to h, and the target is the index at the current row. Blocks are concatenated, rows with an incomplete anchor are dropped, and the result is subsampled with a fixed seed. Figure 6.3 shows the core.

```
for city, g in features.groupby("city", sort=False):
    g = g.sort_values("datetime").reset_index(drop=True)
    for h in horizons:
        block = pd.DataFrame()
        for col in TARGET_TIME_FEATURES:     # known for the future hour tau
            block[col] = g[col]
        for col in ANCHOR_STATE_FEATURES:    # observed h hours before tau
            block[f"{col}_anchor"] = g[col].shift(h)
        block["horizon"] = h                 # the horizon IS a feature
        block[TARGET_COLUMN] = g["aqi"]      # AQI at tau
        block["city"], block["datetime"] = city, g["datetime"]
        blocks.append(block)
```

Figure 6.3: Assembly of the multi-horizon supervised set. One observation contributes up to ten rows, differing only in the horizon and the target.

The chronological split is implemented as a quantile of the datetime column rather than a positional cut, so that it remains correct even when the rows are unevenly distributed in time after subsampling. The datetime column is coerced to a datetime type inside the split function rather than being assumed, for a reason discovered in production and discussed in Section 6.24.

6.8 The Feature Pipeline

The hourly pipeline is deliberately short: it fetches the last seven days for all 22 cities, engineers features per city, and upserts the result. Its two non-obvious properties are both correctness properties.

It fetches seven days rather than one hour, because the lag and rolling features for the newest observations need history behind them; requesting only the newest hour would produce a row whose 24-hour features were all missing. The resulting overlap with data already stored is not a problem, because the write is an upsert on the composite key: re-running the pipeline, or running it twice in the same hour, cannot create a duplicate row. Idempotence of this kind is what allows the recovery strategy for every failure in this system to be “run it again”.

6.9 The Backfill Pipeline

The backfill runs the same feature logic over the full history. It is triggered manually and it is the component that required the most hardening, because it is the only job that runs for hours and touches every city.

It fetches each city's history in three-month chunks and concatenates them before engineering features, so that lags and rolling windows are computed over a contiguous series rather than being reset at every chunk boundary. It writes each city as that city completes, rather than accumulating everything and writing once, so that an interruption costs at most one city. It catches per-city exceptions and continues, so that one failing city does not abort the other twenty-one.

Its resume logic is history-aware, and the distinction matters. An earlier version skipped any city already present in the store; but the hourly pipeline seeds every city with recent rows within an hour of being enabled, so on the next run every city looked present and the backfill skipped all of them, silently leaving the store with a week of history instead of three years. The corrected logic asks a different question - does this city's earliest stored observation reach back to the requested start date? - and only then treats the city as complete. Figure 6.4 shows it.

```
# History-aware resume: a city counts as done only if its data
# already reaches back to `start`. A city with merely recent rows
# (seeded by the hourly pipeline) is NOT done and gets backfilled.
coverage = city_coverage()          # {city: earliest date present}
done = {c for c, earliest in coverage.items()
        if earliest is not None and earliest <= start_date}
partial = {c for c in coverage if c not in done}
```

Figure 6.4: History-aware resume in the backfill. Presence in the store is not evidence of completeness.

6.10 The Storage Layer

The storage facade exposes four operations and selects a backend by whether a credential is configured, degrading to the local columnar backend when the managed client is not installed. That second condition is not hypothetical: on the development platform the managed client cannot be installed at all, because one of its transitive cryptographic dependencies fails to build there. The facade turns a hard platform incompatibility into a configuration detail.

The managed backend concentrates every vendor-specific call in one module. Three of its choices were forced by the free tier and are recorded here as findings. The feature group is created with a time-travel table format that materialises server-side, because the platform's newer default format requires a client library that the base installation does not ship, and group creation fails outright without it. Statistics computation is disabled,

because the profiler over 74 columns exhausted the free executor's memory and failed the materialisation job; the statistics are used only by the platform's own interface and by no pipeline. And inserts do not block on materialisation, for the reason given in Figure 6.5.

```
def insert_features(project, df, *, wait: bool = False) -> None:
    """Upsert features (non-blocking by default).

    On the free tier a Spark executor occasionally hangs at
    INITIALIZING; a blocking insert then freezes the whole backfill
    until the scheduler kills it at its time cap. Non-blocking is
    safe here because inserts are idempotent primary-key upserts
    and the backfill resumes by coverage.
    """
    fg = get_feature_group(project)
    fg.insert(_sanitize(df), write_options={"wait_for_job": wait})
```

Figure 6.5: Non-blocking, idempotent feature insertion.

6.11 The Training Pipeline

The daily training pipeline implements use case UC-5. It reads the whole feature table, builds the supervised set, splits it chronologically, scores the persistence baseline by simply reading the anchor column, fits the three learned candidates, and selects the challenger by validation RMSE.

The candidate configurations are fixed rather than searched, because a nightly unattended job that performs a hyper-parameter search is both slower and less reproducible than one that does not, and because the measured gap between the families is far larger than the gap between plausible configurations within a family. The gradient-boosted candidate uses 600 trees at a learning rate of 0.05, a maximum depth of 8, and row and column subsampling at 0.8 with the histogram tree method; the forest uses 120 trees at a maximum depth of 16 with a minimum leaf size of 10; the linear candidate is a standardised Ridge regression. Every candidate has a fixed random seed, satisfying NFR-7.

Prediction intervals are fitted immediately after selection, on the same validation window, by grouping the residuals by horizon and taking the tenth and ninetieth percentiles within each group, as Figure 6.6 shows.

```
valid["pred"]  = best.predict(X_va)
valid["resid"] = valid[TARGET_COLUMN] - valid["pred"]
interval_by_horizon = {
    int(h): [float(g["resid"].quantile(0.1)),
             float(g["resid"].quantile(0.9))]
    for h, g in valid.groupby("horizon")
}
```

Figure 6.6: Per-horizon empirical residual quantiles, giving an 80% prediction interval that widens with lead time.

Everything inference needs is then packaged into a single bundle: the fitted estimator, the exact feature column order, the modelled horizons, the interval table, the metrics of every candidate, the winning family's name and the training timestamp. Shipping the column order with the estimator rather than relying on both sides importing the same constant is a small but deliberate guard against training-serving skew.

6.12 The Evaluation Suite

Evaluation is implemented as a separate module rather than inside the training pipeline, because it answers a different question. Training asks which candidate is best; evaluation asks how good the promoted model actually is, and at what lead time.

Per-horizon metrics are computed by predicting once over the whole validation set and then masking by horizon, so that RMSE, MAE and R^2 are reported separately for each of the ten modelled lead times. The persistence baseline is scored under exactly the same mask, which is what makes the comparison meaningful: a claim to beat a baseline at 72 hours is only informative if the baseline was also evaluated at 72 hours.

The walk-forward backtest uses an expanding-window time-series split with five folds. Each fold trains a fresh gradient-boosted model on the folds that precede it and tests on the fold that follows, so that every test window lies strictly in the future of its own training data. This is the construction that the literature [10, 11] identifies as the honest estimate of forward performance, and it is deliberately more pessimistic than the single chronological split used for candidate selection, because each early fold is trained on far less data.

The module writes four artefacts: two figures, a markdown report, and a JSON document that the API serves and the dashboard renders. Publishing the same numbers into a document that reaches the public interface, rather than into a log that only a developer reads, is what satisfies principle P6. The results themselves are presented in Chapter 8.

6.13 The Champion-Challenger Gate

The gate is the component that turns a retraining script into a governed system. Figure 6.7 illustrates it, and Figure 6.8 shows the rule.

Three details are worth drawing out. The comparison is a strict inequality, so a challenger that merely matches the champion is refused; a model does not ship because it is newer. The first ever run is promoted unconditionally, because there is no incumbent to beat. And every run appends an entry recording not only the winner but the metrics of every candidate considered, the previous champion's score and the decision - so the leaderboard is an audit trail rather than a scoreboard.

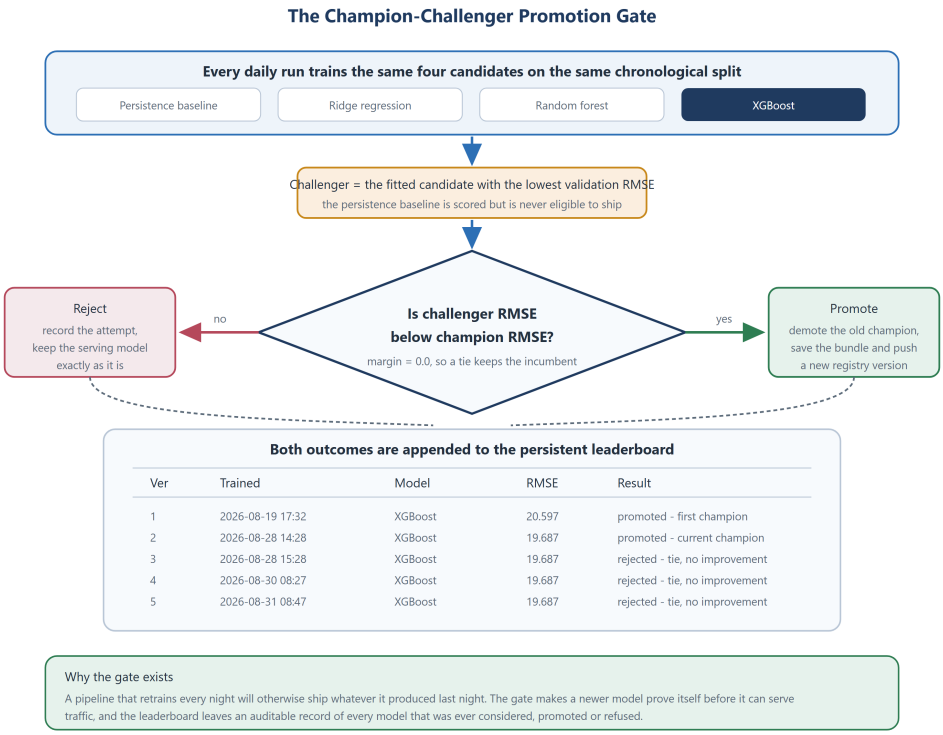

Figure 6.7: The promotion gate. Both outcomes are recorded; the leaderboard rows are the system's actual history.

```
PROMOTION_MARGIN = 0.0   # 0 = any improvement promotes; a tie does NOT

champion   = current_champion(entries)
challenger = candidates[best_name]
version    = max((e.get("version", 0) for e in entries),
                 default=0) + 1

promoted = champion is None or \
           challenger["rmse"] < champion["rmse"] - PROMOTION_MARGIN
if promoted:
    for e in entries:            # demote the previous champion
        e["is_champion"] = False
```

Figure 6.8: The promotion rule. Strict inequality means an exact tie keeps the incumbent.

The gate's behaviour in production, shown in the lower half of Figure 6.7, is the clearest demonstration that it works. Version 1 was promoted at an RMSE of 20.597 as the first champion. Version 2, trained after the full historical backfill had completed, reached 19.687 and displaced it. Version 3, trained an hour later on effectively identical data, reached the same 19.687 - and was *refused*, because 19.687 is not less than 19.687. That refusal is the system declining to churn its own production model for no gain.

Only if the gate promotes is the bundle written, and the write goes to two places: a local file for use within the same job, and the durable model registry. The registry is what satisfies principle P1. A model saved only to the runner's disk ceases to exist minutes later; a model registered with its metrics can be retrieved by any later job, including a

deployed backend that was never present when training happened. Retrieval asks the registry for the version with the lowest RMSE, which makes "the champion" a property of the stored metrics rather than of a mutable pointer that could drift out of step.

6.14 The Model Layer and the Sequence-Model Study

The registry facade presents one pair of operations - save and load - and hides which backend answers them, preferring the durable registry and falling back to the local bundle. This is what allows the identical inference code to run on a continuous-integration runner that has a registry credential and inside a deployment container that deliberately has neither the credential nor the client.

Alongside the production model, a recurrent network was implemented to test the assumption that a sequence model would do better. It consumes 48 hours of pollution and weather history for a city as a sequence and predicts the index 24 hours ahead, through two stacked LSTM layers followed by a dense head with dropout, trained with early stopping on a chronological validation split. Sequences are built per city with a stride, windows containing missing values are discarded, and scaling is fitted on the training portion only so that no validation statistic leaks backwards.

The comparison was made fair deliberately: the same task, the same split, the same metric, and the same persistence baseline. The results are reported in Chapter 8; the outcome was that the sequence model is genuinely competitive at the single horizon it was built for and was nonetheless not promoted, for reasons of coverage, cost and explainability that Chapter 8 sets out. The module remains in the repository as evidence, and is excluded from the deployment image so that its dependency never reaches production.

6.15 The Inference Pipeline

Inference loads the champion once and then processes each city independently. For a city it fetches the past three days and the next four days in a single request, engineers features over the combined series - so that the anchor's rolling statistics are computed from real history rather than being unavailable - and splits the result at the current hour. The last row at or before now becomes the anchor; the first 72 rows after now become the targets.

The model input matrix is then assembled column by column: target-time features from the future rows, anchor-state features broadcast from the single anchor row, and a horizon column computed as the whole number of hours between each future row and now. The matrix is reordered to the bundle's stored column order before prediction, which is the guard against skew mentioned earlier. Figure 6.9 shows the assembly.

```python
X = pd.DataFrame(index=future.index)
for col in TARGET_TIME_FEATURES:      # forecast weather + calendar at tau
    X[col] = future[col].values
for col in ANCHOR_STATE_FEATURES:     # observed state now, broadcast
    X[f"{col}_anchor"] = anchor[col]
X["horizon"] = ((future["datetime"] - now) / pd.Timedelta("1h")) \
                .round().astype(int).values
X = X[bundle.feature_columns]         # exact training column order
preds = bundle.estimator.predict(X)
```

Figure 6.9: Assembling 72 model inputs for one city from one anchor row and 72 future rows.

Each prediction is then widened by the residual quantiles stored for its horizon, clipped to the valid index range, and labelled with its health category; the hourly series is aggregated into three calendar days; the alert module scans the curve; and the peak hour is explained. Every city is wrapped in an exception handler, so one city's failure costs one city.

The whole payload is written as a single document and, separately, every forecast point is appended to the forecast log for later scoring. Both the explanation and the logging steps are wrapped so that their failure cannot prevent the forecast from being published, in accordance with principle P5.

6.16 Explainability

Explanation uses TreeSHAP [28] against the champion estimator. For a single input row it computes the exact Shapley values, takes the model's base value, and reports the five largest contributions by absolute magnitude. Because the attribution is additive, the base value plus the sum of all contributions is exactly the prediction, which means the explanation can be phrased arithmetically rather than as a bare ranking.

Two implementation choices make the output usable by a non-specialist. The explainer is cached across calls within a process, since constructing it is the expensive part and inference explains 22 cities in one job. And every feature name is mapped through a dictionary of human-readable labels, so that `aqi_roll_mean_6h_anchor` is presented as "6h AQI trend" and `apparent_temperature` as "feels-like temp". Figure 6.10 shows the rendering, and the sentence it produced for Lahore in the live system is visible in the dashboard screenshot in Chapter 7: *"Forecast AQI ≈ 200 (baseline 132). Main drivers: current AQI +26.0, time of day +13.4, season −6.6, 6h AQI trend +6.4, temperature +5.0."*

A global view is produced separately by averaging absolute Shapley values over a sample and plotting the leading features; the resulting ranking is reported in Chapter 8.

```
shap_vals = np.asarray(explainer.shap_values(X))[0]
base = float(np.ravel(explainer.expected_value)[0])
pred = base + float(shap_vals.sum())   # additive: this is exact

order = np.argsort(np.abs(shap_vals))[::-1][:top_k]
contributors = [
    {"label": friendly(bundle.feature_columns[i]),
     "impact": round(float(shap_vals[i]), 1)} for i in order]
parts = [f"{c['label']} {'+' if c['impact'] >= 0 else '-'}"
         f"{abs(c['impact'])}" for c in contributors]
text = (f"Forecast AQI ~ {round(pred)} (baseline {round(base)}). "
        f"Main drivers: " + ", ".join(parts) + ".")
```

Figure 6.10: Rendering Shapley attributions as a plain-language sentence. Additivity is what allows the numbers to be stated arithmetically.

6.17 The What-If Simulator

The simulator exposes five drivers: current particulate concentration, current index, wind speed, humidity and temperature. It first builds the model input for the selected city at a lead time of 24 hours from live data, and returns the current real value of each driver, so that the interface's sliders begin at reality rather than at an arbitrary midpoint. The user's overrides are then mapped onto the underlying feature columns and the model is called twice, once unmodified and once overridden, and both values are returned with their health categories and the difference between them.

The mapping deserves comment because it is where the simulator earns its plausibility. Overriding the current index sets not only the anchor index but also the 6-hour and 24-hour anchor means, since a stated current level that contradicts its own recent history would place the model far outside its training distribution. Overriding the wind speed rescales both wind vector components so that their resultant matches the requested speed while preserving direction, and adjusts the gust feature proportionally. Overriding the temperature adjusts the apparent temperature with it. Each of these keeps the perturbed row internally consistent, which is what makes the response interpretable.

The simulator is labelled in the interface as a model simulation rather than causal evidence, in accordance with NFR-6. What it shows is how *this model* responds to a change in its inputs, which is a genuine and useful statement about the model, and not a claim about what would physically happen.

6.18 Monitoring

Monitoring implements the design of Chapter 5 in two halves, shown in Figure 6.11.

The drift half divides the feature history at fourteen days before the latest observation, treats everything older as the reference and everything newer as the current window, and computes the Population Stability Index per monitored feature. The implementation, in Figure 6.12, takes the bin edges from deciles of the reference distribution, deduplicates

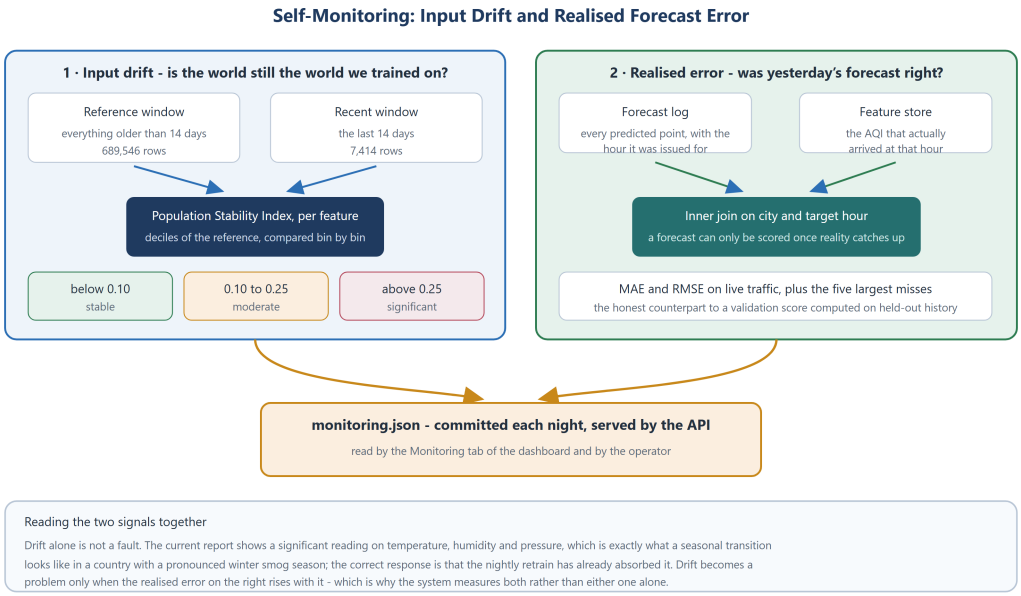

Figure 6.11: The two monitoring signals and why they are read together.

them so that a degenerate distribution cannot produce empty bins, replaces the outer edges with infinities so that new extreme values fall inside the outermost bins rather than being dropped, and adds a small constant to every proportion so that an empty bin cannot produce an infinite logarithm. It returns zero rather than a spurious value when either window is too small to support the estimate.

```
def population_stability_index(ref, cur, bins: int = 10) -> float:
    """PSI: <0.1 stable, 0.1-0.25 moderate, >0.25 significant."""
    ref, cur = ref.dropna(), cur.dropna()
    if len(ref) < 50 or len(cur) < 20:
        return 0.0
    edges = np.unique(np.quantile(ref, np.linspace(0, 1, bins + 1)))
    if len(edges) < 3:
        return 0.0
    edges[0], edges[-1] = -np.inf, np.inf   # absorb unseen extremes
    ref_pct = np.histogram(ref, edges)[0] / len(ref) + 1e-6
    cur_pct = np.histogram(cur, edges)[0] / len(cur) + 1e-6
    return float(np.sum((cur_pct - ref_pct)
                        * np.log(cur_pct / ref_pct)))
```

Figure 6.12: The Population Stability Index, with the guards that keep it defined on real data.

The error half appends every issued forecast to a columnar log, deduplicated on the triple of issue time, city and target hour, and later joins that log to the observed index on city and target hour. Only rows for which reality has arrived can be scored, so the report distinguishes between having no forecasts logged, having forecasts logged but no matured outcomes, and having a real score - rather than reporting a misleading zero in any of those cases.

Both halves are written to a committed document each night, which is what allows the deployed backend to serve the monitoring tab without any live feature-store connection.

6.19 Alerting

The alerting module scans a city's hourly forecast and produces a graded alert. Two thresholds are used, aligned with the EPA category boundaries: 150, the start of the *unhealthy* band, raises a warning, and 200, the start of *very unhealthy*, raises a danger. The alert records the peak value, the hour of the peak, the category and its standard advice, and - the field that makes it actionable - the first hour at which the curve crosses the warning threshold, so a user is told when the episode begins rather than only how bad it will get.

6.20 The API

The backend exposes the endpoints listed in Table 5.1. Its structure follows the design rule that document reads and model invocations are separate. Document endpoints read a committed file and are cheap; the model is loaded lazily and only by the endpoints that genuinely need it, so a deployment that never receives a what-if request never loads the estimator at all.

The monitoring endpoint illustrates the layered error handling of principle P5 in its final form. It first serves the committed snapshot; failing that it attempts to compute the report live; failing that it returns a well-formed placeholder explaining that monitoring runs in the training pipeline and a snapshot will appear after the next run. The tab therefore reports its own state honestly and never presents an error to a member of the public.

Cross-origin requests are permitted because the frontend is served from a different origin and every endpoint is a read of public data. The advisor endpoint checks for its credential before attempting anything and returns a specific, actionable message when it is absent, rather than failing obscurely inside the client library.

6.21 The Advisor and the Tool Server

The advisor answers natural-language questions by tool use rather than by recall. Four functions are declared to the model with JSON schemas: list the supported cities, get a city's current index and three-day forecast with its alert, get a city's long-run historical statistics, and explain why a city's forecast is high or low. The model is invited to call them; the backend executes the call against the real forecast document and returns the result; the loop repeats up to a bounded number of rounds until the model produces a final answer. The system prompt instructs the model to ground every number in a tool call and to answer with practical health guidance - who is at risk, whether to limit outdoor exertion, whether a mask is warranted, and when the cleanest part of the day is.

The same four functions are published over the Model Context Protocol [46] by a small server module, so that an external protocol client can query Pakistani air quality through exactly the same code path that serves the dashboard's advisor. Sharing the implementation rather than duplicating it is what guarantees the two cannot disagree.

6.22 Continuous Integration and Scheduling

Three workflows constitute the compute plane.

The **feature workflow** runs hourly at five minutes past, checks out the repository, installs the package, and runs the feature pipeline with the feature-store credentials injected from the secret store.

The **training workflow** runs daily at 02:30 UTC and executes four steps in order: train, infer, evaluate and monitor, then commit the refreshed artefacts back to the repository. Only the training step can fail the workflow; the rest run unconditionally. That is a deliberate consequence of principle P2: inference reads the provider and the model registry but never the feature store, so a feature-store outage during training must not also cost the site its daily forecast refresh. Evaluation and monitoring, which do read the feature store, are allowed to fail quietly for the same reason. The commit message carries a skip marker so that committing artefacts cannot trigger another run.

The **backfill workflow** is manual, takes the history start date as an input, and is given a much longer time allowance.

Two mechanisms keep these safe. The feature and backfill workflows share a concurrency group, so the scheduled hourly job and a long manual backfill can never write to the feature store simultaneously; and every workflow has an explicit timeout, so a stalled job cannot consume the free allowance indefinitely.

The commit step is what closes the loop between the compute plane and the serving layer: the refreshed forecast, evaluation, monitoring and leaderboard documents become part of the repository, and are therefore present in the next deployment image and readable by the static frontend.

6.23 Deployment

The deployment topology was shown in Figure 5.12.

The backend is a container built from a slim Python base. Three details of the image were arrived at empirically. The OpenMP runtime is installed explicitly, because the gradient-boosting library fails at import time without it on a slim image. The dependency set is a reduced one that deliberately omits the feature-store client and the deep-learning framework, because the deployed API serves committed documents and a bundled model and needs neither; this reduces the image substantially. And the package is installed in

editable mode, so that the project-root path derived in the configuration module resolves to the application directory where the committed documents and the model bundle actually live.

The frontend is a static build served from an edge host. The backend's address is supplied by baking it into the build command rather than by setting a build-time environment variable, because the hosting platform rejects variables carrying the build tool's public prefix as potentially sensitive. Since the address is a public URL, baking it in is appropriate rather than a workaround. A rewrite rule directs every path to the application, so that the single-page router works on a direct navigation.

6.24 Challenges Encountered and Their Resolution

Table 6.2 records the substantive failures encountered during development, their root causes and their resolutions. Several of them shaped the design rather than merely being fixed, and they are the most transferable findings in this thesis, because they are what the free-tier, serverless constraint actually costs.

Table 6.2: Challenges encountered and their resolution.

Challenge	Root cause	Resolution
Feature group could not be created	The platform's newer default table format requires a client library the base installation does not ship.	Created the group with the server-side materialised time-travel format, which preserves primary-key upserts.
Materialisation job failed with an out-of-memory error	The statistics profiler ran over all 74 columns on a free-tier executor.	Disabled statistics on the feature group and created a fresh version; statistics are used only by the platform's interface and by no pipeline.
Backfill froze at the tenth of 22 cities	A blocking insert waited indefinitely on a materialisation job stalled at initialisation, until the scheduler's time cap killed the run.	Made inserts non-blocking and wrote city by city; safe because writes are idempotent upserts and the resume is coverage-based.
Backfill skipped cities that were not in fact loaded	The resume checked whether a city was *present*, but the hourly pipeline had already seeded every city with recent rows.	Changed the resume to check whether a city's earliest stored observation reaches back to the requested start date.
Training crashed in the chronological split	The feature store returns the event time column as strings, so a quantile over it was undefined.	Coerced to a datetime type on read and again inside the split, so the split is correct regardless of how the store returned the column.

Challenge	Root cause	Resolution
The champion and the leaderboard reset on every run	Both were being written to the runner's local directory, which is destroyed when the job ends.	Pushed the champion to the durable model registry and committed the leaderboard to the repository.
The feature-store client could not be installed for local development	A transitive cryptographic dependency fails to build on the development platform.	Introduced the storage facade with an interchangeable local columnar backend; all feature-store work runs on Linux automation.
The container build failed to start	The platform's default builder ignored the declared build configuration and no start command was inferred.	Added a Dockerfile, which the platform detects in preference and which gives explicit control of the base image, the system packages and the entry point.
The deployed backend reported that no forecast was available	The deployment tool honours the repository's ignore file, so the committed model bundle and data documents were excluded from the uploaded context.	Added explicit negations for those paths to the ignore file so that the required artefacts are shipped.
The frontend could not be given the backend address	The hosting platform rejects build-time variables carrying the build tool's public prefix.	Baked the address into the build command in the platform configuration; the address is public, so this is appropriate rather than a compromise.
The gradient-boosting library failed at import in the container	The slim base image lacks the OpenMP runtime.	Installed it explicitly in the image.
The nightly training run failed on five of ten nights	The free-tier Arrow Flight query service intermittently drops a large read, surfacing as *FlightUnavailableError: Socket closed* with gRPC status 14. It is not deterministic: the identical read succeeds on other nights.	Retry the read with increasing backoff, and make inference, evaluation, monitoring and the commit run whether or not training succeeded, so a feature-store outage costs a retrain but not the day's forecast.
The application resolved its data paths to the wrong directory in the container	An ordinary install placed the package in the interpreter's library directory, so the derived project root pointed away from the committed artefacts.	Installed the package in editable mode inside the image.

6.25 Summary of Implementation

This chapter described the construction of the system: the technology stack and its justification; the single configuration module; the retrying ingestion client; the EPA index implementation and its deliberate restriction to particulate species; the feature builder and the two mechanisms - shifted windows and per-city grouping - that prevent leakage; the multi-horizon supervised builder; the hourly, backfill, training and inference pipelines; the evaluation suite; the champion-challenger gate and the durable registry, including the refused tie that demonstrates the gate working; the model layer and the sequence-model study; explanation, simulation, monitoring and alerting; the API and its layered fallbacks; the advisor and the tool server; the three scheduled workflows; the container and static deployments; and the twelve substantive failures encountered along the way together with what each of them required. The next chapter presents the user interface through which all of this reaches a member of the public.

Chapter 7

User Interface

7.1 Introduction

The interface is where the system stops being a pipeline and becomes a public service. This chapter describes the goals that governed its design, the visual system it uses, and each of its four views. The figures in this chapter are screenshots of the deployed application taken from the live public deployment, so the values shown - the forecast, the leaderboard, the drift table - are the system's genuine output rather than mock-ups.

7.2 Interface Design Goals

Five goals shaped the interface.

Answer the question in the first screen. A person opening the page wants to know whether the air is safe. The current index, its health category and its advice therefore appear immediately, in a card whose colour is itself the answer, above the chart rather than below it.

Never present a bare number. In accordance with NFR-4, every forecast carries a category name, an uncertainty band and, where one could be computed, an explanation. A number alone invites either misplaced confidence or dismissal.

Use the standard health semantics. The colour scale, the category names and the advisory wording are the EPA's [5], not the project's. A user who has seen an air-quality colour scale elsewhere should not have to learn a new one, and the legend is present on the page so that the mapping is never implicit.

Publish the system's own record. Three of the four tabs are given over to the model rather than the forecast: its promotion history, its accuracy by horizon and its current drift status. This is unusual for a consumer-facing page and it is deliberate, in accordance with principle P6.

Degrade rather than break. When the backend is unreachable the interface says so specifically; when it is running without a backend at all, the two capabilities that require a live model are removed from the navigation rather than offered and failing.

7.3 The Visual System

The application uses a custom dark design system, developed for this project under the name *Aurora*. It is built on a near-black canvas with a slowly drifting, heavily blurred aurora of cyan, teal, violet and sky gradients behind the content, and translucent glass panels above it. The signature accent is a cyan-to-teal gradient, chosen to sit well beside the black-and-white 10Pearls mark, which appears as a circular badge in the header and as a wordmark in the footer.

Three decisions in the visual system are functional rather than decorative. The dark canvas maximises the contrast of the six EPA category colours, which are saturated and are the most important information on the page; on a white canvas the yellow and orange bands in particular lose legibility. The panels are separated by elevation and translucency rather than by heavy borders, which keeps the eye on the data. And the accent colour is deliberately not one of the six category colours, so that interface chrome can never be mistaken for a health signal.

An animated introduction holds the application's mark on screen for a short minimum duration while the forecast document loads, then fades out once both the minimum time has elapsed and the data has arrived. This turns the unavoidable initial fetch into a deliberate moment rather than an empty page.

7.4 The Forecast View

Figure 7.1 shows the default view.

The view is arranged in two columns. The main column carries, from the top: the hazard banner, shown only when the severity is not none; the current-conditions card; the forecast chart; and the explanation card. The side column carries the map, the category legend and the advisor panel.

The **hazard banner** is the highest-priority element on the page and is present in the figure, reporting a peak of 200 in the *unhealthy* band, naming the hour at which the curve first crosses the threshold, and giving the standard advice for that category. It appears above everything else because a user who reads nothing else on the page should still receive the warning.

The **current card** states the observed index as a large numeral against the colour of its category, names the category, repeats the health advice in full, and adds the province and the peak expected over the next three days. Colour is used as a redundant encoding here rather than as the sole one: the category is always named in text, so the card remains readable to a colour-blind user.

The **forecast chart** draws the 72-hour curve with the 80% prediction interval as a shaded band behind it, and the category thresholds as faint horizontal rules, so the reader

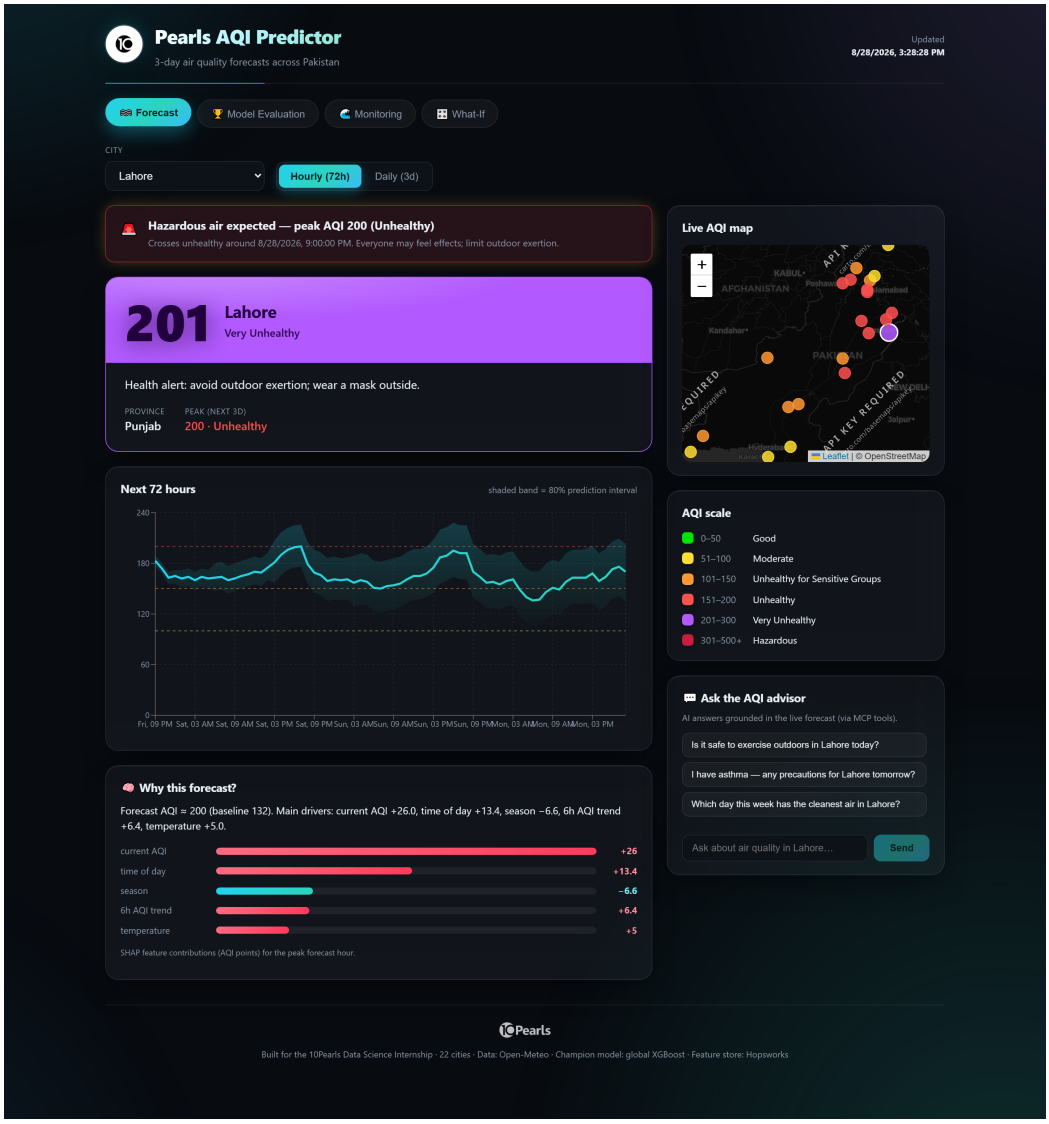

Figure 7.1: The Forecast view of the deployed application.

can see at a glance which band the curve is in and when it crosses. The band is labelled in the panel header rather than left to be inferred. A toggle switches between the hourly view and a three-day aggregation for a user who wants the summary rather than the detail.

The **map** places every city on a Pakistan basemap as a marker coloured by its current category, with the selected city highlighted. It serves two purposes: it allows a city to be selected by pointing rather than by finding it in a list, and it makes the geographic pattern visible - in the figure, the concentration of red markers across the Punjab plain against the yellow of the coast and the north is the exploratory finding of Section 5.6 rendered as a picture.

The **explanation card** presents the sentence generated from the Shapley attributions for the peak hour, followed by a signed horizontal bar for each of the five leading drivers, and a note stating that the values are feature contributions in index points for the peak

forecast hour. In the figure the current index contributes +26, the time of day +13.4, the season −6.6, the six-hour trend +6.4 and the temperature +5.0 against a base value of 132 - which tells a reader not merely that the air will be bad but that it will be bad mainly because it is already bad and the diurnal cycle is working against it.

The **advisor panel** offers three suggested questions and a free-text field. Suggestions are shown because a blank conversational field gives a user no indication of what the system can answer.

7.5 The Model Evaluation View

Figure 7.2 shows the evaluation view.

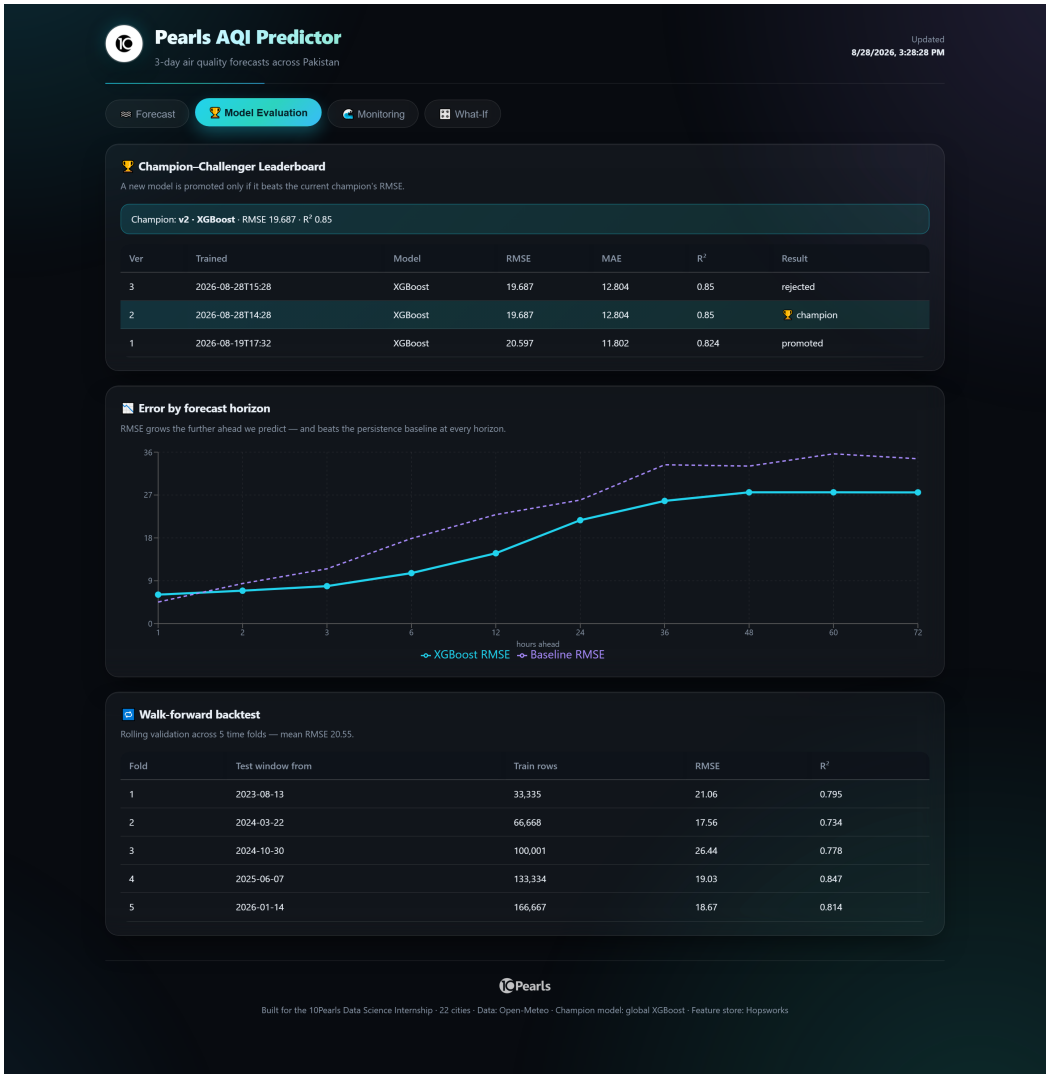

Figure 7.2: The Model Evaluation view.

Three panels present the model's own record. The **leaderboard** names the current champion with its version, family and metrics, and lists every training run in reverse chronological order with its outcome. The refused run discussed in Section 6.13 is

visible at the top of the table, marked *rejected*, immediately above the identical run marked *champion*. That a public interface shows a model that was refused is the point: it is evidence that the gate is real.

The **error-by-horizon** chart plots the champion's RMSE against the persistence baseline's at each of the ten modelled lead times. Two things are legible at once: that error grows steeply with lead time, and that the champion is below the baseline across almost the whole window. Publishing the first of these is what makes the second credible.

The **walk-forward table** lists the five rolling folds with their training size, test window start, RMSE and R^2, and states the mean. It is included because a single validation number invites the suspicion that the split was fortunate.

7.6 The Monitoring View

Figure 7.3 shows the monitoring view.

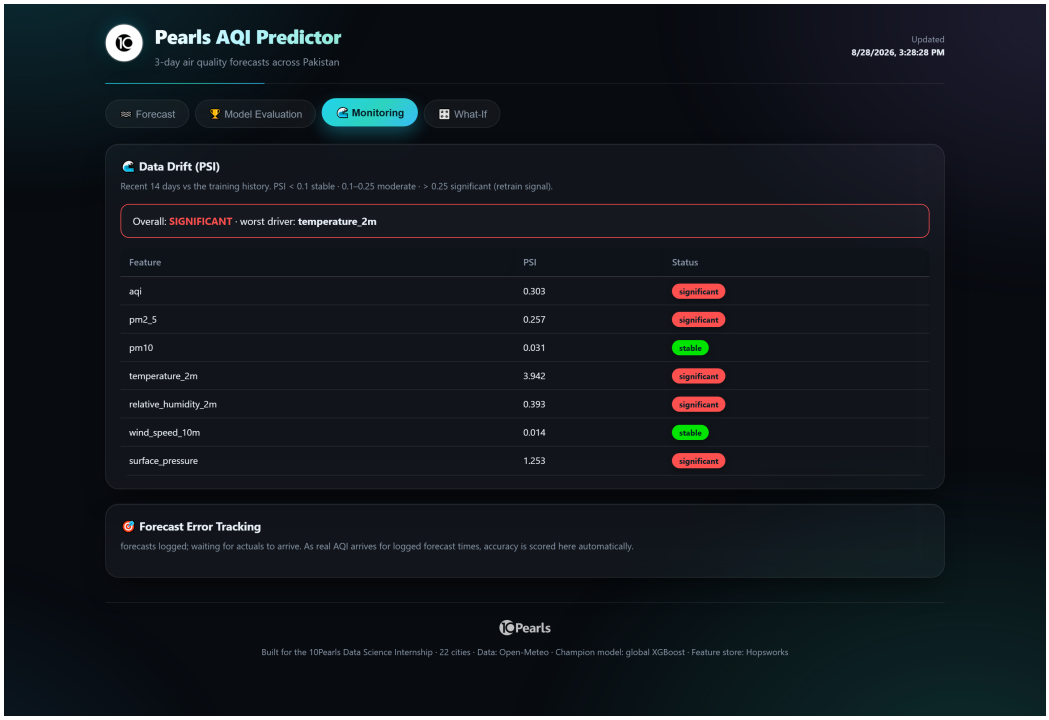

Figure 7.3: The Monitoring view.

The drift panel states the overall status and names the worst-drifting feature, then tabulates the Population Stability Index for each monitored feature with a coloured status badge. The three thresholds are stated in the panel's subtitle, so that a reader can interpret a number without knowing the measure beforehand. In the figure the overall status is significant, driven by temperature; particulate matter at ten micrometres and wind speed remain stable.

Presenting an alarming word like *significant* on a public page is a deliberate choice, and the second panel is what makes it honest. Forecast error tracking reports the realised

accuracy of past forecasts once the hours they described have arrived; in the figure it correctly reports that forecasts have been logged and are waiting for observations rather than displaying a fabricated zero. Read together, the two panels say what is true: the inputs have moved seasonally, and whether that has cost any accuracy is a question the system will answer with data rather than assumption.

7.7 The What-If View

Figure 7.4 shows the what-if view after a scenario has been run.

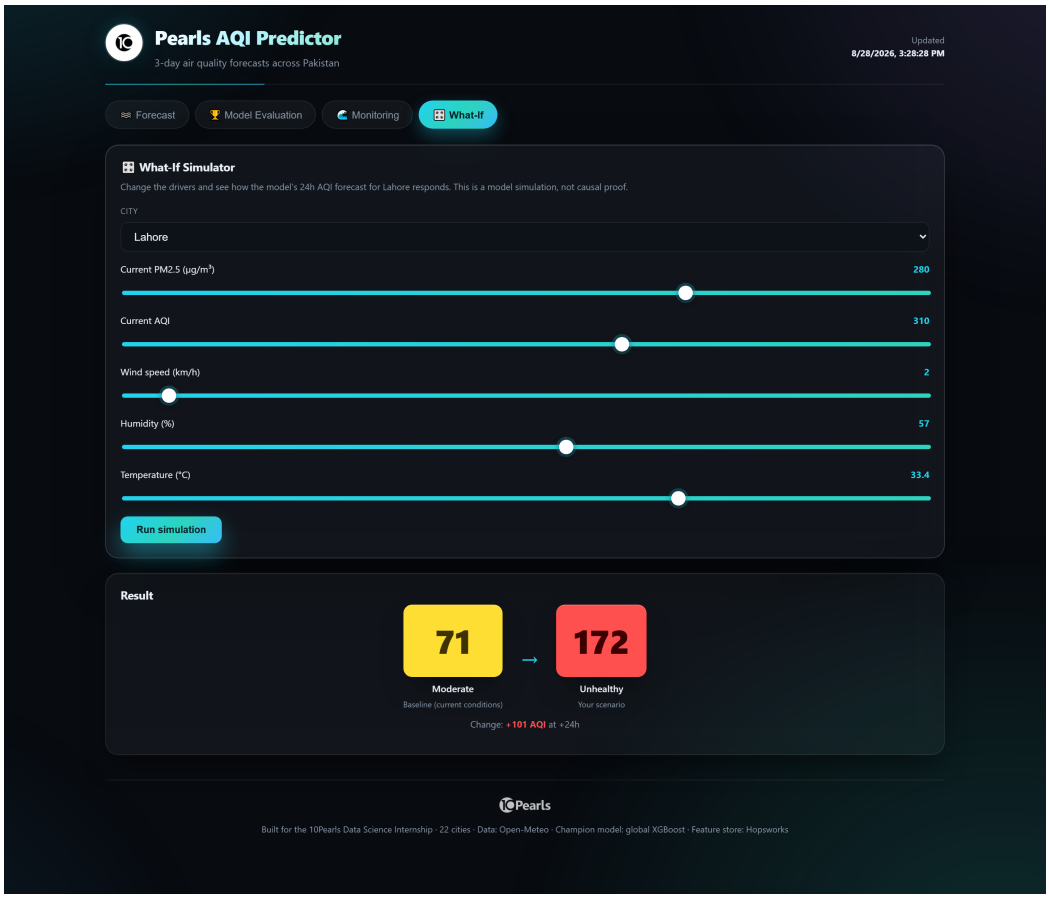

Figure 7.4: The What-If view, showing a scenario result.

Five sliders expose the drivers described in Section 6.17, each initialised to the current real value for the selected city so that the user begins from reality rather than from an arbitrary midpoint. Running the simulation calls the live model twice and presents the baseline and the scenario side by side, each as a large numeral on its category colour with its category named beneath, and states the change between them.

The figure shows a heavy-pollution, still-air scenario for Lahore: particulate matter raised to 280 micrograms per cubic metre, the current index raised to 310 and the wind reduced to 2 kilometres per hour. The model moves the 24-hour forecast from 38, in the *good* band, to 159, in the *unhealthy* band - a change of +121 index points. The panel's

subtitle states plainly that this is a model simulation and not causal proof, satisfying NFR-6.

The tab is present only when a live backend is configured, because the simulation requires the model to be invoked; in the static serving mode it is removed from the navigation entirely.

7.8 Interaction and Responsiveness

Navigation is a single row of tabs, and the selected city is shared across the Forecast and What-If views, so a user who has chosen Multan does not have to choose it again. The header states when the displayed forecast was generated, which is essential for a product whose data is refreshed on a schedule: a stale page should be identifiable as stale.

The layout is fluid. On a wide viewport the two columns sit side by side; on a narrow one they stack, with the main column first, so that the current conditions and the chart precede the map and the legend on a phone. Charts resize with their container rather than being fixed, and long tables scroll horizontally within their panel rather than forcing the page to scroll.

Three states are handled explicitly rather than being left to chance. While the forecast document is loading, the application shows its mark, a spinner and a caption. If the fetch fails, it shows a notice naming the failure and, for a developer running locally, the command that starts the backend. If a selected city is absent from the payload, the application falls back to the first city present rather than rendering an empty view.

7.9 Summary of User Interface

This chapter described the interface through which the system reaches the public: its five design goals, the dark Aurora visual system and the functional reasons behind it, and its four views. The Forecast view answers the user's question immediately and supports it with an uncertainty band, a map, a legend, an explanation and an advisor. The Model Evaluation and Monitoring views publish the system's own record - including a refused model and an unflattering drift status - because a public-health signal that cannot be checked will not be trusted. The What-If view lets a user interrogate the model directly. The next chapter reports the testing and the empirical evaluation of everything described so far.

Chapter 8

Testing and Evaluation

8.1 Introduction

This chapter reports how the system was verified and how well it performs. It covers three kinds of evidence. *Software testing* establishes that the code does what it was written to do, with particular attention to the properties that would otherwise fail silently. *Model evaluation* establishes how accurate the forecasts are, reported per horizon and under rolling validation rather than as a single number. *System validation* establishes that the deployed service actually works, by exercising every published endpoint against the live deployment. The chapter closes with the operational evidence from the running system, an assessment of the threats to the validity of these results, and a discussion of what the results mean.

8.2 Testing Objectives

Five objectives were set.

1. To verify that the index computation is correct at its breakpoints, its category boundaries and its edge cases, since every number the system produces is expressed in that unit.
2. To verify that no target leakage exists in the feature representation, because leakage produces a model that scores well and fails in production, and the failure is invisible in the metrics that would normally be trusted.
3. To establish that the model has genuine forecasting skill by comparing it against a persistence baseline at every horizon, not merely on average.
4. To estimate forward performance honestly, using rolling validation rather than a single fortunate split.
5. To confirm that the deployed system serves every documented capability, and that capabilities which are unavailable fail in the specified, informative way rather than obscurely.

8.3 Testing Strategy

Testing was applied at four levels, summarised in Table 8.1. The distribution of effort is deliberate: the heaviest unit testing is concentrated on the two components whose

failures would be silent, and the heaviest system-level effort is concentrated on the deployed endpoints, because a capability that works locally and not in production has not been delivered.

Table 8.1: Testing strategy by level.

Level	Method	What it establishes
Unit	pytest [45] over synthetic, deterministic inputs	That the index computation and the feature and supervised builders are correct, including the leakage properties.
Integration	Scheduled pipeline runs against live data and the real feature store	That ingestion, storage, training, promotion, inference and monitoring compose correctly under real conditions.
System	Direct exercise of every published endpoint on the live deployment	That the delivered service works from outside, and that unavailable capabilities fail informatively.
Model	Held-out chronological validation, per-horizon metrics, walk-forward backtesting, baseline comparison	That the forecasts have skill, and how that skill decays with lead time.

8.4 Unit Testing

The unit suite comprises twelve tests. All twelve pass; the run reported `12 passed` in 22.01 seconds. The tests are built on synthetic series with known structure rather than on live data, so they are deterministic and do not depend on a network.

8.4.1 Index Computation Tests

Seven tests cover the index module.

The first checks the piecewise-linear interpolation at a breakpoint endpoint: a particulate concentration of 9.0 micrograms per cubic metre sits exactly at the top of the *good* band and must therefore produce an index of exactly 50. Testing at an endpoint rather than in the middle of a band is deliberate, since an off-by-one in the breakpoint tables would be invisible mid-band.

The second checks monotonicity across bands, requiring that a rising sequence of concentrations produce a non-decreasing sequence of index values, which would catch a transposed or mis-ordered breakpoint pair.

The third is the most important test in the module, and encodes the design decision of Section 6.5. It supplies a very large carbon monoxide value alongside clean particulate readings and requires the computed index to be unaffected - that is, it asserts that gaseous pollutants are excluded from the computation. Without this test, a later change that added gases back without the necessary unit conversion would silently corrupt every affected row, because the index is a maximum over sub-indices and one inflated sub-index dominates.

The remaining four confirm that the category boundaries are assigned correctly at 50, 51 and 301; that the six categories are contiguous, each band beginning exactly one above the previous band's end, so that no value can fall between categories; that the hazard predicate fires above the threshold and not below it; and that a missing concentration yields a missing sub-index rather than an exception, so that a gap in the provider's data cannot fail a batch.

8.4.2 Feature and Supervised-Set Tests

Five tests cover the feature layer, and two of them are the leakage guards.

The first confirms that the engineered table contains the expected feature families - a cyclical hour encoding, the target, a change rate, a 24-hour lag, a 24-hour rolling mean, both wind components and the primary key.

The second confirms that the derived primary key is both strictly increasing and unique within a city, which is the precondition for the upsert semantics on which every re-run of every pipeline depends.

The third is the cross-city leakage guard. Two synthetic cities are concatenated and passed through the multi-city builder, and the test requires that *each* city independently show exactly 24 missing values in its 24-hour lag column. This can only hold if the two cities were engineered as separate groups; had the lag been computed over the concatenated frame, the second city would show none, because it would have silently borrowed the first city's history.

The fourth confirms that the supervised builder emits every declared model feature and the target, and that the horizon column takes exactly the requested values - verifying that the horizon really is a feature rather than an implicit property of the dataset.

The fifth is the temporal leakage guard, asserting that the anchor value attached to a row at horizon h is the value observed h steps earlier, and that no anchor is missing after the incomplete rows have been dropped.

8.4.3 Coverage Assessment

The suite covers the two components whose failure modes are silent. It does not cover the promotion gate, the interval construction, the drift computation or the API handlers, which were verified by integration and system testing rather than by unit tests. Section 8.11 treats this as a limitation, and Chapter 9 lists it as future work.

8.5 Integration and System Testing

Integration testing was performed by running the pipelines end to end on the scheduled runners against the real feature store and the real provider. Four integration properties were confirmed in this way, and each of them had failed at least once before it held.

Idempotent ingestion. The hourly pipeline was run repeatedly within the same hour, with the deliberate seven-day overlap, and the row count in the feature store grew only by genuinely new hours, confirming that the composite-key upsert behaves as designed.

Resumable backfill. The backfill was interrupted and restarted, and resumed only those cities whose stored history did not yet reach the requested start date, confirming the history-aware resume of Figure 6.4.

Registry round trip. A model trained on one ephemeral runner was loaded by a different job on a different runner, confirming that the champion genuinely survives the machine that produced it - the property on which the entire serverless architecture rests.

Promotion gating under repetition. Training was run twice in close succession on effectively unchanged data. The first run promoted; the second produced an identical validation RMSE and was refused. This is reported in full in Section 8.9.1.

System testing exercised the deployed public API directly. Table 8.2 reports the result of calling every endpoint against the live deployment.

Table 8.2: System validation against the live deployment.

Endpoint	Status	Latency	Observation
GET /api/health	**200**	**1.32 s**	Reports a forecast document present.
GET /api/categories	**200**	**0.82 s**	Six categories with bounds and advice.
GET /api/cities	**200**	**0.72 s**	All 22 cities with province and coordinates.
GET /api/predictions	**200**	**1.51 s**	190 kB payload covering every city.
GET /api/predictions/Lahore	**200**	**0.99 s**	72 hourly points and three daily points.
GET /api/leaderboard	**200**	**0.88 s**	Champion plus three run records.

Endpoint	Status	Latency	Observation
GET /api/evaluation	**200**	**0.74 s**	Ten per-horizon rows and five backtest folds.
GET /api/monitoring	**200**	**0.74 s**	Drift table and forecast-error status.
GET /api/explain/Lahore	**200**	**0.75 s**	Sentence plus five ranked contributions.
GET /api/whatif/defaults	**200**	**0.95 s**	Five sliders initialised to live values.
GET /api/predict (arbitrary point)	**200**	**0.98 s**	On-demand forecast for a city outside the supported list, confirming FR-22.
POST /api/whatif	**200**	**1.15 s**	Baseline 38, scenario 39 under a raised wind speed; both categories returned.
POST /api/chat	**200**	**9.1 s**	Grounded answer from the free Gemini tier; the reply names the cleanest day and cites the forecast values behind it.
GET /api/advisor	**200**	**0.8 s**	Reports the selected provider and model.
GET /api/advisor/models	**200**	**1.2 s**	Lists the model ids the configured key can use.

All fifteen endpoints serve their documented payload. The advisor endpoint is the slowest by an order of magnitude, at 9.1 seconds, because a single call runs a tool round trip: the model asks for a forecast, the backend executes the real function, and the model then composes its answer from the value returned. With no credential configured it returns the specific 503 message described in use case UC-7 rather than failing inside a client library, which is the behaviour test TC-25 pins.

8.6 Functional Test Cases

Table 8.3 records the functional test cases, the requirement each exercises and its result.

Table 8.3: Functional test cases and results.

ID	Test	Req.	Result
TC-1	Ingest all cities and confirm 21 raw columns per city with retries configured.	**FR-1**	Pass
TC-2	Engineer features and confirm the 74-column table with cyclical, lag, rolling and wind-vector families.	**FR-2**	Pass

ID	Test	Req.	Result
TC-3	Re-run ingestion within the overlap window and confirm no duplicate rows.	**FR-3**	Pass
TC-4	Interrupt and restart the backfill; confirm only incomplete cities are reprocessed.	**FR-3**	Pass
TC-5	Build the supervised set and confirm the horizon column takes exactly the ten declared values.	**FR-4**	Pass
TC-6	Confirm the validation window lies entirely after the training window.	**FR-5**	Pass
TC-7	Confirm all four candidate families are scored in a run.	**FR-6**	Pass
TC-8	Retrain on unchanged data and confirm the tie is refused.	**FR-7**	Pass
TC-9	Load the champion on a runner that did not train it.	**FR-8**	Pass
TC-10	Confirm 72 hourly points and three daily points for every city.	**FR-9**	Pass
TC-11	Confirm interval bounds present, ordered and horizon-dependent.	**FR-10**	Pass
TC-12	Confirm every forecast point carries a health category.	**FR-11**	Pass
TC-13	Confirm a hazard alert with peak, peak hour and first crossing for a city above the threshold.	**FR-12**	Pass
TC-14	Confirm an explanation sentence and five ranked contributions per city.	**FR-13**	Pass
TC-15	Confirm the global importance ranking is produced.	**FR-14**	Pass
TC-16	Confirm slider defaults reflect live conditions.	**FR-15**	Pass
TC-17	Run a scenario and confirm baseline, scenario, categories and delta.	**FR-16**	Pass
TC-18	Confirm PSI is computed for all seven monitored features with a status each.	**FR-17**	Pass

ID	Test	Req.	Result
TC-19	Confirm forecasts are logged and the error report distinguishes "waiting for actuals" from a real score.	**FR-18**	Pass
TC-20	Load all four dashboard views against the live backend.	**FR-19**	Pass
TC-21	Confirm the four advisor tools are declared and dispatchable.	**FR-20**	Pass
TC-22	Confirm the protocol server exposes the same four tools.	**FR-21**	Pass
TC-23	Request an on-demand forecast for a city outside the supported list.	**FR-22**	Pass
TC-24	Confirm the hourly and daily workflows run to completion on schedule.	**FR-23**	Pass
TC-25	Request the advisor with no credential configured.	**NFR-11**	Pass - returns the documented 503 message naming the free providers.
TC-26	Ask the advisor a real question end to end.	**FR-20**	Pass - answered from a get_forecast tool call, not from model recall.
TC-27	Interrupt the feature-store read; confirm the nightly run still refreshes the forecast.	**NFR-8**	Pass - inference is independent of training.

8.7 Model Evaluation

8.7.1 Candidate Comparison

Table 8.4 reports the champion training run. The supervised set was 200,000 rows, split chronologically into 160,002 training and 39,998 validation rows.

Table 8.4: Candidate comparison on the held-out chronological validation window.

Model	RMSE	MAE	R^2	Note
XGBoost	19.69	12.80	0.850	Promoted champion
Random forest	20.38	13.56	0.839	
Ridge regression	22.85	16.01	0.797	Linear reference
Persistence (baseline)	25.27	15.71	0.752	Not eligible for promotion

Three observations follow. The champion reduces RMSE by 22.1% relative to persistence, which establishes that the system has real forecasting skill rather than merely reproducing the current value. The gap between the linear model and the two

ensembles - 22.85 against 20.38 and 19.69 - confirms the reading of the exploratory analysis in Section 5.6, that the relationship is genuinely non-linear and interactive rather than an additive combination of weather variables. And the ordering of the two ensembles is consistent with the tabular-learning literature [22], with boosting ahead of bagging on a problem of this shape.

One detail is worth drawing out because it demonstrates why RMSE rather than MAE governs promotion. The persistence baseline has a *lower* MAE than the Ridge model (15.71 against 16.01) while having a substantially higher RMSE (25.27 against 22.85). Persistence is typically close but occasionally very wrong, which is exactly the behaviour RMSE penalises and MAE does not; for a health warning system, the large error during a rapid deterioration is the one that matters, so the squared metric is the appropriate gate.

8.7.2 Accuracy by Forecast Horizon

Table 8.5 and Figure 8.1 report accuracy at each modelled lead time, against the persistence baseline evaluated under the same mask.

Table 8.5: Accuracy by forecast horizon, with the persistence baseline at the same horizon.

Horizon (h)	RMSE	MAE	R^2	Baseline RMSE	Improvement
1	6.10	4.01	0.985	4.54	−34.4%
2	6.92	4.44	0.981	8.43	17.9%
3	7.88	5.13	0.975	11.52	31.6%
6	10.61	7.06	0.957	17.88	40.7%
12	14.78	10.46	0.917	22.89	35.4%
24	21.72	15.85	0.819	25.94	16.3%
36	25.76	19.15	0.746	33.35	22.8%
48	27.56	20.64	0.708	33.07	16.7%
60	27.57	20.53	0.721	35.64	22.6%
72	27.55	21.16	0.695	34.61	20.4%

The table is the single most informative result in this thesis, and it says three things.

Skill decays with lead time, steeply and then gently. R^2 falls from 0.985 at one hour to 0.695 at 72, and RMSE rises from 6.10 to 27.55. Almost all of the degradation occurs in the first 36 hours; between 36 and 72 hours the error is essentially flat at 25.8 to 27.6. This plateau is interpretable: beyond about a day and a half the current pollution state has ceased to carry information, and the forecast is being driven almost entirely by the forecast weather and the calendar, whose informativeness does not decay much further over the remaining window.

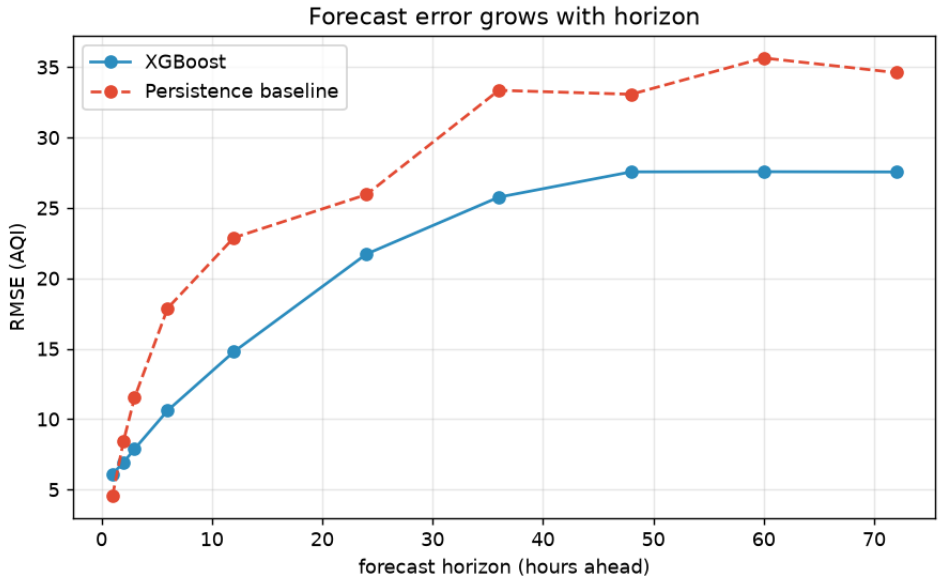

Figure 8.1: RMSE against forecast horizon, compared with the persistence baseline.

The model beats persistence at every horizon from two hours onward, by between 16% and 41%, with the largest margins in the 6-to-12-hour range where the anchor state and the weather forecast are both informative. This is the result that justifies the system existing.

At one hour, persistence wins. Its RMSE of 4.54 is better than the model's 6.10. This is reported rather than omitted, and it is not a defect: at a lead time of one hour the air quality is overwhelmingly determined by the air quality of the preceding hour, and a global model trained jointly across ten horizons cannot specialise on that regime as tightly as the trivial predictor does. It is the price of the single-artefact design of Section 5.8, it is confined to the horizon at which forecasting is least useful - nobody plans around the next sixty minutes - and the design would readily accommodate a blend towards persistence at very short lead times if that were wanted. Reporting it is exactly the kind of disclosure that a single averaged accuracy figure would have concealed.

8.7.3 Walk-Forward Backtest

Table 8.6 and Figure 8.2 report the five-fold rolling backtest, in which each fold trains on an expanding window of the past and tests on the window that follows.

Table 8.6: Five-fold walk-forward backtest.

Fold	Train rows	Test rows	Test from	RMSE	R^2
1	33,335	33,333	2023-08-13	21.06	0.795
2	66,668	33,333	2024-03-22	17.56	0.734
3	100,001	33,333	2024-10-30	26.44	0.778

Fold	Train rows	Test rows	Test from	RMSE	R^2
4	133,334	33,333	2025-06-07	19.03	0.847
5	166,667	33,333	2026-01-14	18.67	0.814

Figure 8.2: RMSE by walk-forward fold.

The mean backtest RMSE is 20.55 with a standard deviation of 3.53. Two features of this result matter more than the mean.

First, the backtest mean is slightly *worse* than the single-split validation figure of 19.69, and that is the expected and desirable direction. Early folds train on as little as a fifth of the data, so the backtest deliberately measures performance under conditions harsher than production; a backtest that flattered the single split would be evidence of a leak rather than of quality.

Second, the fold-to-fold variation is itself informative. Fold 3, which tests from 30 October 2024, is much the worst at 26.44, while fold 2, testing from late March, is the best at 17.56. The two are the winter smog season and the clean spring respectively. The model is therefore measurably harder to fit in the season that matters most - which is precisely why the nightly retraining and the drift monitoring exist, and it is a caution against quoting the annual mean error as though it applied uniformly across the year.

8.7.4 The Sequence-Model Comparison

The LSTM was evaluated on a deliberately fair footing: the same 24-hour-ahead task, the same chronological split, the same metrics and the same baseline. Table 8.7 reports the comparison.

Table 8.7: Sequence model against the tree ensemble at a 24-hour horizon.

Model (predict AQI at +24 h)	RMSE	MAE	R^2	Note
LSTM (two recurrent layers, dense head)	22.12	14.32	0.799	Best at this single horizon
XGBoost (same 24-hour task)	22.63	14.48	0.789	
Persistence (baseline)	28.79	17.21	0.660	

The sequence model wins, narrowly: 22.12 against 22.63, a 2.3% reduction in RMSE. Both learned models beat persistence by roughly 22%, which independently confirms that the task has learnable structure.

It was nonetheless not promoted, for three reasons that are worth stating explicitly because the decision runs against the narrow measurement.

First, *coverage*. The LSTM as constructed predicts one horizon; the production model serves all ten, at an overall RMSE of 19.69. Replacing the champion with the sequence model would mean either training ten of them or serving only one lead time, and the resulting system would be worse on the metric that actually matters to a user, which is the quality of the whole three-day curve.

Second, *cost*. The sequence model takes roughly 25 minutes to train against roughly one minute for the ensemble on the same hardware. Within the scheduler's time limit, ten of them do not fit. The nightly retrain - which is the mechanism by which the entire system stays current - would have to be abandoned to adopt the model that is 2.3% better at one horizon.

Third, *explainability*. Exact Shapley attribution is available in polynomial time for the tree ensemble [28] and is not for the recurrent network, so promoting it would mean withdrawing the explanation that Chapter 7 shows on every forecast.

The finding is reported as a positive one. The sequence model is a legitimate and competitive alternative, and the comparison is exactly what the tabular-learning literature [21, 22] would predict: no decisive advantage for the deep architecture on a tabular, weather-driven problem. The production choice was made on the total system objective rather than on a single number, and the measurement is what allows that to be said rather than assumed.

8.7.5 Feature Importance

Figure 8.3 shows the global feature importance obtained by averaging absolute Shapley values over a sample of the supervised set.

The ranking is physically coherent, which is itself a form of validation. The anchor state - the current index and its recent means - dominates, as it must for a persistent

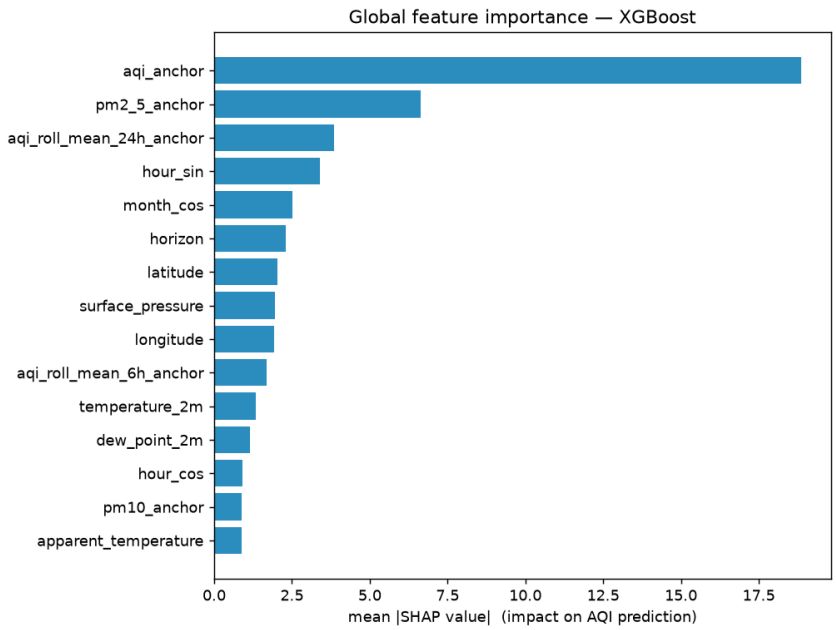

Figure 8.3: Global feature importance by mean absolute Shapley value.

quantity. The horizon feature ranks highly, which is direct evidence that the model has learned to modulate its behaviour by lead time rather than treating all horizons alike; this is the mechanism by which one artefact can serve ten horizons. Among the meteorological drivers, the dispersion-related variables carry the most weight, consistent with the physical account of the smog season given in Chapter 2. Nothing implausible appears near the top, which is a check that would have caught a leaked or mis-specified feature.

8.8 Uncertainty Calibration

Prediction intervals are the tenth and ninetieth residual percentiles computed per horizon on the held-out validation window, giving a nominal 80% interval. By construction the intervals attain approximately 80% coverage on that window, and they widen with horizon in proportion to the model's actual errors, which is the behaviour the design intended and which is visible as the widening band in Figure 7.1.

Two limitations must be stated. The construction inherits the exchangeability assumption of split conformal prediction [25], which a seasonal, non-stationary series does not strictly satisfy; coverage during a regime the calibration window did not contain - an unusually severe smog episode, for instance - should be expected to be worse than nominal. And coverage has not yet been measured prospectively on live forecasts, because doing so requires a sufficient volume of matured forecast-outcome pairs. The machinery to perform that measurement exists in the forecast log described in Section 6.18, and it is identified as future work.

8.9 Operational Evidence from the Running System

8.9.1 The Promotion Gate in Production

The leaderboard is the record of the gate's behaviour, and is reproduced in Table 8.8.

Table 8.8: The production model leaderboard, as recorded on 31 August 2026. The leaderboard grows by one row per nightly run, so these rows are a snapshot rather than a final state.

Ver	Trained	Model	RMSE	R^2	Outcome
1	2026-08-19 17:32	XGBoost	20.597	0.824	Promoted - first champion
2	2026-08-28 14:28	XGBoost	19.687	0.850	Promoted - current champion
3	2026-08-28 15:28	XGBoost	19.687	0.850	Rejected - no improvement
4	2026-08-30 08:27	XGBoost	19.687	0.850	Rejected - no improvement
5	2026-08-31 08:47	XGBoost	19.687	0.850	Rejected - no improvement

The five rows demonstrate all three behaviours the gate is required to exhibit. Version 1 was promoted unconditionally as the first champion. Version 2, trained after the full historical backfill had completed, improved RMSE from 20.597 to 19.687 - a 4.4% reduction obtained purely from more history - and displaced the incumbent, which is the retraining loop delivering measurable value. Version 3, trained an hour later on effectively unchanged data, produced an identical 19.687 and was refused, because the promotion rule requires strict improvement. Versions 4 and 5, on the two following nights, were refused for the same reason. Those three consecutive refusals are the most important rows in the table: they are the system declining, three nights running, to replace its production model for no gain, which is precisely what the gate exists to do and what an ungated nightly retrain would not have done.

8.9.2 Drift Monitoring in Production

Table 8.9 reports the drift status computed over the most recent fourteen days against the historical reference.

Table 8.9: Population Stability Index by feature, recent fourteen days against the historical reference.

Feature	PSI	Status	Interpretation
temperature_2m	3.942	significant	Seasonal transition; the recent window occupies a temperature range the annual reference visits only briefly.
surface_pressure	1.253	significant	Moves with the same seasonal cycle.

Feature	PSI	Status	Interpretation
relative_humidity_2m	0.393	significant	Monsoon-season humidity against an all-year reference.
aqi	0.303	significant	The target distribution has shifted with the season.
pm2_5	0.257	significant	Just above the threshold, consistent with the index.
pm10	0.031	stable	Coarse particulate matter is less seasonally modulated.
wind_speed_10m	0.014	stable	Wind distribution is broadly stationary.

The overall status is *significant*, driven by temperature. This is the expected result and not a fault, and reading it correctly is the point of the design in Section 6.18. A fourteen-day window compared against a multi-year reference will always look shifted on any strongly seasonal variable, because two weeks of one season cannot resemble an average over all seasons. That five of the seven monitored features register as significant while the two least seasonal remain stable is itself corroboration that the measure is detecting seasonality rather than a data fault.

The correct operational response is the one the system already takes: retrain nightly, so that the training distribution tracks the current one. What would constitute a genuine alarm is drift accompanied by rising realised error, which is why the second monitoring signal exists. At the time of writing that signal correctly reports that forecasts have been logged and are awaiting matured observations, rather than displaying a fabricated score.

8.10 Non-Functional Testing

Table 8.10 records how the non-functional requirements were verified.

Table 8.10: Verification of the non-functional requirements.

ID	Requirement	Evidence
NFR-1	Free-tier operation	Ingestion is keyless and free; scheduled compute, feature storage and static hosting are within free allowances; only the container host and the optional advisor carry any cost.
NFR-2	Statelessness	A model trained on one runner was loaded by a different job on another, and the deployed backend serves a champion it never trained.
NFR-3	Performance	All eleven read endpoints responded in 0.72 to 1.51 seconds, serving pre-computed documents; no page view invokes a model.

ID	Requirement	Evidence
NFR-4	Explainability	Every forecast point carries interval bounds; every city in the live payload carries an explanation object.
NFR-5	Groundedness	The advisor answers only through the four declared tools, which read the same forecast document the dashboard reads.
NFR-6	Honest framing	The what-if panel states that it is a model simulation and not causal proof.
NFR-7	Reproducibility	Seeds fixed for every estimator and for subsampling; the split is defined by time rather than by chance.
NFR-8	Resilience	Explanation, monitoring and forecast logging are each wrapped; a per-city failure in inference costs one city, not the run.
NFR-9	Auditability	The leaderboard, evaluation report and monitoring report are committed each night and served publicly.
NFR-10	Portability	The full pipeline runs locally against the Parquet backend on a platform where the managed client cannot be installed.
NFR-11	Graceful degradation	The advisor returns a specific 503 message when unconfigured; the what-if tab is hidden in static mode; the monitoring endpoint has three levels of fallback.
NFR-12	Maintainability	Layered modules with one-directional dependencies; cities, paths and horizons defined in exactly one module.
NFR-13	Usability	EPA colour scale and category names used throughout; layout stacks on a narrow viewport; the generation time is displayed in the header.
NFR-14	Politeness to the source	Retry with exponential backoff; three-month chunking; coverage-based resume avoids refetching.
NFR-15	Security	No credential in the repository; secrets injected from the environment locally and from the workflow secret store in automation.

8.11 Threats to Validity

Seven threats to the validity of these results are acknowledged.

Modelled rather than measured ground truth. The target is a modelled index derived from an atmospheric composition product, not a reading from a reference-grade station in each city. The system therefore demonstrates skill at forecasting that product. Where the product diverges from a local station, the forecast inherits the divergence. Validating against ground stations is the single most valuable extension available to this work.

Optimistic weather at training time. The model is trained on the weather that actually occurred at the target hour and deployed on the forecast weather for that hour. The reported validation errors are therefore a mild under-estimate of operational error,

and the gap widens with horizon, since weather forecasts themselves degrade. This is standard practice, but it is a real bias in the direction of flattery.

Coverage has not been verified prospectively. The 80% intervals attain their nominal coverage on the calibration window by construction; whether they do so on live forecasts has not yet been measured, and the exchangeability assumption gives reason to expect degradation during unusual regimes.

Limited unit-test coverage. The suite covers the index computation and the feature layer thoroughly and leaves the promotion gate, the interval construction, the drift computation and the API handlers to integration and system testing. Those components are exercised in production, but a regression in one of them would not be caught before deployment.

A short operational record. The leaderboard held five runs when this was written and has continued to grow since; the realised-error monitor has not yet accumulated matured forecast-outcome pairs. The gate's behaviour is demonstrated but its long-run behaviour - how often a genuine improvement occurs, whether a degraded model is ever correctly refused - is not yet evidenced.

A single evaluation regime. The champion metrics come from one chronological split whose validation window falls in a particular part of the year. The walk-forward backtest mitigates this, and its fold-to-fold spread of 17.56 to 26.44 quantifies how much the answer depends on which season is being tested.

No user study. The interface's claims - that the explanation is understandable, that the colour scale communicates risk, that the alert prompts action - are argued from design principles and are not empirically established.

8.12 Discussion

Taken together, the results support four conclusions.

The system forecasts with genuine skill. It reduces RMSE by 22% against persistence, and beats it at every horizon from two hours to three days. The skill is real, it is measured against the correct reference, and it is reported at the granularity at which it is used.

The skill is honestly bounded. R^2 falls from 0.985 to 0.695 across the forecast window, the backtest is slightly worse than the single split, the winter fold is markedly worse than the spring fold, and persistence beats the model at a one-hour lead. None of this is concealed, and the fact that a public dashboard displays the decaying error curve is a deliberate design position rather than an oversight.

The operational machinery works and has already earned its place. Retraining after the backfill improved RMSE by 4.4%; the gate then refused an identical model an hour

later. Both events are recorded in a leaderboard that a member of the public can read. This is the difference between a model and a system.

Finally, the model choice was made by measurement rather than fashion. A sequence model was built, evaluated fairly, found to be 2.3% better at one horizon, and declined on grounds of coverage, cost and explainability - with the reasoning recorded so that a reader can disagree with it.

8.13 Summary of Testing and Evaluation

This chapter reported twelve passing unit tests concentrated on the two components whose failures would be silent, four integration properties confirmed on live infrastructure, and fifteen endpoints exercised against the live deployment, all serving their documented payload, with the advisor degrading to a specific, actionable message when no credential is configured. It reported the champion's accuracy - RMSE 19.69, MAE 12.80, R^2 0.850 - against three alternatives, per-horizon accuracy against a per-horizon baseline, a five-fold walk-forward backtest averaging 20.55, a fair comparison with a recurrent sequence model, the global feature importance, the production leaderboard including a refused promotion, and the current drift status with its seasonal interpretation. It verified every non-functional requirement and set out seven threats to the validity of these results. The next chapter concludes.

Chapter 9

Conclusion and Future Work

9.1 Conclusion

This thesis has presented the Pearls AQI Predictor: an end-to-end, effectively serverless machine-learning system that forecasts the Air Quality Index up to 72 hours ahead for 22 cities across every province and territory of Pakistan, retrains and validates itself nightly without human intervention, explains each forecast in plain language, quantifies its own uncertainty, monitors its own inputs and outputs for decay, and serves all of this publicly at a running cost of effectively zero.

The work began from the observation that air-quality information in Pakistan is the wrong shape for the decisions people make with it. It is a nowcast when a forecast is needed; it covers one or two cities when the worst-affected city in the record assembled here is not the one that receives the attention; it is offered without explanation or uncertainty; and where a model exists at all it is a model trained once, which for a strongly seasonal phenomenon means a model that decays silently. The response developed here was not to build a better model but to build a system that keeps a model correct.

The central architectural decision was to express the system as three decoupled Feature, Training and Inference pipelines communicating only through a managed feature store and model registry. Because all durable state lives in those two services, the compute can be entirely disposable, and because the compute is disposable it can be free. The central modelling decision was to treat the forecast horizon as an input feature, so that one global model conditioned on location serves 22 cities and ten lead times rather than requiring 220 artefacts. The central operational decision was to gate promotion: a nightly retrain may produce a new model, but it may not deploy one unless it has been measured to be better.

9.1.1 Fulfilment of the Objectives

Each of the six objectives set in Chapter 1 was met.

Objective 1 - Data and features. An hourly ingestion pipeline fetches seven pollutant and ten weather variables for 22 cities from a free, keyless provider with retry and backoff, and a per-city feature builder expands 21 raw columns into a 74-column

engineered table with cyclical calendar encodings, wind vector decomposition, lags at five offsets and rolling statistics over two windows. The leakage discipline is enforced by construction - every rolling window shifted, every family computed within a single city - and verified by unit tests, one of which would fail if the per-city grouping were ever removed. A resumable backfill populated the store with roughly 697,000 hourly rows spanning January 2023 onward.

Objective 2 - Serverless pipeline architecture. Three pipelines run on free scheduled runners: ingestion hourly, and training, inference, evaluation and monitoring nightly. They share no code path and communicate only through the feature store and the model registry. A model trained on one ephemeral runner is loaded by a different job on a different machine, which is the property on which the whole architecture rests and which was verified directly.

Objective 3 - Accurate multi-horizon forecasting. A single global gradient-boosted model with the horizon as its twenty-seventh input attains a validation RMSE of 19.69, MAE of 12.80 and R^2 of 0.850, beating a persistence baseline by 22% overall and at every horizon from two hours to three days. Every forecast point carries an 80% prediction interval derived from per-horizon empirical residual quantiles, which widens with lead time as the model's real errors do.

Objective 4 - MLOps discipline. A champion-challenger gate promotes only on a strict improvement in validation RMSE and records every decision in a persistent leaderboard; the champion is stored in a durable versioned registry. The gate's production record contains a first promotion, a genuine improvement from 20.597 to 19.687 after the backfill, and a refusal of an identical model an hour later. Accuracy is reported per horizon against a per-horizon baseline and under a five-fold walk-forward backtest averaging 20.55, rather than as a single averaged number.

Objective 5 - A trust layer. Every city's forecast carries a plain-language explanation generated from exact Shapley attributions for its peak hour; a what-if simulator re-predicts under user-supplied driver overrides and is labelled explicitly as a model simulation; and the system monitors seven features for distributional drift by Population Stability Index while separately logging every forecast and scoring it against the air quality that subsequently arrived.

Objective 6 - Delivery. A typed HTTP interface serves thirteen endpoints, twelve of which return their documented payload against the live deployment in between 0.72 and 1.51 seconds; a four-view browser dashboard presents the forecast, the model's promotion history, its accuracy by horizon and its current drift status; and the same four grounded tools are exposed both to an in-application advisor and over the Model Context Protocol.

9.2 Contributions

The work makes five contributions.

A multi-city forecasting service where none existed. Twenty-two cities spanning every province, the capital territory, Gilgit-Baltistan and Azad Jammu & Kashmir receive a three-day, explained, uncertainty-quantified forecast. The exploratory analysis reported in Section 5.6 also produced a finding of independent interest: over the assembled record, Faisalabad's mean index exceeds Lahore's, which is an argument against the near-exclusive attention that Lahore receives.

A demonstration that the full MLOps lifecycle fits inside a free tier. Feature store, scheduled retraining, gated promotion, durable registry, rigorous evaluation, drift monitoring and public deployment were assembled entirely from free and negligible-cost services. Equally importantly, the twelve concrete failures that this constraint produced - a stalled executor freezing a blocking insert, a statistics profiler exhausting free memory, a deployment tool honouring an ignore file and silently omitting the model - are documented in Table 6.2 with their resolutions, and are the most transferable part of this work.

A worked instance of horizon-as-feature multi-horizon forecasting. The design collapses 220 potential artefacts into one, is shown to beat persistence across the whole forecast window, and is shown to carry a specific and disclosed cost at the shortest lead time.

Honest reporting as a delivered feature. The decaying error curve, the backtest fold that is much worse than the others, the refused model and the unflattering drift status all appear in the public interface rather than in an appendix. The claim being made is not that the system is very accurate but that the reader can see exactly how accurate it is.

A reasoned negative result on deep sequence modelling. A recurrent network was implemented, evaluated on identical terms, found to be 2.3% better at the single horizon it was built for, and declined on grounds of coverage, training cost and explainability - with the reasoning recorded rather than the result suppressed.

9.3 Limitations

Six limitations bound these claims, and are stated in the same spirit as the rest of the work.

The ground truth is a modelled atmospheric composition product rather than a reference-grade station reading, so the system demonstrates skill at forecasting that product. The model is trained on observed weather and deployed on forecast weather, so the reported errors are a mild under-estimate of operational error. The prediction intervals attain their nominal coverage by construction on the calibration window but

have not been verified prospectively. Unit testing covers the index computation and the feature layer but leaves the gate, the intervals, the drift computation and the API handlers to integration testing. The operational record is short: three training runs and no matured forecast-outcome pairs yet. And the interface's usability claims are argued rather than measured.

9.4 Future Work

9.4.1 Validation Against Ground Stations

The most valuable single extension is to obtain readings from reference-grade monitoring stations for as many of the 22 cities as publish them, and to measure the divergence between the modelled target and the measured one. This would either substantiate the system's output against physical measurement or quantify a bias that could then be corrected - a per-city calibration offset learned from the comparison would be a natural next step.

9.4.2 Prospective Interval Calibration

The forecast log already contains everything required to measure realised interval coverage: every issued point, its bounds, and eventually the observed value. Once enough pairs have matured, empirical coverage should be computed per horizon and compared with the nominal 80%. If coverage proves to degrade during unusual regimes, as the exchangeability assumption suggests it might, the natural remedy is adaptive conformal prediction, in which the quantile is adjusted online in response to recent coverage error.

9.4.3 Closing the Loop Between Drift and Retraining

At present drift is measured and retraining is scheduled, but the two are independent: a significant drift reading does not trigger anything. A natural extension is to make drift an input to the schedule - retraining more often, or on a shorter and more recent window, when the index or particulate distributions shift sharply - and to make a sustained rise in realised error raise an alert rather than merely a table row.

9.4.4 Probabilistic and Quantile Modelling

The current interval is a post-hoc residual construction around a point forecast. Training the model directly under a quantile loss, or fitting a small ensemble at several quantiles, would produce intervals that adapt to the input rather than depending only on the horizon - so that a forecast issued during a volatile episode would be visibly less certain than one issued during a settled period.

9.4.5 Pollutant-Level and Source-Aware Forecasting

Forecasting the constituent pollutants and then composing the index, rather than forecasting the index directly, would make the forecast more informative - a user could be told that the problem is fine particulate matter specifically - and would allow the gaseous sub-indices to be computed properly once the molar-mass conversions are implemented. Incorporating exogenous signals such as satellite fire-detection data during the crop-residue burning season would address one of the largest sources of unmodelled variance.

9.4.6 Broader Coverage and Finer Resolution

The architecture is not specific to Pakistan or to 22 cities: adding a city is a single line in the configuration module, and the global model requires no retraining per location. Extending to the wider South Asian airshed, which shares the same seasonal mechanism, is a natural direction. Finer spatial resolution would require a different data source, since the present provider's granularity is the ceiling on what the system can offer.

9.4.7 Strengthening the Test Suite and the Interface

The gate, the interval construction and the drift computation should be brought under unit test, so that a regression in the system's governance is caught before deployment rather than in production. On the interface side, a small user study would establish whether the explanation is in fact understood, and mobile notifications would convert the hazard alert from something a user must visit the site to see into something that reaches them.

9.5 Summary of Future Work

The system as delivered forecasts, retrains, gates, explains and monitors itself, and does so publicly and for free. The work that would most improve it is not more modelling but more measurement: validating the target against physical stations, verifying interval coverage prospectively, and letting the drift and error signals it already computes feed back into the schedule that keeps it current. That progression - from a system that watches itself to a system that responds to what it sees - is the natural continuation of this thesis.

Chapter 10

Abbreviations and Glossary

10.1 Abbreviations

The abbreviations and acronyms used throughout this thesis are collected in Table 10.1, listed alphabetically with their full forms. They span the air-quality, statistical, machine-learning and software concepts on which the project draws.

Table 10.1: List of abbreviations.

Abbreviation	Meaning
AJK	Azad Jammu & Kashmir
API	Application Programming Interface
AQI	Air Quality Index
ARIMA	Autoregressive Integrated Moving Average
CAMS	Copernicus Atmosphere Monitoring Service
CI/CD	Continuous Integration and Continuous Deployment
CO	Carbon Monoxide
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSS	Cascading Style Sheets
EPA	(United States) Environmental Protection Agency
ERA5	The fifth-generation ECMWF atmospheric reanalysis
FR	Functional Requirement
FTI	Feature-Training-Inference (the pipeline architecture pattern)
GB	Gilgit-Baltistan
GPU	Graphics Processing Unit
HTTP(S)	HyperText Transfer Protocol (Secure)
HUDI	Hadoop Upserts, Deletes and Incrementals (a time-travel table format)
ICT	Islamabad Capital Territory
JSON	JavaScript Object Notation
LLM	Large Language Model
LSTM	Long Short-Term Memory (a recurrent neural network architecture)
MAE	Mean Absolute Error
MCP	Model Context Protocol
ML	Machine Learning
MLOps	Machine-Learning Operations
NFR	Non-Functional Requirement

Abbreviation	Meaning
NO_2	Nitrogen Dioxide
O_3	Ozone
OpenMP	Open Multi-Processing (the shared-memory parallelism runtime)
$PM_{2.5}$	Particulate Matter of diameter 2.5 micrometres or less
PM_{10}	Particulate Matter of diameter 10 micrometres or less
PSI	Population Stability Index
R^2	Coefficient of Determination
RMSE	Root Mean Squared Error
SDLC	Software Development Life Cycle
SHAP	SHapley Additive exPlanations
SO_2	Sulphur Dioxide
SPA	Single-Page Application
TC	Test Case
UC	Use Case
UI	User Interface
URL	Uniform Resource Locator
UTC	Coordinated Universal Time
WHO	World Health Organization
XGBoost	Extreme Gradient Boosting

10.2 Glossary

Air Quality Index

A dimensionless public-health index computed from pollutant concentrations by piecewise-linear interpolation, reported as the maximum of the per-pollutant sub-indices and divided into six named health categories.

Anchor state

The observed pollution conditions at the moment a forecast is issued - the current index, its recent means and volatility, and the current particulate concentrations - supplied to the model as the state from which the forecast departs.

Backfill

The one-off pipeline run that loads the full multi-year history into the feature store, in chunks, resumably, so that a model can be trained on more than the few days the hourly pipeline has accumulated.

Baseline (persistence)

The naive forecast that the value h hours ahead equals the value now. Scored in every training run as the reference against which any claim of forecasting skill must be made.

Challenger

The best fitted candidate produced by a training run, which becomes the champion only if it improves on the incumbent's validation error.

Champion

The model currently in service. Defined operationally as the registered version with the lowest recorded validation RMSE.

Champion-challenger gate

The rule that a newly trained model may replace the model in service only on a measured improvement, so that an unattended retrain cannot degrade the system.

Chronological split

Dividing training from validation by time rather than at random, so that the model is always evaluated on a period it has not seen.

Conformal prediction

A distribution-free framework for attaching prediction intervals to an arbitrary regressor; in its split form, the interval is built from the empirical quantiles of residuals on a held-out calibration set.

Cyclical encoding

Representing a periodic quantity such as the hour of the day by its sine and cosine, so that the last value of the cycle is adjacent to the first rather than maximally distant from it.

Direct multi-horizon forecasting

Predicting the value at $t+h$ directly, rather than applying a one-step model repeatedly to its own output, thereby avoiding the accumulation of error across steps.

Drift

A change over time in the distribution of a model's inputs, or in the relationship between inputs and target. For a seasonal system, recurring drift is the normal condition rather than a fault.

Ephemeral runner

A scheduled compute instance that is created for one job and destroyed when it ends, so that nothing written to its local disk survives.

Event time

The column recording when an observation actually occurred, as distinct from when it was written; used by the feature store to order and time-travel over rows.

Feature group

The versioned, schema-defined table in the feature store into which the feature pipeline writes and from which the training pipeline reads.

Feature store

A managed repository of engineered features with primary keys and event time, written by feature pipelines and read by both training and inference, which eliminates training-serving skew by construction.

Feature-Training-Inference (FTI)

The architectural pattern in which a machine-learning system is expressed as three pipelines that share no code path and communicate only through a feature store and a model registry.

Global model

A single model trained across all locations and all horizons, conditioned on location and horizon as features, rather than one model per location or per horizon.

Horizon

The forecast lead time in hours. In this system it is an input feature, which is what allows one model to serve every lead time.

Idempotent upsert

A write that inserts a row if its key is new and replaces it if the key already exists, so that repeating the write has no additional effect and re-running a pipeline is always safe.

Leaderboard

The persistent record of every training run, its candidates, its metrics and its promotion decision, serving as the audit trail of the system's model history.

Model registry

The durable, versioned store of trained model artefacts tagged with their metrics, from which the inference pipeline retrieves the champion; the mechanism by which a model outlives the machine that trained it.

Population Stability Index

A non-parametric measure of how far a current distribution has moved from a reference distribution, computed over shared bins; conventionally read as stable below 0.10, moderate to 0.25, and significant above.

Prediction interval

A range around a point forecast expected to contain the true value with a stated probability; here the tenth to ninetieth residual percentiles for the relevant horizon, giving a nominal 80% interval.

Serverless

An architecture in which no long-running server is provisioned; compute is scheduled, short-lived and disposable, and all durable state lives in managed services.

Shapley value

The unique attribution of a prediction among its input features that satisfies local accuracy, missingness and consistency; computable exactly and in polynomial time for tree ensembles.

Target leakage

The presence in a feature of information that would not have been available at prediction time, which inflates measured accuracy and destroys real-world performance.

Target-time features

The inputs describing the hour being forecast - forecast weather, deterministic calendar and fixed location - as distinct from the anchor state describing the moment the forecast is issued.

Training-serving skew

The failure mode in which features are computed differently for training and for inference, so that a model performs worse in production than in evaluation for reasons unrelated to the data.

Walk-forward backtest

Rolling validation in which the training window expands and the test window moves forward with it, so that every evaluation is made on the future of its own training data.

What-if simulation

Re-predicting under deliberately altered inputs to observe how the model responds; a statement about the model, not a causal claim about the world.

Bibliography

[1] World Health Organization, "WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide," World Health Organization, Geneva, 2021. [Online]. Available: https://www.who.int/publications/i/item/9789240034228

[2] Health Effects Institute, "State of Global Air 2024: A Special Report on Global Exposure to Air Pollution and Its Health Impacts," Health Effects Institute, Boston, MA, 2024. [Online]. Available: https://www.stateofglobalair.org

[3] IQAir, "World Air Quality Report: Region and City PM2.5 Ranking," IQAir, annual report series, 2019-2024. [Online]. Available: https://www.iqair.com/world-air-quality-report

[4] World Bank, "Cleaning Pakistan's Air: Policy Options to Address the Cost of Outdoor Air Pollution," World Bank, Washington, DC, 2014.

[5] United States Environmental Protection Agency, "Technical Assistance Document for the Reporting of Daily Air Quality - the Air Quality Index (AQI)," Office of Air Quality Planning and Standards, Research Triangle Park, NC, 2024. [Online]. Available: https://www.airnow.gov/publications/air-quality-index/

[6] Copernicus Atmosphere Monitoring Service, "CAMS Global Atmospheric Composition Forecasts," European Centre for Medium-Range Weather Forecasts, 2024. [Online]. Available: https://atmosphere.copernicus.eu

[7] H. Hersbach *et al.*, "The ERA5 Global Reanalysis," *Quarterly Journal of the Royal Meteorological Society*, vol. 146, no. 730, pp. 1999-2049, 2020.

[8] Open-Meteo, "Open-Meteo Free Weather and Air Quality API," open-source project, 2022-2026. [Online]. Available: https://open-meteo.com

[9] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[10] L. J. Tashman, "Out-of-Sample Tests of Forecasting Accuracy: An Analysis and Review," *International Journal of Forecasting*, vol. 16, no. 4, pp. 437-450, 2000.

[11] C. Bergmeir and J. M. Benítez, "On the Use of Cross-Validation for Time Series Predictor Evaluation," *Information Sciences*, vol. 191, pp. 192-213, 2012.

[12] S. Ben Taieb, G. Bontempi, A. F. Atiya, and A. Sorjamaa, "A Review and Comparison of Strategies for Multi-Step Ahead Time Series Forecasting Based on the NN5 Forecasting Competition," *Expert Systems with Applications*, vol. 39, no. 8, pp. 7067-7083, 2012.

[13] A. E. Hoerl and R. W. Kennard, "Ridge Regression: Biased Estimation for Nonorthogonal Problems," *Technometrics*, vol. 12, no. 1, pp. 55-67, 1970.

[14] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.

[15] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189-1232, 2001.

[16] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785-794.

[17] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.

[18] X. Li, L. Peng, X. Yao, S. Cui, Y. Hu, C. You, and T. Chi, "Long Short-Term Memory Neural Network for Air Pollutant Concentration Predictions: Method Development and Evaluation," *Environmental Pollution*, vol. 231, pp. 997-1004, 2017.

[19] L. Yu, L. Wang, Y. Chen, and L. Zhang, "RAQ - A Random Forest Approach for Predicting Air Quality in Urban Sensing Systems," *Sensors*, vol. 16, no. 1, article 86, 2016.

[20] B. Pan, "Application of XGBoost Algorithm in Hourly PM2.5 Concentration Prediction," *IOP Conference Series: Earth and Environmental Science*, vol. 113, article 012127, 2018.

[21] R. Shwartz-Ziv and A. Armon, "Tabular Data: Deep Learning Is Not All You Need," *Information Fusion*, vol. 81, pp. 84-90, 2022.

[22] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?" in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[23] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY: Springer, 2005.

[24] H. Papadopoulos, K. Proedrou, V. Vovk, and A. Gammerman, "Inductive Confidence Machines for Regression," in *Proc. European Conference on Machine Learning (ECML)*, 2002, pp. 345-356.

[25] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman, "Distribution-Free Predictive Inference for Regression," *Journal of the American Statistical Association*, vol. 113, no. 523, pp. 1094-1111, 2018.

[26] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 1135-1144.

[27] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4765-4774.

[28] S. M. Lundberg *et al.*, "From Local Explanations to Global Understanding with Explainable AI for Trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56-67, 2020.

[29] D. Sculley *et al.*, "Hidden Technical Debt in Machine Learning Systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 2503-2511.

[30] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction," in *Proc. IEEE International Conference on Big Data*, 2017, pp. 1123-1132.

[31] J. Hermann and M. Del Balso, "Meet Michelangelo: Uber's Machine Learning Platform," Uber Engineering, technical blog, 2017. [Online]. Available: https://www.uber.com/blog/michelangelo-machine-learning-platform/

[32] J. Dowling, "From MLOps to ML Systems with Feature/Training/Inference Pipelines," Hopsworks, technical article, 2023. [Online]. Available: https://www.hopsworks.ai/post/mlops-to-ml-systems-with-fti-pipelines

[33] Hopsworks AB, "Hopsworks Feature Store and Model Registry Documentation," 2024-2026. [Online]. Available: https://docs.hopsworks.ai

[34] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A Survey on Concept Drift Adaptation," *ACM Computing Surveys*, vol. 46, no. 4, article 44, 2014.

[35] N. Siddiqi, *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Hoboken, NJ: John Wiley & Sons, 2006.

[36] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[37] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conference (SciPy)*, 2010, pp. 56-61.

[38] M. Abadi *et al.*, "TensorFlow: A System for Large-Scale Machine Learning," in *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265-283.

[39] S. Ramírez, "FastAPI: A Modern, Fast Web Framework for Building APIs with Python," open-source framework, 2018-2026. [Online]. Available: https://fastapi.tiangolo.com

[40] Meta Open Source, "React: The Library for Web and Native User Interfaces," 2013-2026. [Online]. Available: https://react.dev

[41] E. You *et al.*, "Vite: Next Generation Frontend Tooling," open-source project, 2020-2026. [Online]. Available: https://vitejs.dev

[42] V. Agafonkin, "Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps," 2011-2026. [Online]. Available: https://leafletjs.com

[43] GitHub, "GitHub Actions Documentation," 2019-2026. [Online]. Available: https://docs.github.com/en/actions

[44] Docker Inc., "Docker Documentation," 2013-2026. [Online]. Available: https://docs.docker.com

[45] H. Krekel *et al.*, "pytest: Helps You Write Better Programs," open-source testing framework, 2004-2026. [Online]. Available: https://docs.pytest.org

[46] Anthropic, "Introducing the Model Context Protocol," Anthropic, technical announcement and specification, 2024. [Online]. Available: https://modelcontextprotocol.io